UNIVERSITÉ DE LORRAINE

École doctorale IAEM Lorraine

Secure Electronic Voting: Postal Voting and Probabilistic Properties

THÈSE

présentée et soutenue publiquement le 19 novembre 2026

pour l'obtention du

Doctorat de l'Université de Lorraine

(mention informatique)

par

Léo Louistisserand

Composition du jury

Rapporteurs :	David Baelde	Professeur, ENS Rennes, IRISA, France
	Pascal Lafourcade	Professeur, Université Clermont Auvergne, LIMOS, France
Examinateurs :	Caroline Fontaine	Directrice de recherche, CNRS, LMF, France
	Sylvain Ruhault	ANSSI, France
	Sophie Tourret	Chargée de recherche, INRIA, LORIA, France
Encadrants :	Véronique Cortier	Directrice de recherche, CNRS, LORIA, France
	Pierrick Gaudry	Directeur de recherche, CNRS, LORIA, France

Laboratoire Lorrain de Recherche en Informatique et ses Applications — UMR 7503

Mis en page avec la classe thesul.

Contents

Introduction **1**

Partie I Background **9**

Chapter 1 Voting background **11**

1.1 Electronic voting setting 11

1.2 Desirable properties 12

1.2.1 Vote secrecy 12

1.2.2 Result integrity 13

1.3 Cryptographic primitives 13

1.3.1 Hiding a vote through encryption 13

1.3.2 Authenticating ballots 14

1.3.3 Computing the result 14

1.4 Toy protocol examples 15

1.5 Examples of attacks 16

Chapter 2 Formal methods background **19**

2.1 Syntax 19

2.2 Semantics 22

Partie II Analysing and designing postal voting protocols

Chapter 3 Analysis of existing postal voting protocols **29**

3.1 Overview of existing postal voting protocols 29
3.1.1 STROBE 29
3.1.2 RemoteVote and SAFE Vote 30
3.1.3 Devillez et al.'s protocol 32
3.1.4 Other schemes 32
3.2 Generic attack scenarios 32
3.2.1 Threat model 33
3.2.2 Privacy 33
3.2.3 Verifiability 34
3.3 Instantiation of the attacks on existing systems 34
3.3.1 STROBE 35
3.3.2 RemoteVote 35
3.3.3 SAFE Vote 36
3.3.4 Devillez et al.'s protocol 36

Chapter 4 Presentation of the Vote&Check protocol **39**

4.1 High level description 39
4.1.1 Participants 39
4.1.2 Protocol phases 40
4.2 Detailed description 40
4.2.1 Channels and authentication 40
4.2.2 Setup phase 41
4.2.3 Voting phase 43
4.2.4 Tally phase 43
4.2.5 Verification phase 43
4.3 Security claims 44
4.3.1 Privacy 44
4.3.2 Verifiability 44
4.4 Discussion 45

Chapter 5 Security analysis of Vote&Check **47**
5.1 The verifiability framework . . . 47
5.1.1 Processes . . . 47
5.1.2 Properties . . . 48
5.2 Model . . . 49
5.2.1 Agents . . . 49
5.2.2 Communication between agents . . . 49
5.2.3 Modeling corruption . . . 50
5.2.4 Control flow . . . 51
5.3 Privacy . . . 51
5.4 Verifiability . . . 53
5.5 Increasing ProVerif precision . . . 55
5.5.1 Protocol . . . 56
5.5.2 Queries . . . 56
5.5.3 Soundness result . . . 58
5.6 Results . . . 59
5.7 Conclusion . . . 59

Partie III Properly proving probabilistic properties

Chapter 6 A framework for probabilistic protocols **63**
6.1 Syntax . . . 63
6.1.1 Messages . . . 63
6.1.2 Process algebra . . . 63
6.2 A first semantics . . . 65
6.3 Refining CCK model . . . 68

Chapter 7 A sufficient condition for probabilistic security **73**
7.1 From probabilistic to deterministic . . . 73
7.2 Overview of the proof . . . 73
7.3 Arsenal . . . 75
7.4 Proof . . . 78

7.5 Checking the traceable hypothesis 79
7.5.1 Mini-ProVerif semantics 80
7.5.2 Checking traceability 82
7.6 Conclusion 83

Chapter 8 Case study analysis **85**
8.1 Presentation of the protocols 85
8.1.1 Modeling probabilistic voting protocols 85
8.1.2 STROBE 85
8.1.3 BeleniosCaI 89
8.2 Checking traceability 91
8.3 Towards deterministic security 91
8.4 Perspectives 93
8.4.1 Modeling short strings 93
8.4.2 Different flavours of deterministic security 94
8.4.3 Limitation 95

Conclusion **97**

Bibliography **99**

Introduction

In ancient Greece, every Athenian citizen took part in political, legislative, and judicial decisions. The assembly of citizens, called Ecclesia, gathered several times a year to vote on laws and elect the magistrates, who were in charge of the civil and military administration. After debating, they voted by a show of hands to adopt or reject a proposal by a simple majority.

One specific procedure of the Athenian democracy was called *ostracism*. It allowed citizens to expel one of their own, for example, for treachery or for becoming too powerful. In this case, the decision procedure differed. If an ostracism process were proposed, each citizen could use a piece of ceramic to write someone's name, or have a scribe do so. If at least six thousand *ostraka* were cast against someone, this person had to leave the city for a duration of up to ten years. Ostracism was used only during the fifth century BC. During this period, at least a dozen people were banned in this way.

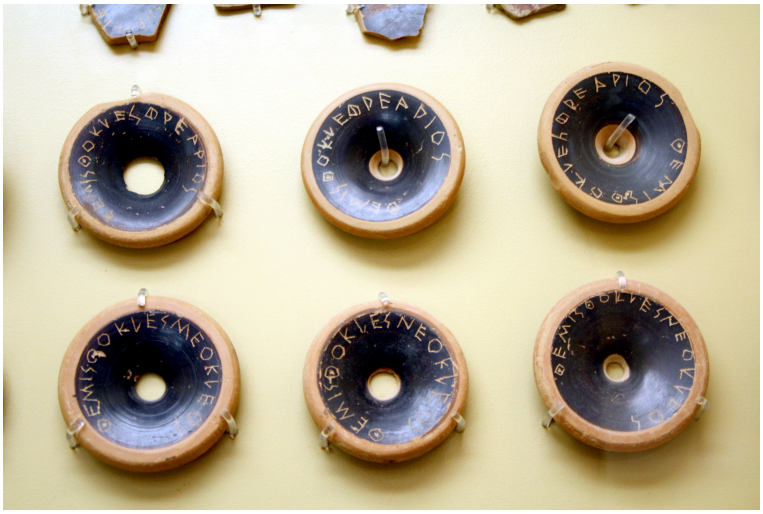


Figure 1: Ostraka

The figure 1 shows some ostraka found during archaeological excavations in the agora. As handwriting examinations suggest that whole piles of these objects have been written by a set of a few different hands, there are suspicions that it consists of a very ancient case of electoral fraud.

Electronic voting nowadays

If the concept of voting has remained more or less the same since Antiquity, its practical deployment has evolved significantly. Hence, the threats against the integrity of the election are no longer limited to a small group of people forging illegitimate voting material by hand. In

particular, the introduction of electronic voting enables local fraud to affect the entire system. We present several examples of countries that use electronic voting in their political elections.

Estonia played a pioneering role by introducing internet voting for parliamentary elections in 2005. Interest in this voting method has continued to grow since then, and more than half of the ballots were cast electronically in 2023 [Tea23]. The system originally used, heavily criticised for its lack of operational security [SFD+14], have been replaced in 2015 by IVXV [HMVW17]. This new system introduces auditability features and is claimed to provide end-to-end verifiability. However, IVXV remains vulnerable [SHR23].

In Switzerland, the Swiss Post internet voting system is used in certain places by all citizens, and only for people living abroad in the rest of the country. They took over the original protocol from Scytl in 2019 and made significant efforts to improve it [EHM23]. This includes increasing transparency, publishing the source code of the various components, holding public penetration testing, and, most importantly, requiring independent evaluation. These programs enabled the detection and correction of some flaws in the protocol, such as the one revealed in [CDG22].

In France, internet voting during political elections is reserved for French people living abroad. For the 2022 legislative election, the system included a third party responsible for a verification procedure [CGGR23]. The deployed system, however, was subject to multiple vulnerabilities [DH23] in both the protocol and the implementation. Discovering these attacks required reversing the protocol, since the specification was not publicly available.

It is important to note that the existence of flaws in systems is not evidence for an attack, but rather the window for a potential attack to remain undetected.

Properties

To make the evaluation of a voting system's security more objective, one can consider several properties it should fulfil. The main properties can be partitioned into two categories: privacy and verifiability. Intuitively, the former are related to vote secrecy, while the latter are related to the integrity of the election result.

Vote privacy means that each voter's vote remains secret. A voting protocol respects ballot secrecy if it does not leak any information about votes other than the election result. A stronger flavour of privacy is called *receipt-freeness* [BT94]. Intuitively, it means that voters do not receive information that could reveal how they voted, as long as they follow the protocol correctly. The main goal of receipt-freeness is to prevent most kinds of vote selling. Demos [KZZ15] is an example of a protocol achieving receipt-freeness. The *coercion resistance* [JCJ05] property is even more demanding. It requires that even dishonest voters cannot prove to anyone else how they voted. However, it is stronger than receipt-freeness since this has to hold even if the voter deviates from the protocol, to account for voters that are observed while voting, for example. Coercion resistance is a strong property and is out of reach of most deployed protocols.

The other main class of properties consists of those related to the election's integrity. Intuitively, the result is correct if the ballot box contains only ballots from eligible voters, at most one per voter, if ballots from honest voters are recorded properly, and if the tallied result corresponds to the contents of the ballot box. We say that a protocol respects the *eligibility* condition if the ballot box can contain only ballots from eligible voters, with a limit of one per voter. We say that a protocol respects the *tallied-as-recorded* property if the announced result always corresponds to the content of the ballot box. Finally, we say that a protocol respects the *cast-as-intended* and *recorded-as-cast* properties respectively, if every ballot indeed contains the vote of the voter that casts it, and if the ballot is not modified between its generation and its recording on the bulletin board. These properties seem to ensure the integrity of the vote.

However, there still can be blind spots. For example, the so-called *clash attack* [KTV12] exploits the fact that two different voters may perform the verification process for the same ballot, and both of them will be convinced that their vote has been counted. Therefore, the stronger notion of *end-to-end (E2E) verifiability* has been introduced. First formalised in [CGGI14], it partitions the voters into three categories: honest voters who perform the verification, honest voters who do not and dishonest voters. Then, a voting protocol is said to be verifiable if at the end of the election, the result always contains all the votes from the honest verifier voters, any number of votes from the honest non-verifier voters and at most one vote per dishonest voter. A stronger notion of verifiability is *accountability* [KTV10a]. The previous verifiability properties require that if a compromised party cheats and deviates from the protocol, at least one honest party will detect the deviation. We say that a protocol is accountable if, in this case, the other party can convince any honest party that the fraud indeed occurred and who the culprit is.

Use of cryptography

One common way to achieve those properties is cryptography. Every vote is hidden from the attacker using encryption and sent to the voting server. The election result is computed from the encrypted ballots using verifiable decryption. Two main paradigms exist to avoid breaking vote privacy by doing so.

Homomorphic encryption allows decryption only of the product of the encrypted votes, which, in this case, yields the sum of the decrypted votes. Thus, individual votes remain secret as they are not decrypted. The other main option is the use of *mixing network*. This cryptographic primitive takes as input a list of ciphertexts and outputs either the associated plaintexts in a shuffled order [Cha81] or a list of rerandomised equivalent ciphertexts [PIK94]. Both these primitives can also provide a proof of correct shuffle, i.e., that the output list indeed corresponds to the specification, for example [KMST16] for the decryption mixnet or [HLKD17] for re-encryption mixnet.

More diverse options coexist to ensure the cast-as-intended property, especially when the voter's device can be corrupt. The *Benaloh challenge* [Ben07] belongs to the audit-or-cast family. After receiving their ballot from their device, the voter decides whether to cast it or audit it. If they choose to audit, the device must reveal the randomness used to compute the encryption, which is possible only if this encryption was performed honestly. Then, the user can get a new ballot on their device and cast or audit it one more time. The reason the voter cannot simply cast a ballot after auditing it is that doing so would violate receipt-freeness. The probability that a voter detects misbehaviour on their device is not overwhelming but sufficient to make a large-scale attack unrealistic. The proper strategy for the voter to maximise their chances of detecting an attack is counterintuitive [Jam23], and the overall usability of this method is quite low [MKRV18]. The BeleniosCaI protocol [CDGG23] is inspired by the Benaloh challenge, but voters can audit their ballots after casting them. When a voting device returns an encrypted vote, it also returns two commitments on control values. If both those values are correct, then the vote is also correctly encrypted. After sending their ballot to the server, once it is recorded on the bulletin board, the voter can audit one of the two control values. As in the Benaloh challenge, the voting device must reveal the randomness used to compute the value being audited. If the voter randomly selects their value to audit, they will detect any attempt of the voting device to lie on the encryption with probability at least $\frac{1}{2}$. The usability of BeleniosCaI seems good, as well as the use of the voters as a source of randomness [CGGL24]. A different paradigm to ensure cast-as-intended against a malicious voting device is the *return codes*. It is used, for example, in CHVote [HKLD17] or by SwissPost [Pos24b]. It assumes an

additional agent responsible for creating the codes and sending them to the voters. Before the election, each voter receives a sheet with one return code per voting option. After they vote, they receive a code from the voting server and compare it to the one corresponding to the option they chose. There are different ways to implement this technique in practice; in all cases, the server must correctly record the vote to return the appropriate response code without learning the vote itself.

Postal voting

All these solutions work well for remote electronic voting. However, this is no longer the case with postal voting, which presents several difficulties. First of all, communication via postal channels takes much longer than electronic communication. This makes a protocol composed of several rounds unrealistic. If the voter does not have an electronic device, it is impossible for them to compute an encryption, a signature, a hash, etc. Moreover, the size of the nonces and of the messages the voter can manipulate is reduced. For example, in the SwissPost system, the voter must enter their Start Voting Key, a 24-character string. Since it is already considered a long string to copy, it provides an upper bound on the size of the data the voter can manipulate. Though, postal voting, as it is usually implemented, offers very few security guarantees. Usually, voters simply send their plaintext vote, hidden in an envelope, along with a copy of their ID or electoral card. Then, the polling station checks that this electoral card corresponds to one of their voters and simply puts the sealed envelope in a physical ballot box. This ballot box is eventually shuffled and tallied as for on-site voting. Several protocols have been designed to make postal voting more secure. Verifiable postal voting systems have a common structure. Voters receive some material by post, issued by one or more authorities (among them, the printer). Voters then select their candidate and return their selection, possibly retaining part of the voting material. At the end of the election, voters typically have access to additional data, for example, on a public bulletin board, and can perform checks to verify that their vote has been properly counted. To provide privacy against a dishonest polling station, as well as some sort of verifiability, Benaloh proposed the STROBE protocol [Ben21] in 2021. In STROBE, a printer generates two valid ballots per voter and sends them. Each voter picks one of these two ballots to vote; the other will be used for the audit. The polling station publishes codes corresponding to the ballot and the chosen option, allowing the voter to verify that their vote has been recorded properly. Finally, the printer reveals the randomness used to form the unused ballot, allowing the voter to audit it. The STROBE protocol has been extended to RemoteVote [CRH22], where the two ballots per voter are merged onto a single sheet to improve usability, and to SAFE Vote, which relies on a scratch-off surface. Those two protocols are quite close to STROBE in terms of cryptography. More recently, a different approach by Devillez et al. [DPP24] relies on return codes to guarantee individual verifiability, and on hiding commitments to guarantee vote privacy. Interestingly, this analysis is the only one that comes with a comprehensive security analysis.

The attack surface of a voting protocol is generally broad, and vulnerabilities may be diverse. For instance, [KS19] identifies many threats to the Swiss postal voting process. This protocol is standard in postal voting practice and uses the double-envelope mechanism. Using formal methods to assert the security of a protocol is good practice.

Formal methods

The use of formal methods allows one to prove that a given protocol satisfies a security property without having to sequentially and exhaustively rule out every imaginable attack.

Two main models exist in this field: the symbolic and the computational model. In Switzerland, both these types of proof are mandatory for an electronic voting system [Cha23]. In this thesis, we will only work in the first one. It consists of a high-level abstraction of messages, cryptographic primitives, the network, the different agents, etc.

The attacker model usually considered is the Dolev-Yao attacker. First introduced by Needham and Schroeder in 1978 [NS78], it has been formalised by Dolev and Yao in 1983 [DY83]. This attacker controls the communication network, i.e., is able to read, copy, alter, block or inject any message. It is stronger than a passive attacker that could only learn the messages exchanged. However, it is limited by cryptography. All cryptographic primitives are idealised and considered unbreakable. For example, the attacker cannot reverse a hash function or decrypt a message without the relevant private key. It can still decrypt the message if it learns the key later in the execution. This attacker model captures a large class of attacks relying on logical errors. On the other hand, it misses some attacks that rely on cryptographic weaknesses, such as Logjam [ABD+18]. This attack exploited a logical flaw in TLS to downgrade the connection to an "export grade" Diffie-Hellman with a too-short key, then used cryptanalysis to exploit it. A method to find flaws due to misuse of cryptographic primitive have been proposed in [CCD+23]. However, it requires an ad-hoc construction rather than being systematic.

By being abstract and quite simple, the symbolic model leaves ample room for automation. Several tools exist to aid the analysis of security protocols. We are now going to present some of them.

The MaudeNPA tool [EMM06] represents the agents of a protocol, as well as the attacker, as state machines. The properties are verified through a backwards-narrowing search, starting from a state corresponding to the falsified property. If this search succeeds, this means that an attack has been found against the protocol. Otherwise, if the search space is finite, then the property is satisfied. The computation may also not terminate. MaudeNPA has been used to formalise industrial protocols like the Group Domain Of Interpretation Protocol [MSC03].

The Tamarin prover [BCDS25] is based on multiset rewriting. It considers the transition system formed by a multiset of facts representing the user's state, the attacker's knowledge, and the messages exchanged over the network. This allows to specify the protocol under study, as well as the security properties to fulfil. Then, Tamarin derives constraints from those specifications and calls a constraint solver to find a behaviour of the protocol that contradicts one of these properties. Tamarin is not fully automated but rather interactive: in case of non-termination, the user can guide the proof with lemmas. It has been used to analyse widely deployed protocols such as EMV [BSTP20], TLS [CHH+17], or OPC-UA [DPP+17].

The ProVerif tool relies on applied pi-calculus [AF01], which allows to specify protocols and equational theories. Applied pi-calculus only considers the arity of the functions, as well as a set of rewriting rules that represents their behaviour. Agents are modelled as elements of a simple process algebra. ProVerif has been used to analyse a wide range of protocols, including Signal [KBB17], communication protocols such as TLS [BBK17] and EDHOC [JKKR23], and voting protocols [CDC23]. It is based on Horn clauses and saturation and is fully automated. The resolution procedure is correct: when it declares a property true, then this property holds in the model. It is, however, not complete: sometimes the computation does not terminate, and sometimes its overapproximation leads to finding an attack that does not exist.

Other tools exist to analyse protocols against a computational attacker. It provides more

precise security guarantees at the cost of more complex modelling. Easycrypt [BDG+12] relies on program verification tools like relational Hoare logic. It has been used to prove the security of the standard post-quantum key encapsulation mechanism ML-KEM [AAOB+24], and postal voting protocols such as Helios [CSD+17] or Belenios [CDS+18]. With Cryptoverif [Bla07], the user describes the security property under study as a security game, as in pen-and-paper proofs. Then, they proceed by successive reductions and end up with a game that corresponds to the security hypothesis (e.g., IND-CCA2). More recently, the Computationally Complete Symbolic Attacker [BCL12] has been introduced to construct symbolic proofs valid in the computational model. The Squirrel prover [BDJ+24] mechanises proofs written in this logic. The user first specifies the protocol, then proves the security property using tactics. [BKS26] provides a proof of the FOO voting protocol using Squirrel.

Formally analysing a protocol means specifying the protocol, the properties and the attacker. It is easier to make a mistake without noticing it while doing so than to find it. For instance, the famous attack from [Low95] on the Needham-Schroeder protocol [NS78] was discovered almost twenty years after its release. Specifying a property is a difficult task, for example [Low97] defines five different notions of authentication. A flaw in the model can lead to missing an attack against the protocol. Even in a very simple protocol like Helios, flaws are still found several years after its publication [BBMP21].

Analysing a voting protocol is especially difficult. The desired properties are both equivalence properties for vote privacy and correspondence properties for verifiability. Moreover, the appropriate notion of equivalence, *observational equivalence*, is not suitable for algorithmic analysis. Instead, ProVerif considers a stronger notion, *diff-equivalence*, that implies the previous one. However, this overapproximation may also lead to unrealistic attacks. A blind spot of the symbolic model is the limited scope for primitives. [DLM24] shows that an incomplete modelling of a mixnet can lead to missing some attacks. Indeed, specifying all the Diffie-Hellman equations would lead to a non-terminating system, so the authors of [DGK+14] missed an attack by underapproximating their primitives. The same attack applies to the voting protocol [HS11]. Finally, the tools like Tamarin or ProVerif are not designed to handle probabilities. While neglecting improbable scenarios is arguably a valid model, this strategy does not work for small but not negligible probabilities. For example, in STROBE [Ben21], the voter picks a ballot to cast out of two, each outcome happening with a probability of $\frac{1}{2}$. The Benaloh challenge, or BeleniosCaI [CDGG23], also involves a probabilistic execution flow. Another source of non-neglectability is the use of short strings, generally to ensure usability. In the formal analysis of the SwissPost system [Pos24a], the possibility for the attacker to guess a return code is not considered. Even if this would only be possible with low probability, it is not impossible.

Contributions

Our first contribution concerns postal voting. Our goal was to design a simple and verifiable postal voting scheme and to prove its security. First, since most postal voting protocols share a common structure, we created a generic model for postal voting protocols. Then, we presented generic potential attacks on this model targeting desirable verifiability and confidentiality properties. We apply those attacks to protocols of the literature, including STROBE [Ben21], RemoteVote [CRH22], SAFE Vote [CRH22] and Devillez et al. [DPP24]. Some of those attacks work against all protocols and seem unavoidable. Thus, we introduce weaker properties that a postal voting system should satisfy: *weak end-to-end verifiability* and *privacy without complaints*. Compared to classic end-to-end verifiability, weak end-to-end verifiability tolerates the attacker modifying the vote of a voter who does not execute the verification procedure. Privacy

without complaints accounts for the fact that a voter whose vote has not been counted and who complains reveals information about their vote. Then, we design a postal voting protocol that meets our requirements: building the simplest possible protocol, using only basic cryptography and relying on few agents. Our protocol is tracker-based and inspired by Selene [RRI16]. The intuition behind the protocol is that each voter gets two pieces of secret information that will generate their tracker. Thus, they can compute it, but the authorities issuing these secrets cannot. The polling station knows both secrets as they need them to publish the tracker on the board, but they do not know who issued the ballot, as we assume the postal channel is anonymous. We analyse our protocol using the ProVerif tool. The automation enabled us to write proofs for all corruption scenarios across different users, ensuring both privacy and verifiability. The specificity of the postal context led us to introduce novel modeling, for example for postboxes, voters without smartphones, and anonymous complaints.

Our second contribution explores a way to analyse probabilistic protocols. The goal is to provide an accurate modelling of processes involving probabilistic branching in their control flow, like STROBE [Ben21] or BeleniosCaI [CDGG23]. [CCK23] recently introduced a formalism to account for non-negligible probabilities in symbolic models. Intuitively, the only syntactic difference with pi-calculus is the addition of $+_p$, a new binary operator on processes: $P +_p Q$ goes to P with probability p and to Q with probability $1 - p$. The CCK model no longer considers execution traces, as in the pi-calculus, but rather execution trees. We identify a subclass of protocols and a subclass of properties, that we call traceable protocols and k-simple properties, for which we can reason. We describe k-simple properties as the set of winning states for the adversary, for example, having flipped at least k votes without the affected voters complaining. Then, our goal is to bound the probability that the attacker reaches such a state with a function of k. Therefore, we introduce a finer semantics than that of [CCK23], which keeps track of the probabilistic choices that have been passed. This allows us to define the *deterministic security* property, which derives directly from the one we want to prove but does not involve probability. Intuitively, a protocol is deterministically secure if the attacker cannot cheat to win in one branch without being detected in another. We prove a theorem that reduces the analysis of a k-simple property to the analysis of the corresponding deterministic security property. We propose an algorithm that relies on the ProVerif tool to verify the traceable hypothesis for a given protocol and apply it to STROBE and BeleniosCaI. Finally, we present an ongoing work to also verify the deterministic security hypothesis using ProVerif.

Both those contributions are common work with Véronique Cortier, Alexandre Debant and Pierrick Gaudry.

Our third contribution is a joint work with Florian Moser, but we chose not to present it in this thesis. We designed a scheme for the electronic collection of signatures in the context of Swiss popular initiatives. The notion of privacy in the e-collecting context is very different from that in electronic voting, since revealing someone's participation in a collection directly leaks their opinion. Thus, the main idea is to hide a citizen's participation in a collection among a multitude of possible participations. We start the protocol by encrypting 0 for each voter and each collection. When someone wants to sign a collection, they encrypt a 1 and reencrypt their other ciphertexts. Some zero-knowledge proofs prevent citizens from signing collections or signing a collection more than once.

Structure of the thesis

This thesis is divided into three parts. Part I introduces the concepts that we use later in the thesis. Chapter 1 gives the main security properties expected from an electronic voting scheme,

as well as the most common cryptographic mechanisms to achieve them. Chapter 2 presents a simplified version of the ProVerif syntax and semantics. In Part II, we expose our contribution on postal voting protocols. Chapter 3 provides attacks against state-of-the-art protocols. We introduce our protocol proposal, Vote&Check, in Chapter 4, as well as its security analysis using the ProVerif tool in Chapter 5. Finally, we present our contribution to the analysis of probabilistic protocols in Part III. Chapter 6 recalls the protocol language from [CCK23] and restricts it for our use cases. Chapter 7 states and proves our main results. Chapter 8 presents examples of our method on state-of-the-art probabilistic protocols.

Part I

Background

Chapter 1

Voting background

In this chapter, we present the main concepts related to secure remote voting. We first detail the most common actors of a voting protocol, as well as its different phases. A voting scheme must satisfy some security requirements, represented by properties we list here. We give examples of cryptographic primitives used in practice to enforce these properties. Finally, we provide toy examples to illustrate canonical attacks on electronic voting protocols.

1.1 Electronic voting setting

We first present the different entities that usually take part to an electronic voting protocol. Each of these participants has a precise role, described in the specification, that they are supposed to follow meticulously. We usually distinguish different phases in the protocol.

The **Election organiser** is the authority in charge of the vote. During the Setup phase, they define the voters list, the different voting options, the election dates, etc. This information is published on the **Bulletin board**, a secure append-only board publicly accessible. Achieving these properties is a tough task in practice [HSB21]. For example, a bulletin board run by a malicious authority could send different views to different users. The **Registar** generates and sends their material to each voter. In the context of postal voting, the **Printer** typically generates the ballots. A **Verification authority** may also be involved in the computations, to ensure security properties.

During the Voting phase, the **Voters** may have to perform some verifications on their voting material. We sometimes distinguish the voter from the **Voting device**, to conduct a more precise security analysis. Then, they fill their ballot and send it to the **Voting server**. The server can perform some checks, like verifying that the voter is eligible, or that a signature or a zero-knowledge proof is correct. Depending on the protocol, the server publishes data related to the ballot on the bulletin board during the voting phase or after the election. In the context of postal voting, we rather speak of the **Ballot collector** instead of voting server.

The Tally phase starts once the election is over. The **Talliers** use the public information from the bulletin board, their private key and sometimes some other private information to compute the outcome. They publish the result on the bulletin board, possibly alongside proofs of correct decryption or other kind of audit data.

Finally, during the Audit phase, a set of independent **Auditors** may be required to performs some verification to ensure that the result is correct. To guarantee individual verification, voters may be required to check that their ballot is indeed published on the bulletin board.

1.2 Desirable properties

We split the properties of a voting protocol to achieve into secrecy properties and integrity properties. It is easy to build a fully secret protocol, as well as a protocol offering complete integrity. Nevertheless, combining both of them is still an open problem. For example, if we aim for a fully verifiable system, we can assume a public key infrastructure and have every voter publishing their plaintext vote on the bulletin board, with a signature. Anyone is then able to compute the result. Conversely, for a fully private voting protocol, we can have every voter sending their vote to the ballot collector via an anonymous channel, and the ballot collector privately computing the result. Links between vote secrecy and result integrity have been abundantly studied [CMFP$^+$10, CL18].

1.2.1 Vote secrecy

Intuitively, we say that a voting protocol achieves vote secrecy when an attacker cannot learn the votes of honest voters by any mean. There are plenty of notions of **vote privacy** in the literature. One challenge to define privacy is to take into account that the result of the election itself leaks information. In [KR05], there are two virtual settings where two specific voters vote respectively for v_0 and v_1 and for v_1 and v_0. The attacker has to find an execution for which they can decide which case they are observing. The system is said private if no such execution exists. In game-based definitions of ballot privacy [BY86, BCG$^+$15], the attacker generates two worlds by choosing a set of voters, a multiset of same cardinality of votes and attributing arbitrarily the votes to the voters in two different ways. Then, one of these two settings is chosen at random and the attacker has to guess in which setting they play. The system is said private if the probability to guess for the attacker is only neglectably higher than $\frac{1}{2}$. In entropy-based definitions [CHVW05, CMFP$^+$10], the information of the attacker is represented by random variables. For example, if the only information the attacker has is that the ballot box contains n_0 votes for v_0 and n_1 votes for v_1, then the random variable V denoting a voter's vote will associate to v_0 a likelihood of $\frac{n_0}{n_0+n_1}$ and to v_1 a likelihood of $\frac{n_1}{n_0+n_1}$. A voting system is said private if conditioning by the additional information the attacker can get during the election does not change the probability distribution of V.

A stronger secrecy notion than vote privacy is **coercion resistance** [KTV10b]. While for the former property, the attacker has to figure out how an honest voter voted, in the latter, we assume that the voter also misbehaves and tries to convince the attacker that they voted in a certain way. We want this kind of proof to be impossible to limit vote selling. One approach to guarantee coercion resistance is fake credential [JCJ05, CGY24]. Each voter will be provided a random bitstring as voting credential by the registrar. If an attacker tries to coerce them, they can generate any bitstring and provide it to the attacker. The attacker has no way to distinguish it from a valid one. During the tallying phase, all votes with invalid credential will be silently discarded. One other approach to avoid coercion is revoting [MPP24, LQT20]. Every voter is able to cast any number of votes, and only the last one will be counted in the tally. If a voter is forced by the attacker to vote in a certain manner, they can revote later if they want to change their vote. Of course, this require the voter to be free from coercion during the last minute of the election.

1.2.2 Result integrity

To ensure trust in the election, a voting system must guarantee that the result of the election really corresponds to the votes that have been cast. Here again, there are different manners to formalise this notion [CGK+16].

Individual verifiability. The first kind of verifiability lies at the voter level. During the voting and/or the audit phase, a voter can perform verifications to ensure that their vote has been properly counted. Individual verifiability is often separated in two subproperties. Usually, the voter will encrypt their vote before casting it, using a voting device for this purpose. We say that the vote is **cast-as-intended** if the ciphertext computed by the voting device is indeed an encryption of what the voter selected. Then, we say that the vote is **recorded-as-cast** if the ciphertext arrives in the ballot box without being altered or dropped.

Universal verifiability. We express other aspects of verifiability at a global level. Public information will often only include the list of encrypted votes. We say that the ballots are **tallied-as-recorded** if the result computed by the talliers indeed corresponds to the content of the ballot box. To avoid ballot stuffing, one needs to ensure that the ballots on the bulletin board are all legitimate. We say that the protocol respects **eligibility** if every recorded ballot comes from a distinct legitimate voter.

End-to-end verifiability. Slicing the result integrity into a set of simpler properties makes it more convenient to analyse on case studies. However, it also weakens the global claim since it is not always clear how these properties combine altogether. This gap can result in missing out-of-scope attacks such as clash attacks [KM17]. Hence, the end-to-end verifiability property aims to capture all the integrity properties at once. In [CDC23], a voting scheme achieves end-to-end verifiability if the results contains all the votes from honest voters that performed the verification, a subset of the votes from honest voters that have not performed verification, at most one vote per compromised voter and nothing else.

Accountability. Voting protocols often specify some tests to perform during the audit phase, that should succeed if and only if no irregularities happened during the election. However, it is less common to indicate what to do if one of more of these tests fail. The accountability property [KTV10a, DDG+24] aims to bridge this gap by designating who misbehaved. We say that a protocol satisfies accountability if the auditors can either provide a proof that everyone behaved honestly, or a proof of who cheated.

1.3 Cryptographic primitives

We mention some cryptographic primitives that are commonly used to guarantee the properties mentioned in Section 1.2.

1.3.1 Hiding a vote through encryption

A very convenient way to build a remote voting scheme is to use **public-key encryption**. The talliers generate a public key before the election, and the voter uses it to encrypt their vote before sending it to the server. The scheme should be probabilistic, to avoid the attacker retrieving the votes just by sequentially encrypting every possible option. The ElGamal encryption [Elg85] is

one the most widely used cryptosystems for electronic voting [Adi08, CGG19]. The ElGamal scheme allows **verifiable decryption**. More precisely, when decrypting a ciphertext, the holder of the private key can compute a proof that the plaintext they got is indeed the decryption of this ciphertext. One does not need the private key to check the proof. The verifiable decryption gives the tallied-as-recorded almost for free. ElGamal encryption also has the advantage of being **homomorphic**, ie the product of the encryption of two plaintexts is an encryption of the sum of these two plaintext. In the voting context, it allows the talliers to compute the result of the election by multiplying all encrypted votes and decrypting the sum, rather than decrypting each individual vote. This results in a privacy increase, since the plain votes remain secret. To avoid a dishonest tallier to illegitimately decrypt votes, the ElGamal scheme finally affords **threshold decryption**. Instead of generating a standard key pair, the talliers can run a multiparty computation to generate a public key and multiple key shares, each known to only one tallier. These key shares allow to perform partial decryption of ciphertexts, and a certain number k of partial decryptions (the threshold) can be combined to compute the plaintext. Then, to break the privacy of a voter by decrypting their ballot, an attacker needs at least k talliers to collude and illegally provide them their partial decryption. Threshold decryption is compatible with verifiable decryption.

The ElGamal scheme is not the only one that allows to build remote voting protocols. For example, the Paillier cryptosystem [Pai99, DJN03] offers the same good properties, based on different computational assumptions. In the following, we use the symbols pk and sk for the public and the private key of an asymmetric encryption scheme, and aenc and adec to represent the encryption and decryption algorithms. We write $\mathsf{aenc}(m, r, \mathsf{pk})$ for the encryption of the message m under the public key pk, using the randomness r. When clear from context, we may omit r or pk.

1.3.2 Authenticating ballots

During the tally phase, at most one vote per eligible voter should be counted. If there exists a public key infrastructure, the voters can sign their ballots with a **digital signature** scheme before sending them to the server. Then, the server will only accept a ballot if it has a valid signature corresponding to an eligible public key, and at most one ballot per key. It is impossible to compute a valid signature without the signing key, hence the eligibility property is granted.

The server should only accept ballots that contain valid votes. Even if the votes are encrypted, a voter can prove that their vote belongs to the set of allowed options using a **zero knowledge proof** (ZKP) [BS23]. Intuitively, a ZKP allows one to prove one knows a secret satisfying a given relation without revealing it. In our example, the voter can prove that they know a random seed and a valid voting option such that their ballot is an encryption of this option with this randomness and the tally public key. No knowledge is required to verify the proof besides the statement. This primitive can also be used by the registrar, to convince a voter that their credential is valid [JCJ05].

1.3.3 Computing the result

It is not always possible to compute the result by relying on homomorphic encryption and decrypting the sum of all encrypted votes. For example, if the voting scheme uses a rank-based counting like Condorcet [dC85] or Single Transferable Vote (STV), the result is not computed by simply summing the votes so summing the encryption would make no sense. Instead, the talliers, as well as anyone wanting to audit the outcome, need to perform complex operations on

the plain votes. To retrieve those plain votes without breaking vote privacy by giving the link between voters and ballots, the verification authority can run a **mixnet** protocol [TW10, BG12] before the talliers decrypt the votes. Intuitively, the mixnet reencrypts and shuffles all ballots, such that it is impossible for an observer to track a specific ballot. In the Swiss Post protocol, there are four control components that sequentially shuffle the ballots to break the link between the voters and the plaintexts. A mixnet is relatively complex, its analysis can be tedious [CFS25] and frequently leads to errors [DLM24, CDG25].

1.4 Toy protocol examples

We give here two examples of toy protocols to illustrate the security properties as well as some classical attacks. In both examples, we assume there is a public list of voters $V_1, \ldots, V_n$, and we note the different voting options $v_1, \ldots, v_k$. We also assume that there exists secure communication channels between each pair of participants. More precisely, we assume these channels to be private and authenticated, except for the channels from the voters to the ballot collector that we assume to be private and anonymous.

Example A: TrackVote. In our first protocol, the registrar initiates the protocol by generating a voting token a_i for each voter V_i. They send these tokens to their recipient using the secure channels. During the voting phase, each voter creates a ballot by simply joining their voting token a_i and their vote v_i. Then, they send this ballot to the voting server over an anonymous channel. When the voting server receives a ballot with a new voting token, they publish it on the bulletin board. During the tally phase, one can simply compute the result by summing all the plain votes on the board. During the audit phase, each voter can verify that their vote has been properly counted by looking for their token v_i on the board, and checking that is is associated to their vote.

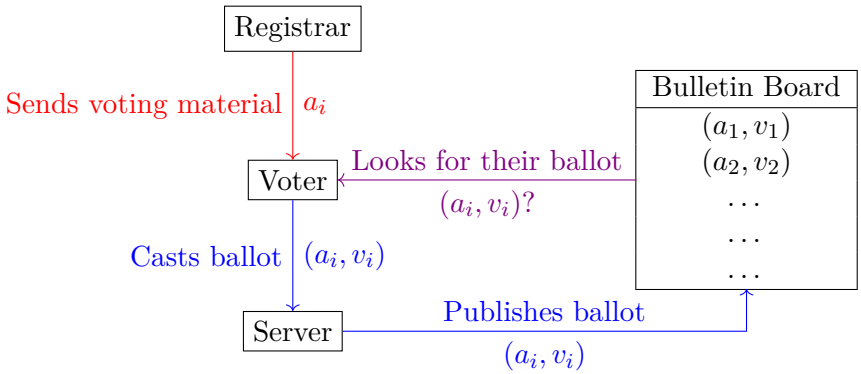


Figure 1.1: Overview of TrackVote. A different colour is assigned to each phase: red for the setup phase, blue for the voting phase, and violet for the verification phase.

Example B: Mini-Helios. For our second protocol inspired by [Adi08], we assume there exists a PKI: the list of voter is public, and each of them has a signing key sk_i (for the voter of index i). The association between voters and verification key is written on the bulletin board. During the setup phase, each voter checks that their public key is correctly registered on the

bulletin board. The talliers compute a shared key and publish the associated public key pk_T on the board. During the voting phase, the voters compute:

- an encryption of their vote v_i with a random seed r_i: $x_i = \mathsf{aenc}(v_i, r_i, \mathsf{pk}_T)$.
- a zero-knowledge proof π_i that they know r_i such that $x_i = \mathsf{aenc}(v_i, r_i, \mathsf{pk}_T)$
- a signature σ_i of x_i and π_i with the secret key sk_i.

They send their ballot (x_i, π_i, σ_i) to the voting server through an electronic channel. The server accepts all well-formed ballots and publishes them on the bulletin board. During the tally phase, the talliers sum the most recent ciphertexts corresponding to each verification key, and partially decrypt the sum. They publish their shares on the board, as well as proofs of correct partial decryption. Then, the voting server combines all the shares to compute the result. During the audit phase, each voter checks that their ballot is properly recorded on the board. The auditors verify all the signatures, zero-knoledge proofs and proofs of correct decryption, as well as the final computation of the result from the shares.

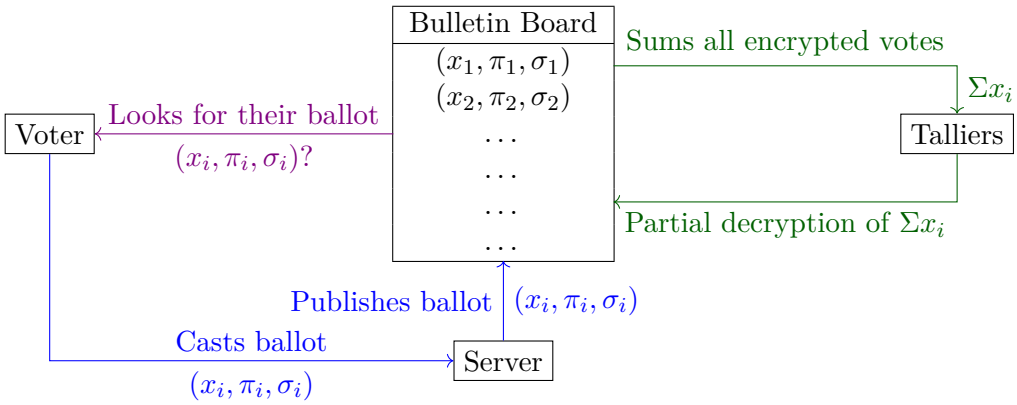


Figure 1.2: Overview of Mini-Helios. A different colour is assigned to each phase: blue for the voting phase, green for the tally phase, and violet for the verification phase.

1.5 Examples of attacks

In this section, we outline some common attacks in electronic voting and instantiate them on our toy examples.

Injecting a ballot In TrackVote, the ballots arriving to the voting server are not authenticated. Therefore, an attacker can alter the result of the election in support of v_1 by forging fake voting tokens $a'_1, \ldots, a'_l$ and sending illegitimate ballots $(a'_1, v_1), \ldots, (a'_l, v_1)$ to the voting server. Neither the server nor the auditors can distinguish forged tokens from real ones, so the attack will be undetected.

Droping/modifying a ballot In both examples, if the voting server is malicious, they can simply drop any ballot they receive. If the concerned voter does not perform the verification, or does not complain, this will not be detected. In TrackVote, the server can even modify the votes of the voters that do not verify. This attack works better in a context where the attacker can

know which voter conducted the verification during the previous elections, as they can choose their target more cleverly.

Replaying a ballot This class of attacks against vote secrecy was originally discovered against the Helios protocol [CS11]. We consider the first notion of vote secrecy that we mentioned in Section 1.2.1. The attacker has to distinguish between two executions of the protocol where two distinguished voters, Alice and Bob, vote respectively v_0 and v_1 in the first execution and v_1 and v_0 in the second. Alice and Bob are going to vote as expected. Their ballots $(x_{\mathsf{Alice}}, \pi_{\mathsf{Alice}}, \sigma_{\mathsf{Alice}})$ and $(x_{\mathsf{Bob}}, \pi_{\mathsf{Bob}}, \sigma_{\mathsf{Bob}})$ will be published on the bulletin board as soon as they arrive at the server, during the voting phase. Then, if a third voter Charlie is corrupt, he can send the same vote as Alice, signed with his own key. They cast $(x_{\mathsf{Alice}}, \pi_{\mathsf{Alice}}, \sigma_{\mathsf{Charlie}})$ and the server publishes it on the board. The talliers will decrypt the sum of the ballots and the result will either be $\{\!\{v_0, v_0, v_1\}\!\}$ or $\{\!\{v_0, v_1, v_1\}\!\}$, the duplicated vote revealing Alice's.

Clash attack This kind of attack was first described in [KTV12]. In TrackVote, it requires the registrar authority to be dishonest. Then, if they suspect several voters will cast the same vote, they can cancel all but one of their vote by providing them the same voting token. These voter will simply cast their ballot without noticing. The server will receive a lot of duplicate ballots, and only one of them will be recorded. During the audit phase, every voter will look at the same line on the bulletin board and think it belongs to them.

Chapter 2

Formal methods background

In this section, we present a grammar to analyse security protocols. Its syntax and semantics allow to model both the protocol and the security properties it must fulfil. The cryptographic operations are abstracted and represented by function symbols and an equational theory. The process algebra consists of a simplified version of the ProVerif language [Bla16, BCC22a], with a semantics capturing the ability of the attacker. Security properties are modeled by correspondence queries and injective queries [CDC23], and provide a high degree of expressivity.

2.1 Syntax

We consider a set of names $\mathcal{N} = \mathcal{N}_{\mathsf{pub}} \cup \mathcal{N}_{\mathsf{priv}}$, with $\mathcal{N}_{\mathsf{pub}}$ and $\mathcal{N}_{\mathsf{priv}}$ two infinite disjoint sets, and a set of variables $\mathcal{X} \cup \mathcal{X}_{\mathsf{fail}} \cup \mathcal{X}_{\mathbb{N}}$, with $\mathcal{X}, \mathcal{X}_{\mathsf{fail}}$ and $\mathcal{X}_{\mathbb{N}}$ three disjoint sets. We call the elements of $\mathcal{X}_{\mathsf{fail}}$ *may-fail variables*, and the element of $\mathcal{X}_{\mathbb{N}}$ *time variables*. We also consider functions symbols: a set of events $\mathcal{F}_{ev}$, a set of tables $\mathcal{T}bl$, a set of constructors $\mathcal{F}_c$ and a set of destructors $\mathcal{F}_d$, all of them being finite and disjoint. For each destructor $h \in \mathcal{F}_d$, we consider an ordered set of rewriting rules of the form $h(U_{i,1}, \ldots, U_{i,k}) \to U_i$, where $U_i, U_{i,1}, \ldots, U_{i,k}$ are *may-fail terms*. We consider each constructor and destructor either public or private. We assume the existence of two special constructors of arity 0: true and false.

Example 1. *Among other things, we use constructors and destructors to model cryptographic primitives.*

- *An ideal hash function is a one-way function, it cannot be inverted. Hence, we represent it using only a constructor* hash*, or arity 1.*
- *Asymmetric cryptography uses pairs of keys. We consider a function* pk *that computes the public key corresponding to a given private key. Asymmetric encryption takes as input a plaintext, a randomness and a public key, we model it by a constructor* aenc *of arity 3. Decryption can be performed using the destructor* adec*. The only associated rewriting rule is* $\mathsf{adec}(\mathsf{aenc}(m, r, \mathsf{pk}(\mathsf{sk})), \mathsf{sk}) \to m$*.*
- *An example of a destructor with several rewriting rules appears when modeling a disjunctive zero-knowledge proof. We consider for example a zero-knowledge proof that a ciphertext encrypts either a* 0 *or a* 1*, used in electronic voting for instance. We define a constructor* $\mathtt{ZKP}(x, r, \mathsf{pk})$*, and an associated destructor* $\mathtt{check_ZKP}(\pi, x, \mathsf{pk})$*. This destructor is defined by the rewriting rules* $\mathtt{check_ZKP}(\mathtt{ZKP}(\mathsf{aenc}(0, r, \mathsf{pk}), r, \mathsf{pk}), \mathsf{aenc}(0, r, \mathsf{pk}), \mathsf{pk}) \to \mathsf{true}$*, and, symmetrically,* $\mathtt{check_ZKP}(\mathtt{ZKP}(\mathsf{aenc}(1, r, \mathsf{pk}), r, \mathsf{pk}), \mathsf{aenc}(1, r, \mathsf{pk}), \mathsf{pk}) \to \mathsf{true}$*.*

The Figure 2.1 shows the syntax of terms, expressions and processes.

M, N	$:=$		Terms
		x	variable ($x \in \mathcal{X}$)
		n	name ($n \in \mathcal{N}$)
		$f(M_1, \ldots, M_k)$	construction ($f \in \mathcal{F}_c$)
U	$:=$		May-fail terms
		M	term
		fail	failure
		u	may-fail variable
D	$:=$		Expressions
		M	term
		fail	failure
		$h(D_1, \ldots, D_k)$	application ($h \in \mathcal{F}_c \cup \mathcal{F}_d$)
P, Q	$:=$		Processes
		0	null process
		$\texttt{out}(N, M); P$	output
		$\texttt{in}(N, x); P$	input
		$P \mid Q$	parallel composition
		$!P$	replication
		$\texttt{new}\ n; P$	name restriction
		$\texttt{insert}\ t(M_1, \ldots, M_k); P$	table insertion ($t \in \mathcal{T}bl$)
		$\texttt{get}\ t(x_1, \ldots, x_k)\ \texttt{suchthat}\ D\ \texttt{in}\ P\ \texttt{else}\ Q$	table access ($t \in \mathcal{T}bl$)
		$\texttt{let}\ x = D\ \texttt{in}\ P\ \texttt{else}\ Q$	conditional declaration
		$\texttt{event}(ev(M_1, \ldots, M_k)); P$	event ($ev \in \mathcal{F}_{ev}$)

with k being the arity of the considered function.

Figure 2.1: Syntax of terms, expressions and processes

Terms are representing messages. Names represent nonces, keys, channels, etc. Variables can represent an input from another agent, or the result of a computation. Expressions represent computations. We present later how to evaluate an expression. Either it fails or it returns a term. The smallest possible process is 0, and is used as atom to build complex ones. The input and output operations represent a communication. In the process $\texttt{in}(N, x); P$, the term N is a channel, and x is a variable that will be substituted during the execution. The parallel composition $P|Q$ models two processes running in parallel. The replication $!P$ represents an arbitrary number of copies of a process running in parallel, e.g., a server with an unbounded number of sessions or clients. Name restriction $\texttt{new}\ n$ represents the creation of a fresh private name, for instance an agent generating a new key, drawing a nonce at random or opening a new communication channel. Events are used to keep track of actions during the execution of the protocol. For example, when a voter V casts a vote v, we can execute $\texttt{event}(\texttt{Voted}(V, v))$. In a messaging protocol, when the user A sends a message to or receives a message from the user B, we execute $\texttt{event}(\texttt{Sent}(A, B, m))$ or $\texttt{event}(\texttt{Received}(A, B, m))$.

We say that a name n is free in the process P if n appears in P and if P does not contain any $\texttt{new}\ n$. Similarly, we say that the variable x is free in P if x appears in P and if P does not contain any $\texttt{in}(N, x)$, $\texttt{let}\ x = D\ \texttt{in}$ nor $\texttt{get}\ t(x)\ \texttt{suchthat}\ D\ \texttt{in}$. We say that a process P is closed if if has no free variables. We note $fn(P)$ and $fv(P)$ the free names and the free

variables of P. We say that a term is a ground term if it does not contain variables.

Example 2. *We can write the TrackVote protocol from Section 1.4 in this syntax. One different process will represent each different agent of the protocol. We consider a constructor* $c^1_{\mathsf{auth}}(V)$ *that takes as input a voter and represents a secure channel between this voter and the registrar, and a constructor* $c^2_{\mathsf{auth}}(V)$ *that does the same thing with the server. We also consider a public name* c_{pub} *representing the bulletin board: it is the channel where the server publishes the accepted ballots. The registrar generates a nonce a for each voter* V*:*

```
1  let Registrar(V) =
2    new a;
3    out(c_auth1(V), a);
4    0
```

The voter receives their voting token a and sends their ballot (a, v) *to the server. Later, they look for their ballot on the board.*

```
1  let Voter(V, v) =
2    in(c_auth1(V), a);
3    out(c_auth2(V), (a, v));
4    event CAST(V, v)
5    in(c_pub, (=a, =v));
6    event HAPPY(V);
7    0
```

The line 5 is syntactic sugar taking a pair as input and comparing it to (a, v)*. We will explain in the semantics section how to realize it. Finally, the server takes the ballots of the voters and publishes them on the bulletin board.*

```
1  let Server(V) =
2    in(c_auth2(V), (a, v));
3    out(c_pub, (a, v));
4    event RECORDED(v);
5    0
```

The whole protocol is the parallel composition of those processes, for any number of voters:

```
1  let TrackVote =
2    !new V; (
3      Registrar(V) |
4      (new v; Voter(V, v)) |
5      Server(V)
6    )
```

Definition 1 (Substitution)**.** *We call substitution* σ *an application from the variables* $\mathcal{X}$ *to the terms, from the may-fail variables to the may-fail terms, and from the time variables to the integers. We extend* σ *to terms inductively:* $n\sigma = n$ *and* $f(M_1, \ldots, M_k)\sigma = f(M_1\sigma, \ldots, M_k\sigma)$*. We also extend* σ *to the expressions:* $\mathsf{fail}\sigma = \mathsf{fail}$*, and* $h(D_1, \ldots, D_k)\sigma = h(D_1\sigma, \ldots, D_k\sigma)$*. We proceed as well for tables, processes, multiset of processes, etc. We say that a substitution* σ *grounds a term* M *if* $M\sigma$ *is a ground term.*

Substitutions allow to write protocols using variables. For instance, in a voting protocol, the voting client receives a vote v from the user (a variable) and sends the encrypted vote $\mathsf{aenc}(v, r, pk)$ to the server with r a fresh randomness and pk the public key of the election. During the execution, we will consider the substitution that associate v to the term the voter gave to their voting machine.

Definition 2 (Evaluation)**.** *Let D be an expression. If D is a term M, then we say that D evaluates to M and we write $D \Downarrow M$. If $D = \mathsf{fail}$, then $D \Downarrow \mathsf{fail}$ as well. Otherwise, $D = h(D_1, \ldots, D_k)$. Let $U_1, \ldots, U_k$ be may-fail terms such that $\forall j \in \{1, \ldots, k\}, D_j \Downarrow U_j$. If $h \in \mathcal{F}_c$ is a constructor and if $U_1, \ldots, U_k$ are terms, then $h(D_1, \ldots, D_k) \Downarrow h(U_1, \ldots, U_k)$. If $h \in \mathcal{F}_d$ is a destructor, it is defined by its list or rewriting rules $h(U_{i,1}, \ldots, U_{i,k}) \to U_i$ for $i \in \{1, \ldots, n\}$. If there exists an index $i \in \{1, \ldots, n\}$ and a substitution σ such that $\forall j \in \{1, \ldots, k\}, U_{i,j}\sigma = D_i$, then let i_0 be the smallest such index. We pose $h(D_1, \ldots, D_k) \Downarrow U_{i_0}\sigma$. In all other cases, $h(D_1, \ldots, D_k) \Downarrow \mathsf{fail}$.*

2.2 Semantics

Definition 3 (Configuration)**.** *We call configuration a triplet $\mathcal{E}, \mathcal{P}, \mathcal{T}$ with $\mathcal{P}$ a multiset of processes, $\mathcal{E}$ a set of names, and $\mathcal{T}$ a set of tables.*

We use $\mathcal{E}$ to record which private names have already been used in the protocol, to represent the extremely low probability that two agents independently draw the same nonce. The multiset $\mathcal{P}$ lists all the processes that are currently running. At the beginning of the execution, one process represents the attacker and typically each agent is represented by an other process. Finally, $\mathcal{T}$ is the set of table facts of the form $tab(M_1, \ldots, M_n)$, which means that the tuple $M_1, \ldots, M_n$ has been inserted in the table tab. A table can be used to store pairs (user, password), for example.

The semantics of processes is defined on Figure 2.2. It is a reduction relation between configurations.

$$\mathcal{E}, \mathcal{P} \uplus \{\!\{0\}\!\}, \mathcal{T} \to \mathcal{E}, \mathcal{P}, \mathcal{T} \qquad \text{(Null process)}$$
$$\mathcal{E}, \mathcal{P} \uplus \{\!\{P \mid Q\}\!\}, \mathcal{T} \to \mathcal{E}, \mathcal{P} \uplus \{\!\{P, Q\}\!\}, \mathcal{T} \qquad \text{(Parallel)}$$
$$\mathcal{E}, \mathcal{P} \uplus \{\!\{!P\}\!\}, \mathcal{T} \to \mathcal{E}, \mathcal{P} \uplus \{\!\{P, !P\}\!\}, \mathcal{T} \qquad \text{(Replication)}$$
$$\mathcal{E}, \mathcal{P} \uplus \{\!\{\texttt{new } n; P\}\!\}, \mathcal{T} \to \mathcal{E} \cup \{n'\}, \mathcal{P} \uplus \{\!\{P|_{n \to n'}\}\!\}, \mathcal{T} \quad \text{if } n' \in \mathcal{N}_{\mathsf{priv}} \setminus \mathcal{E} \qquad \text{(Restriction)}$$
$$\mathcal{E}, \mathcal{P} \uplus \{\!\{\texttt{out}(N, M); P, \texttt{in}(N, x); Q\}\!\}, \mathcal{T} \to \mathcal{E}, \mathcal{P} \uplus \{\!\{P, Q|_{x \mapsto M}\}\!\}, \mathcal{T} \qquad \text{(Communication)}$$
$$\mathcal{E}, \mathcal{P} \uplus \{\!\{\texttt{let } x = D \texttt{ in } P \texttt{ else } Q\}\!\}, \mathcal{T} \to \mathcal{E}, \mathcal{P} \uplus \{\!\{P|_{x \mapsto M}\}\!\}, \mathcal{T} \quad \text{if } D \Downarrow M \qquad \text{(Let-1)}$$
$$\mathcal{E}, \mathcal{P} \uplus \{\!\{\texttt{let } x = D \texttt{ in } P \texttt{ else } Q\}\!\}, \mathcal{T} \to \mathcal{E}, \mathcal{P} \uplus \{\!\{Q\}\!\}, \mathcal{T} \quad \text{if } D \Downarrow \mathsf{fail} \qquad \text{(Let-2)}$$
$$\mathcal{E}, \mathcal{P} \uplus \{\!\{\texttt{event}(ev); P\}\!\}, \mathcal{T} \xrightarrow{\texttt{event}(ev)} \mathcal{E}, \mathcal{P} \uplus \{\!\{P\}\!\}, \mathcal{T} \qquad \text{(Event)}$$
$$\mathcal{E}, \mathcal{P} \uplus \{\!\{\texttt{insert } t(M_1, \ldots, M_n); P\}\!\}, \mathcal{T} \to \mathcal{E}, \mathcal{P} \uplus \{\!\{P\}\!\}, \mathcal{T} \cup \{t(M_1, \ldots, M_n)\} \qquad \text{(Insertion)}$$
$$\mathcal{E}, \mathcal{P} \uplus \{\!\{\texttt{get } t(x_1, \ldots, x_n) \texttt{ suchthat } D \texttt{ in } P \texttt{ else } Q\}\!\} \to \mathcal{E}, \mathcal{P} \uplus \{\!\{P\sigma\}\!\}, \mathcal{T} \quad \text{if } D\sigma \Downarrow \mathsf{true} \text{ and } t(x_1, \ldots, x_n)\sigma \in \mathcal{T} \qquad \text{(Reading-1)}$$
$$\mathcal{E}, \mathcal{P} \uplus \{\!\{\texttt{get } t(x_1, \ldots, x_n) \texttt{ suchthat } D \texttt{ in } P \texttt{ else } Q\}\!\} \to \mathcal{E}, \mathcal{P} \uplus \{\!\{Q\}\!\}, \mathcal{T} \quad \text{if } \forall \sigma, \neg D\sigma \Downarrow \mathsf{true} \text{ or } t(x_1, \ldots, x_n)\sigma \notin \mathcal{T} \qquad \text{(Reading-2)}$$

Figure 2.2: Semantics of processes

The Null process rule removes the 0 processes without doing anything else. The Parallel rule simply separates P and Q, to allow to execute them in any order. The Replication rule provides any number of copies of P from $!P$. When the Communication rule is executed, we substitute in the rest of the process the input to the corresponding variable. Similarly, the Let-1 rule substitutes the result of the computation in all the following. The `new` also triggers a substitution. We have to rename the nonce to take into account replications. For example, if we want to model an unbounded number of clients generating a key as $!(\texttt{new } k; P(k))$, with P using k to encrypt messages, all keys will be distinct. With the renaming, every agent uses a different k. Events purely disappear from the configuration when executing the Event rule, but remain in the trace. Finally, the Insersion and Reading rules interact with the table set

$\mathcal{T}$. A message written on a channel can be read only once and is blocking as long as it is not received. Conversely, a message on a table can be read any number of times.

Example 3. *The let can be used to model a conditional branch, thus explaining the absence of* `if` *in our model. Indeed, we introduce a destructor* $\mathtt{Equals}(D_1, D_2)$*, with the rewriting rules* $\mathtt{Equals}(D_1, D_1) \to \mathtt{true}$ *and* $\mathtt{Equals}(D_1, D_2) \to \mathsf{false}$*. We also introduce the* `If` *destructor, defined by* $\mathtt{If}(\mathtt{true}) \to \mathtt{true}$*. Then, the process* $\mathtt{let}\ x = \mathtt{If}(\mathtt{Equals}(D_1, D_2))\ \mathtt{in}\ P\ \mathtt{else}\ Q$ *behaves as one would expect* $\mathtt{if}\ D_1 = D_2\ \mathtt{then}\ P\ \mathtt{else}\ Q$ *to behave. We can express more complex formulas with the destructors* `And` *and* `Or`*, defined by* $\mathtt{And}(\mathtt{true}, \mathtt{true}) \to \mathtt{true}$ *and* $\mathtt{Or}(D, \mathtt{true}) \to \mathtt{true}; \mathtt{Or}(\mathtt{true}, D) \to \mathtt{true}$*.*

Definition 4 (Trace)**.** *Let* C_0 *be a configuration. We call* trace of C_0 *any finite sequence* $Tr = C_0 \to C_1 \to \ldots \to C_n$*, where* $\forall i \in \{0, \ldots, n-1\}, C_i \to C_{i+1}$ *is a reduction rule of the semantics. We write* $Tr[i]$ *for the configuration* C_i*. We say that* Tr *has* n *steps, and we call* i*-th step of* Tr *the step* $C_i \to C_{i+1}$*. We pose* $steps(Tr) = \{0, \ldots, n\}$*. We write* $C_0 \to^* C_n$ *if there exist configurations* $C_1, \ldots, C_{n-1}$ *such that* $C_0 \to C_1 \to \ldots \to C_{n-1} \to C_n$*. We call* $\mathbb{T}(C_0)$ *the set of all traces starting from* C_0*, and* $\mathbb{T}$ *the set of all traces.*

A trace represents an execution of the protocol, so we have to consider all traces to take into account all the possible executions.

Definition 5 ($\mathcal{N}_{\mathsf{pub}}$-adversary)**.** *We call* $\mathcal{N}_{\mathsf{pub}}$*-adversary any closed process* Q *that does not contain events nor tables, that does not use private constructor or destructor, and such that* $fn(Q) \subseteq \mathcal{N}_{\mathsf{pub}}$*.*

The attacker consists of the well-known *Dolev-Yao* adversary [DY83]. They are able to read any message sent on a public channel of the network or inject any message on this channel. They can perform any computation that requires only public names and function symbols. If they want to use a private name that they may learn, for example through a key compromission, then they use a variable instead and bind this variable to the result of an input or a computation. Finally, they can inject any message computed this way in any public channel, or private channel they got access. By default, the attacker cannot insert element in a table. Nevertheless, if we want to allow the attacker to write on the table t, for example if it represents a list whose administrator is compromised, we can add a dedicated process $!(\mathtt{in}(c, x); \mathtt{insert}\ t(x))$ where c is a public channel. Similarly, if we want the adversary to be able to read the elements of a table, we must add a specific process to our model.

In π-calculus, a lot of properties can be specified as a *correspondence query*. Intuitively, a correspondence query is a property of the form "if event A happened, then event B happened before". For example, in a messaging protocol, we want to be sure that when a user receives a message and think it comes from an other user, then the latter sent this exact message to the former. Thus, when user A receives a message m and thinks it comes from user B, we trigger $\mathtt{event}(\mathtt{Received}(A, B, m))$. Likewise, we trigger the event $\mathtt{event}(\mathtt{Sent}(B, A, m))$ when user B sends the message m to user A. To analyse our security property, we write the query $\mathtt{event}(\mathtt{Received}(A, B, m)) \Rightarrow \mathtt{event}(\mathtt{Sent}(B, A, m))$. We present the complete syntax of queries on Figure 2.3. We can also verify relations on terms. For instance, in a voting protocol with an untrusted voting device, when a user wants to vote v and gets an encryption x from their voting device, we trigger $\mathtt{event}(E(v, x))$. Then, we verify that for all execution of the protocol, $\mathtt{event}(E(v, x)) \Rightarrow x = \mathtt{aenc}(v, r, pk)$ for certain randomness r and for the public key of the election pk. Injective events allows to check properties of the form "in all executions, event B is executed more times than event A", useful for example in electronic voting to check that there

is less ballots in the ballot box than voters. Temporal facts are useful to order events. In a trace tr, $\texttt{event}(E)@i$ means that the event E has been executed at the i-th step of tr. For example, if event E should only trigger once: $\texttt{event}(E)@i \wedge \texttt{event}(E)@j \Rightarrow i = j$.

pev	$:=$		
		$\text{event}(ev(M_1, \ldots, M_k))$	event ($ev \in \mathcal{F}_{ev}$)
		$\text{inj-event}(ev(M_1, \ldots, M_k))$	injective event
F	$:=$		Temporal fact
		pev	
		$pev@i$	$i \in \mathbb{N} \cup \mathcal{X}_{\mathbb{N}}$
α	$:=$		Atomic formula
		F	temporal fact
		$M = N$	equality
		$M \neq N$	disequality
		$M \leqslant N$	inequality
ψ, ψ'	$:=$		Query conclusion
		$\top$	true
		$\bot$	false
		α	atomic formula
		$\psi \wedge \psi'$	conjunction
		$\psi \vee \psi'$	disjunction
q	$:=$		Correspondence query
		$F_1 \wedge \ldots \wedge F_n \Rightarrow \psi$	

Figure 2.3: Syntax of queries

Definition 6 (Satisfaction of a formula)**.** *Let $Tr \in \mathbb{T}$ be a trace. We define $\vdash$, the satisfaction relation of a ground formula as follows.*

- $Tr \vdash \texttt{event}(ev)@i$ *if* $Tr[i] \xrightarrow{\texttt{event}(ev)} Tr[i+1]$.
- $Tr \vdash \texttt{inj} - \texttt{event}(ev)@i$ *if* $Tr[i] \xrightarrow{\texttt{event}(ev)} Tr[i+1]$.
- $Tr \vdash \texttt{event}(ev)$ *if* $\exists i \in steps(Tr), Tr[i] \xrightarrow{\texttt{event}(ev)} Tr[i+1]$.
- $Tr \vdash \texttt{inj} - \texttt{event}(ev)$ *if* $\exists i \in steps(Tr), Tr[i] \xrightarrow{\texttt{event}(ev)} Tr[i+1]$.
- $Tr \vdash M = N$ *if* $M = N$.
- $Tr \vdash M \neq N$ *if* $M \neq N$.
- $Tr \vdash M \leqslant N$ *if* $M \leqslant N$.

We extend the satisfaction relation to query conclusions:

- $Tr \vdash \top$

- $Tr \nvdash \bot$
- $Tr \vdash \psi \wedge \psi'$ *if* $Tr \vdash \psi$ *and* $Tr \vdash \psi'$
- $Tr \vdash \psi \vee \psi'$ *if* $Tr \vdash \psi$ *or* $Tr \vdash \psi'$

Definition 7 (Correspondence query)**.** *Let $\mathcal{P}$ be a multiset of processes. Let $q = F_1 \wedge \ldots \wedge F_n \Rightarrow \psi$ be a correspondence query such that $fn(q) \subseteq \mathcal{N}_{\mathsf{pub}} \cup \mathcal{N}_{\mathsf{priv}}$, and such that q contains no* $\mathtt{inj} - \mathtt{event}$.

We say that $\mathcal{P}$ satisfies q against $\mathcal{N}_{\mathsf{pub}}$-adversaries and write $\mathcal{P} \models q$ if for all $\mathcal{N}_{\mathsf{pub}}$-adversary $\mathcal{A}$, for all trace $Tr \in \mathbb{T}(\emptyset, \mathcal{P} \cup \{\mathcal{A}\}, \emptyset)$, for all substitution σ, if $Tr \vdash F_1\sigma, \ldots, Tr \vdash F_n\sigma$, then there exists a substitution σ' such that $F_1\sigma' = F_1\sigma, \ldots, F_n\sigma' = F_n\sigma$, and $Tr \vdash \psi\sigma'$.

Due to the quantification of the substitutions σ and σ', in a correspondence query, all the variables before the $\Rightarrow$ are universally quantified, while the ones occurring in the conclusion are existentially quantified.

Example 4. *We present here an example of an attack against a query. We consider again the TrackVote protocol, and its model from Example 2. One natural property one could want to verify is that if a voter performed the verification successfully, then their vote has been recorded. We model this property by the query* $\mathtt{event}(\mathtt{CAST}(V, v)) \wedge \mathtt{event}(\mathtt{HAPPY}(V)) \Rightarrow \mathtt{event}(\mathtt{RECORDED}(v))$. *However, this query is not satisfied by this protocol, even without any adversary. Indeed, a trace does not require to execute the processes entirely. It is possible to execute the* `Registrar` *and the* `Voter` *processes to the end, but to stop the* `Server` *process between the output and the event. Thus,* $\mathtt{event}(\mathtt{CAST}(V, v))$ *and* $\mathtt{event}(\mathtt{HAPPY}(V))$ *are executed, but* $\mathtt{event}(\mathtt{RECORDED}(v))$ *is not. This kind of undesirable behavior can be avoided by switching the lines 3 and 4 of the* `Server` *process. Furthermore, this query does not faithfully capture the verifiability of the protocol. Indeed, the* `RECORDED` *event takes as input a vote, but does not link it to a voter. Thus, a trace with two different voters with the same vote v could perform the verification and trigger the* `HAPPY` *event while only one ballot being recorded. Injective correspondence queries are a tool to fill this blind spot.*

We consider only a restricted class of the possible injective correspondence queries.

Definition 8 (Injective correspondence query)**.** *Let $\mathcal{P}$ be a multiset of processes. Let $q = \mathtt{inj} - \mathtt{event}(E_0(M_1, \ldots, M_k)) \wedge F_1 \wedge \ldots \wedge F_n \Rightarrow \bigvee_{i=1}^{m} \mathtt{inj} - \mathtt{event}(E_i(M_1^i, \ldots, M_{k_i}^i)) \wedge \psi_i$ be a correspondence query such that $fn(q) \subseteq \mathcal{N}_{\mathsf{pub}} \cup \mathcal{N}_{\mathsf{priv}}$, and that $F_1, \ldots, F_n, \psi_1, \ldots, \psi_m$ contain no* $\mathtt{inj} - \mathtt{event}$.

We say that $\mathcal{P}$ satisfies q against $\mathcal{N}_{\mathsf{pub}}$-adversaries and write $\mathcal{P} \models_{\mathtt{inj}} q$ if it verifies the two following conditions:

For all $\mathcal{N}_{\mathsf{pub}}$-adversary $\mathcal{A}$, for all trace $Tr \in \mathbb{T}(\emptyset, \mathcal{P} \cup \{\mathcal{A}\}, \emptyset)$, for all substitution σ, if $Tr \vdash F_1\sigma, \ldots, Tr \vdash F_n\sigma$ and if $Tr \vdash \mathtt{event}(E_0(M_1\sigma, \ldots, M_k\sigma))$, then there exist an index $l \in \{1, \ldots, m\}$ and a substitution σ' such that $F_1\sigma' = F_1\sigma, \ldots, F_n\sigma' = F_n\sigma, M_1\sigma' = M_1\sigma, \ldots, M_k\sigma' = M_k\sigma$, and that $Tr \vdash \mathtt{event}(E_l(M_1^l\sigma', \ldots, M_{k_l}^l\sigma'))$ and $Tr \vdash \psi_l\sigma'$.

Moreover, in this case, if we note $A = \{j \in steps(Tr), Tr[j] \xrightarrow{\mathtt{event}(E_0(M\sigma'))} Tr[j+1]\}$ and $B = \{j \in steps(Tr), \exists i \in \{1, \ldots, m\}, Tr[j] \xrightarrow{\mathtt{event}(E_i(M_i\sigma'))} Tr[j+1]\}$, there exists an injective function $f : A \hookrightarrow B$ such that if $Tr[j] \xrightarrow{\mathtt{event}(E_0(M\sigma'))} Tr[j+1]$, then $Tr[f(j)] \xrightarrow{\mathtt{event}(E_l(M_l\sigma'))} Tr[f(j)+1]$.

Example 5. *We keep going with the TrackVote model example, and its modification from 4. Compared to the simple query from Example 4, the injective query* $\texttt{event}(\texttt{CAST}(V, v)) \wedge \texttt{inj} - \texttt{event}(\texttt{HAPPY}(V)) \Rightarrow \texttt{inj} - \texttt{event}(\texttt{RECORDED}(v))$ *requires the* `RECORDED` *events to be all distinct.*

Part II

Analysing and designing postal voting protocols

Chapter 3

Analysis of existing postal voting protocols

Remote voting is convenient in a lot of cases where voter cannot physically move to a polling station: citizens living abroad, or in hospital, in detention, etc. Several solutions exist in this case. Depending on the situation, it is possible to vote online, in a dedicated polling station (cast officer coming at the home of someone to record their vote), or over the post. Postal voting is widely used in practice. In Germany, almost thirty per cent of the ballots have been cast by mail during the 2017 federal election [WL20]. However, vote by correspondence as it is commonly deployed does not offer much security guarantees. The usual process for this is the double envelop mechanism. Each voter puts their plaintext vote inside an envelop, then inserts this envelop and their electoral card in a bigger envelop. When the polling station receives a ballot, they open it, check the eligibility with the electoral card and put the small envelop inside the ballot box. A compromised polling station can therefore learn the vote of any voter, block votes, or add arbitrary votes. Several protocols have been designed to make postal voting more secure. The STROBE schemes, as well as its extensions RemoteVote and SAFE Vote, rely on ballot trackers to ensure verifiability. A different approach is taken by Devillez et al., where return codes guarantee that the votes are recorded properly. These systems suffer from weaknesses due to the postal setting, and the first ones are vulnerable to different security breaches. After introducing these protocols, we present generic attacks of our finding and how to instantiate them. In this overview, we will sometimes mention our own proposal, Vote&Check, for the sake of comparison; this scheme will be presented and analysed in the next chapters.

3.1 Overview of existing postal voting protocols

3.1.1 STROBE

The STROBE [Ben21] voting scheme is a postal voting scheme designed by Josh Benaloh. It aims to provide verifiability while staying as close as possible to traditional vote by mail. The different entities involved are the printer, the postal service, the cast officer, the voters, and a set of trustees. There is a public board where the printer, the cast officer and the trustees can write any message.

STROBE uses probabilistic homomorphic encryption (for example ElGamal) under the public key of the set of trustees. For each voter, the printer encrypts the votes, i.e. the identity matrix of size the number of candidates (blank vote is encoded as a candidate named “None”).

Candidate	Encryption	Code
Alice	$(\mathsf{aenc}(1), \mathsf{aenc}(0), \mathsf{aenc}(0), \mathsf{aenc}(0))$	Q4
Bob	$(\mathsf{aenc}(0), \mathsf{aenc}(1), \mathsf{aenc}(0), \mathsf{aenc}(0))$	D6
Charlie	$(\mathsf{aenc}(0), \mathsf{aenc}(0), \mathsf{aenc}(1), \mathsf{aenc}(0))$	L7
None	$(\mathsf{aenc}(0), \mathsf{aenc}(0), \mathsf{aenc}(0), \mathsf{aenc}(1))$	E1

(a) Structure of the encryption

Alice	☐	Q4
Bob	☐	D6
Charlie	☐	L7
None	☐	E1
1960-857c-c5db-3939		

(b) Paper ballot

Encryption	Code
$(\mathsf{aenc}(0), \mathsf{aenc}(1), \mathsf{aenc}(0), \mathsf{aenc}(0))$	D6
$(\mathsf{aenc}(0), \mathsf{aenc}(0), \mathsf{aenc}(0), \mathsf{aenc}(1))$	E1
$(\mathsf{aenc}(0), \mathsf{aenc}(0), \mathsf{aenc}(1), \mathsf{aenc}(0))$	L7
$(\mathsf{aenc}(1), \mathsf{aenc}(0), \mathsf{aenc}(0), \mathsf{aenc}(0))$	Q4

Ballot code: 1960-857c-c5db-3939

(c) Information published on the board

Figure 3.1: Voting material in the STROBE protocol.

Each line of the encrypted matrix is hashed, only the last byte of the hash is kept as a short code (shown on Figure 3.1a). This is repeated until all short codes are different. Lines of the matrix are permuted such that the short codes are ranked alphabetically. The identifier of this ballot is the hash of the permuted matrix. The permuted matrix, the short codes and the ballot identifier are published on the board, as well as zero-knowledge proofs (produced by the printer) that this matrix is the encryption of a permutation (cf Figure 3.1c). Ballots (shown on Figure 3.1b) are paired and the pairings are made public. Each voter receives by postal mail two paired ballots. They can verify that the ballots are well-formed by looking if the information on the board corresponds to what they have received. Then they simply choose one of the ballots, tick the box of their choice and send it by postal mail to the cast officer. The cast officer publishes the ballot identifier and the chosen short code, that a conscientious voter can verify. Finally, the trustees multiply all the selected encryptions and decrypt the product, giving the result of the election. The printer also reveals the random numbers used to encrypt the unused ballots; auditors can then check that they were well-formed.

3.1.2 RemoteVote and SAFE Vote

RemoteVote and SAFE Vote [CRH22] are two postal voting schemes inspired by STROBE. The goal was to gain everlasting privacy as well as some usability. The setting (agents, bulletin board, short codes printed on the ballot, etc) is the same as in STROBE. These schemes also use perfectly hiding commitment, denoted by com. Most notably, each voter does not receive two ballots anymore but only one. We highlight here the main changes made.

Candidate	Encryption	Commitment	Code
Alice	$x_0 = (\mathsf{aenc}(1), \mathsf{aenc}(0), \mathsf{aenc}(0), \mathsf{aenc}(0))$	$\mathsf{com}(x_0)$	Q4
Bob	$x_1 = (\mathsf{aenc}(0), \mathsf{aenc}(1), \mathsf{aenc}(0), \mathsf{aenc}(0))$	$\mathsf{com}(x_1)$	D6
Charlie	$x_2 = (\mathsf{aenc}(0), \mathsf{aenc}(0), \mathsf{aenc}(1), \mathsf{aenc}(0))$	$\mathsf{com}(x_2)$	L7
None	$x_3 = (\mathsf{aenc}(0), \mathsf{aenc}(0), \mathsf{aenc}(0), \mathsf{aenc}(1))$	$\mathsf{com}(x_3)$	E1

(a) Structure of the encryption

Alice	☐	L1	Q4
Bob	☐	P2	D6
Charlie	☐	C6	L7
None	☐	T2	E1
1960-857c-c5db-3939			

(b) Paper ballot

Commitment 1	Commitment 2	Code 1	Code 2
$\mathsf{com}(x_0)$	$\mathsf{com}(x'_0)$	L1	Q4
$\mathsf{com}(x_1)$	$\mathsf{com}(x'_1)$	P2	D6
$\mathsf{com}(x_2)$	$\mathsf{com}(x'_2)$	C6	L7
$\mathsf{com}(x_3)$	$\mathsf{com}(x'_3)$	T2	E1

Ballot code: 1960-857c-c5db-3939

(c) Information published on the board

Figure 3.2: Voting material in RemoteVote.

RemoteVote In RemoteVote, the talliers begin by generating a thresholdized public key and publishing it on the bulletin board. Then, the printer generates for each voter two parts of ballot. Therefore, they generate a seed R used to compute all the randomness used in this part. They encrypt each option with an homomorphic encryption scheme, and compute a commitment of this encryption. They also compute a shortcode for this option, like in STROBE, by truncating the hash of the commitment. Finally, they pair the parts of ballot and generate for each pair a long hash of all the commitments. They publish the commitments, short codes and long codes on the bulletin board, as indicated in Figure 3.2c. Then, they send to each voter a printed ballot that contains the name of the candidates, the short codes and the long code, but neither the encryptions nor the commitments.

At the end of the setup phase, for each ballot, a part will be audited and spoilt. In STROBE, the voters themselves choose which part to audit. Instead, in RemoteVote, a random number in generated by the auditors. This nonce determines which part of each ballot is audited. The printer reveals the seed R of the corresponding part. The auditors verify that the computations were correct, and the voters verify that the information on the board is consistent with their printed ballot.

The voting phase takes place in the same way as in STROBE, the voters simply fill their ballot and send it by the post to the ballot collector. When the ballot collector receives a ballot, they publish the corresponding long code and the selected options' short codes on the bulletin board. The printer opens the commitments of the selected options, and the talliers decrypt these ciphertext after shuffling them using a mixnet.

SAFE Vote In SAFE Vote, the idea of pairing the ballots disappears and is replaced by an kind of audit-or-cast procedure. The ballots are generated as in STROBE, except that the printer adds the randomness used to encrypt the data on the paper, hidden behind a scatch-off surface. Each voter also receives only one ballot. If they want to audit it, they scrape the mask to reveal the randomness, and use it to recompute the ballot. If something is wrong, they have a proof to complain. Otherwise, they simply request a new ballot.

3.1.3 Devillez et al.'s protocol

The approach proposed in [DPP24] by Devillez et al. still relies on verification codes, but goes further away from STROBE. It has higher security features than the other three schemes we mentioned. This scheme has also been fully formalised and analysed, using the Universal Composition security framework.

In this protocol, the printer generates all the voting material to be sent to voters, together with the associated cryptographic data to be sent to other parties. The paper ballots received by the voters contain one selection sheet, that is very similar to a classical paper ballot, with the only addition of a random token tk. The voter ticks the boxes near the candidates they select, and they send back this sheet by post. They also receive from the printer a list of return codes, 2 for each candidate, corresponding to whether or not they selected this candidate. They also get the value $\mathsf{hash}(\mathsf{tk})$.

After the result is announced, the voters can contact (electronically) a verification server, send them $\mathsf{hash}(\mathsf{tk})$, and they will receive back the return codes corresponding to their choices.

In order to make this process secure, the verification server does not receive directly the return codes from the printer. Instead, the printer will send them encrypted to talliers, who share the decryption key. These talliers operate after the tally; they collectively decrypt only the relevant return codes and send them to the verification server.

3.1.4 Other schemes

Other approaches [BRT13, MBC+21] are also called postal voting as voters cast their ballot by post but they need a computer to generate the voting material and they also need to print the material themselves. Hence, even if they help a lot to avoid breaking voter's privacy, they are closer to Internet voting.

Finally Prêt-à-Voter [RBH+09] and Pretty Good Democracy (PGD) [RT09] are voting systems where voters can vote using only paper ballots. However, these protocols are designed for on-site voting, which leads to a different threat model. As it is not obvious to turn them into (remote) postal voting, we chose to not consider them in our overview.

3.2 Generic attack scenarios

Verifiable postal voting systems have a common structure. Voters receive some material by post, that has been issued by one or several authorities (among them, the printer). Voters then select their candidate and send back their selection, possibly retaining some part of the voting material. At the end of the election, voters typically have access to some additional data, for example on a public bulletin board, and they can perform some checks to verify that their vote has been properly counted. We list generic attacks against privacy and verifiability which may apply to any system of this type if it does not implement dedicated countermeasures. For each property, we consider several corruption scenarios and mention possible countermeasures if they

exist: either the attack assumes only external attackers, or it assumes a corrupted printer, or a corrupted ballot collector. When it is relevant, we also mention which countermeasure we chose during the design of our protocol Vote&Check. Of course, this list of attacks is not exhaustive.

3.2.1 Threat model

The list of parties involved in a postal voting system varies across the systems but typically contains at least: the voters; a printer, that prints and sends the voting material; a ballot collector; and a bulletin board. The ballot collector is in charge of collecting the ballots sent by voters and announcing the result, possibly publishing some data on a bulletin board. While voters, printer, and ballot collector may all be dishonest, the bulletin board is typically assumed to be a secure, public, and append-only board that anyone can see in an authenticated manner.

In what follows, we list potential attacks against privacy and verifiability, when at most one of these parties is corrupted. Some postal voting systems involve other parties, that may also be corrupted but these system specific scenarios are not considered here.

3.2.2 Privacy

Targeted privacy attack (corrupted voters). A dishonest voter Charlie gives his material to Alice (e.g., drops his material in Alice's mailbox) and keeps a copy. Alice uses this material instead of hers. Then Charlie uses the verification mechanism to learn Alice's vote. A more powerful variant is when Charlie can actually generate valid ballots by himself, which allows him to attack privacy of several voters, while keeping his right to vote.

Possible countermeasures: by signing the material (or part of it), one can prevent Charlie from generating valid ballots by himself. Moreover, if this signature binds the identity of the voter it could prevent Charlie from switching his material with Alice's. However, it introduces a new trusted party (the signer). Of course the trust can be put elsewhere, e.g., assuming the presence of secure channels between the voter and some authorities/parties. This is the mechanism used in [DPP24] and, to some extent, in Vote&Check as well.

Full privacy breach (corrupted printer). Some authority, typically the printer, knows the link between the voter and some identifying data (e.g., an authentication token). This identifying data may appear on the bulletin board, next to the vote in clear. In that case, the authority (honest but curious) breaks vote privacy of all voters. A variant of the attack is when the authority generates all possible encrypted votes for a voter and the selected encrypted ballot appears on the bulletin board. The ballot identifies the voter and hence vote privacy is broken again. This is a powerful attack since the privacy of all voters is broken w.r.t. this authority, without any detection.

Possible countermeasures: a trivial countermeasure is to keep the bulletin board secret (of course it raises other challenges for verifiability). Another approach is to distribute the generation of the identifying data as done in Vote&Check.

Privacy breach by complaints (corrupted ballot collector). When votes appear in clear on the paper ballot, the authority collecting the ballots may alter or remove a vote (without knowing who voted for it) and see who complains.

Such an attack also applies to pure Internet voting schemes as well, such as sElect [KMST16], Selene [RRI16], or Hyperion [RRR21]. This attack is detectable, by construction, but a few complaints may not draw attention. This attack was not detected in their respective privacy proofs [BDS17, KMST16] because they assume that the collecting authority behaves honestly w.r.t. the honest voters.

Possible countermeasures: when the verification fails, we can make the voter complain to a third party (e.g. a judge). Then, the judge will reveal how many voters complained but not their identity. Thus, the attacker needs the judge to be compromised to conduct this attack.

3.2.3 Verifiability

Ballot stuffing and vote flooding attacks (external attackers). In some systems, anyone may create a fake ballot and vote, which leads to ballot stuffing. This is typically avoided by authenticating some part of the voting material. A weaker form of this attack is when an attacker may find sufficient information (e.g., on the bulletin board) such that they can cast a vote without knowing for whom they voted. This allows to artificially increase the turnout and change the proportion of votes for each party, allowing e.g., a party to reach a certain quorum. We call this attack *vote flooding*.

Possible countermeasure: binding each ballot to a unique identifier prevents these attacks. However, this binding must remain secret until it is used to prevent flooding attacks. This binding can be ensured by different authorities (like in [DPP24] or Vote&Check) to avoid single point of failures.

Weak eligibility attack (corrupted printer). An authority (typically the printer) can keep copies of the voting material and vote for absentees.

Possible countermeasure: Such an attack seems unavoidable if the printed material is solely sufficient to cast a vote.

Clash attack (corrupted printer). An authority (typically the printer) may send the same ballot to two voters that vote the same way (this can be guessed easily for some voters). Then only one of the two votes is counted. Such an attack leaves traces since the ballot collector will typically receive duplicated ballots. However, it is hard to decide what to do when two identical ballots are received since some dishonest voters may also try to vote twice by reusing their material. Hence, it is hard to identify who misbehaved.

Possible countermeasure: binding the ballots (or voters) to a unique identifier prevents this attack. However, it requires that injectiveness of the mapping is guaranteed by an authority other than the printer who is assumed to be compromised in this case.

Alter votes of non-verifying voters (corrupted ballot collector). Not all voters perform the verification steps. For non-verifying voters, an authority may try to flip their vote. Note that the collecting authority may always drop the vote (since the voters do not verify) but flipping the vote yields a more powerful attack.

Possible countermeasure: while there are techniques to prevent vote flipping in Internet voting protocols, we are not aware of a technique that is usable for postal voting since they require cryptographic operations performed by the voter/voting device.

3.3 Instantiation of the attacks on existing systems

The attacks we presented in Section 3.2.3 are generic in the sense they apply to a broad class of remote voting protocols. All of them weaken at least one of the protocol we presented, some seem inherent to postal voting and are present in all cases. The Section 3.3.4 recapitulates the situation.

3.3.1 STROBE

Most of the generic attacks mentioned on Section 3.2 apply to the STROBE protocol, in different threat models.

Full privacy breach. The printer is supposed to not retain the link between voters and ballots. However, there is no way to verify that a dishonest printer indeed deleted it and if they have not, then they can learn the correspondence between plaintext votes and voters, by looking at the selection code and recalling to which candidate it corresponds.

Targeted privacy breach. If Charlie is a dishonest voter and wants to learn Alice's vote, he can copy his ballot and drop the original one in Alice's mailbox. In STROBE, ballots are all similar and unrelated to the voters, so that Alice has no way to distinguish her ballot from Charlie's. If she votes with the latter, Charlie will learn her vote by reading the selection code on the public board.

Privacy breach by complaints. The cast officer collects all the ballots. They know all plaintext votes, but they do not know the link between those votes and the voters. A compromised officer can willingly drop ballots for a specific candidate. Then, they can deduce that all the voters who complain have voted for this candidate.

Vote flooding. Using the public information from Figure 3.1c, an attacker can produce fake ballots, indistinguishable from real ones. They just have to rank the selection codes in a random order and send the forged ballot to the polling station to have it counted. This will artificially raise the turnout, as well as the score of the below-average candidates.

Alter votes of non-verifying voters. The cast officer receives all plaintext votes and have to record the corresponding selection codes on the public board. A compromised officer can drop the ballots they dislike or record other selection codes instead. This will be undetected if the corresponding voters do not verify.

Among all of these, Full privacy breach and Alter non-verifying voters are acknowledged in the STROBE paper: it is made very clear by the author that the printer is assumed to be honest, and there is no claim of verifiability if the voter does not verify. The other three attacks seem to have been overlooked by the author.

3.3.2 RemoteVote

The main difference between STROBE and RemoteVote is that each voter receives a single ballot instead of two, to improve usability. The ballot contains the two columns of selection codes that would have been on the two STROBE ballots. Similarly, one of these columns is spoiled after the election for verification while the second is used for tallying. But unlike in STROBE, it is not the voter who arbitrarily chooses which column is audited, but it is the result of a computation with the data of the ballot and a nonce randomly chosen after the election by a third party. This makes a *clash attack* possible, that we now describe.

If the printer suspects several voters to cast the same vote, they may provide them with the same ballot. The polling station cannot distinguish this from one voter sending multiple copies of their ballot. Since which column is spoilt is the result of an unpredictable but deterministic computation, it is the same for all the copies of the ballot, so each voter will be able to conduct the verifiability even if only one vote has been recorded for all of them. This attack would have been detected in STROBE, where the voters with the same two ballots would have cast either one or the other. The polling station would have accepted only one of them and the voters that used the other one could have detected it.

Another major difference with STROBE is that, in RemoteVote, there is no permutation.

Hence, from the data that is published, anyone can create a fake ballot, and vote for the candidate of their choice. Therefore, the vote flooding attack that was present in STROBE becomes a more powerful ballot stuffing attack.

3.3.3 SAFE Vote

In this variant, each ballot contains a single column of selection codes. The randomness that allows to perform the audit for this ballot is printed directly on the ballot, but is concealed behind a scratch-off surface.

This does not really change the situation. The clash attack by the printer works exactly the same as in RemoteVote. Also, since the audit is not performed by the authorities, anyone can use the public data to create fake ballot, putting random values behind the scratch-off. We therefore have the same ballot stuffing attack as in RemoteVote.

3.3.4 Devillez et al.'s protocol

The scheme is claimed to be secure as soon as there is no collusion between two authorities, nor between one authority and the post. However, we noticed a path to a clash attack when the printer colludes with the post. If the printer suspects that two voters will vote in the same way, it can send them exactly the same voting material, and send to the authorities the same cryptographic data for these voters. In this attack, the post will drop one of the ballots sent by those two voters. Based on the cryptographic material received from the election office, the verification server will answer correctly to both voters, unless they check that there are no duplicates in the data they received from the printer. While this check is not present in the description of the protocol, the authors had this verification in mind, because the security proof explicitly relies on it.

Compared to our Vote&Check protocol, other more fundamental drawbacks of the protocol by Devillez et al. are the following. First, it is not really universally verifiable; one has to trust authorities for this property. Furthermore, it uses more advanced cryptographic tools, such as a distributed threshold key generation, for the talliers. Finally, it requires more authorities than in Vote&Check, which might be a problem for practical deployment.

		STROBE [Ben21]	RemoteVote / SAFE Vote [CRH22]	Devillez et al. [DPP24]	Vote&Check (this work)
	# of authorities (excl. post and auditor)	2 + **1**	2 + **1**	4 + **1**	3
	public board	yes	yes	no	yes
privacy attacks	targeted privacy breach	✘[new]	✘[new]	✔	✔
	full privacy breach	✘	✘	✔	✔
	privacy breach by complaints[new]	✘	✘	✘	✘
verifiability attacks	ballot stuffing	✔	✘[new]	✔	✔
	vote flooding	✘[new]	✘[new]	✔	✔
	weak eligibility[new]	✘	✘	✘	✘
	clash attack	✔	✘[new]	✔*	✔
	alter non-verifying voters	✘	✘	✘	✘
	universal verifiability	✔	✔	proxy	✔

Figure 3.3: Application of the generic attacks to postal voting schemes. For the number of authorities, we indicate with a bold font "+ **1**", when one authority is actually a set of thresholdized authorities, typically the decryption trustees. The symbol ✔ means we did not find this kind of attack on the protocol, while the symbol ✘ indicates that the protocol is subject to this attack. The symbol [new] means that this attack is our finding; otherwise, it was part of the threat model of the authors of the said protocol. When the symbol qualifies a property, it means that this is a refined security property that we introduce in this paper. Finally, for [DPP24], we marked the symbol with *, because a verification step required for this property is left implicit in the description of the protocol.

Chapter 4

Presentation of the Vote&Check protocol

In this section, we present Vote&Check, our proposal to address the secure postal voting problem. Our main idea is to give each voter a tracker that they will use to control that their vote has been recorded properly. The verification step and the tally phase are simple since all votes are processed in clear. To avoid an authority to break voters' privacy, the generation of the tracker is split between two agents. Our protocol offers stronger security guarantees than the other schemes while requiring less participants and only standard cryptography. We provide a quick overview of Vote&Check before presenting in details the protocol and the security goals it achieves.

4.1 High level description

4.1.1 Participants

The Vote&Check protocol involves the following participants:

- **Printer**. This entity is responsible for generating and sending by postal mail the voting material to the voters.
- **Tracker Server**. This server provides a tracker to each voter, to let voters check that their ballot is counted.
- **Election Office**. This entity receives the postal ballots from the voters, and in the end publishes the votes in clear, together with verification data.
- **Public board**. A publicly readable place, that contains the result and verifiability data.
- **Auditors**. One or several entities who perform consistency checks of what is written on the board.
- **Voters**. The voters are assumed to have a valid postal address, and, for verifiability, an electronic way to receive securely data from the Tracker Server, and to read the public board.

In the description of Vote&Check, we will often separate the honest voters in two groups: those who perform all the optional verifications steps, thus requiring an electronic device which

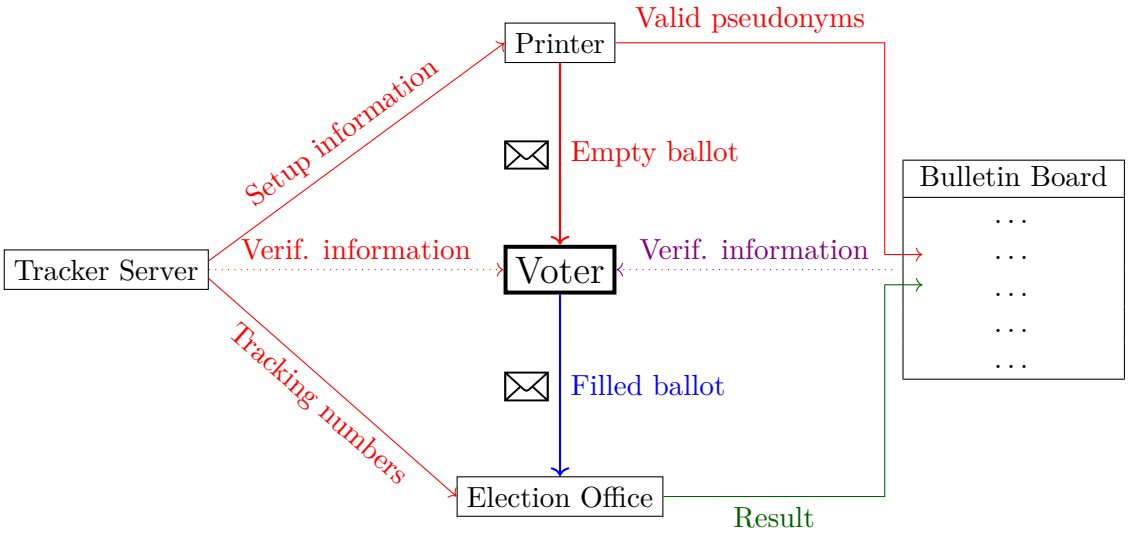


Figure 4.1: **Vote&Check protocol overview**. A different colour is assigned to each phase: red for the setup phase, blue for the voting phase, green for the tally phase, and violet for the verification phase. Dotted lines are optional: voters can still vote if they are offline.

is able to scan QR-codes and which is connected to the Internet, and those who stick to the traditional steps that require only pen and paper. Voters in the first group are called *conscientious*, and the others are called *offline*. For the security analysis, we will also consider a group of corrupt voters that do not necessarily follow the protocol, but may use their voting material or server access to help the attacker breaking privacy or verifiability.

4.1.2 Protocol phases

The protocol is divided into several phases: setup, vote, tally and verification, as shown on Figure 4.1. During the setup, authorities send material to voters: they get voting material (Figure 4.2) and a verification sheet, by post, from the Printer. Conscientious voters also request and receive electronically verification material from the Tracker Server. The Tracker Server sends tracking information to the Election Office. During the voting phase, voters fill in their paper ballot with a pen, and send them, by post, to the Election Office. During the tallying phase, the Election Office performs some validity checks and publishes every valid vote next to the corresponding tracking information. During the verification phase, conscientious voters can verify that their vote has been recorded using their verification material, while auditors control the integrity of the result.

4.2 Detailed description

The details of each phase of the protocol, with the flow of messages between the participants, are summarized in Figure 4.3.

4.2.1 Channels and authentication

The protocol relies on secure channels between the participants, which guarantee integrity, authenticity, and confidentiality of all messages. The exceptions are that reading the board does

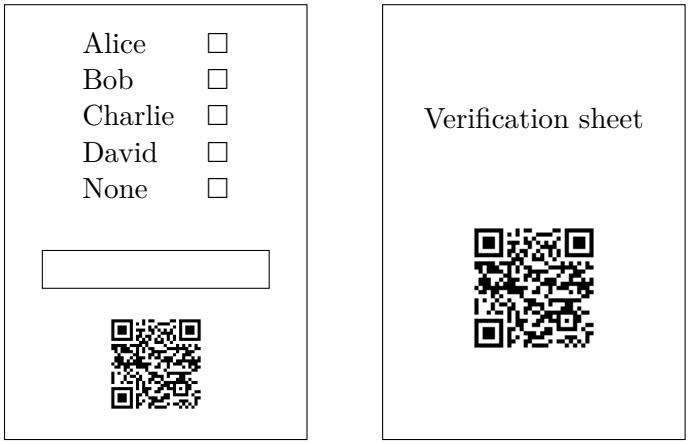


Figure 4.2: **Example of voting material**. On the ballot (left), the QR-code contains the voter's pseudonym a_i, the ballot's credential c_i, and signatures of a_i and of (a_i, c_i) by the Printer. The empty box is for the voter to write their nonce n_i. On the verification sheet (right), the QR-code contains the signature of (a_i, c_i, V_i) by the Printer.

not require authentication and that the postal channels are anonymous.

Writing on the board requires authentication and we assume that each piece of data on the board is available together with the identity of the writer of this data, which could be implemented by each writer signing the data that they put on the board. We leave these implicit, because we consider them as part of the public board functionality.

Furthermore, the Printer needs to sign some material that is sent to the voter, and this is made explicit in the protocol. Any classical signature scheme, like Schnorr or ECDSA, can be used, as long as it fits in a QR-code. The public key of Printer is assumed to be known by all parties. We denote by sig the Printer's signature function, and we omit the key. We also omit the fact that the signature must be bound to a precise election, so that it can not be replayed, for instance if there are two rounds of elections with the same participants. Therefore, the notation $\mathsf{sig}(x)$ that we will use from now must be understood as $\mathsf{sig}_{\mathsf{sk(Printer)}}(\mathsf{context}, x)$. The protocol will also use a cryptographic hash function, denoted by hash.

4.2.2 Setup phase

Let $\mathcal{V} = \{V_i, i \in \mathcal{I}\}$ be the set of all n voters, where $\mathcal{I} = [1, n]$. We assume $\mathcal{V}$ is known by all the authorities.

First, for every voter V_i, the Tracker Server generates a pseudonym a_i (a nonce) and a tracker t_i (another nonce). The Tracker Server sends their pseudonym and tracker (a_i, t_i) to each voter V_i over an electronic channel. The Tracker Server also sends the permuted list $\{(a_{\pi(i)}, t_{\pi(i)}), i \in \mathcal{I}\}$ to the Election Office, where π is the permutation over $\mathcal{I}$ that sorts the a_i alphabetically. Finally, the Tracker Server sends the list $((V_i, a_i), i \in \mathcal{I})$ to the Printer.

The Printer checks that the pseudonyms are pairwise distinct and that the identities match the already published list $\mathcal{V}$. If it is the case, they generate for each voter V_i a credential c_i and send to V_i over the postal channel a ballot containing a_i, c_i, $\mathsf{sig}(a_i)$ and $\mathsf{sig}(a_i, c_i)$ in a QR-code, plus a sheet that contains $\mathsf{sig}(a_i, c_i, V_i)$. This last signature is sent to the voter in a separate sheet so that the Election Office will not receive it, to protect voter's privacy. This signature is there to guarantee the voter that they are the intended recipient of this voting material and

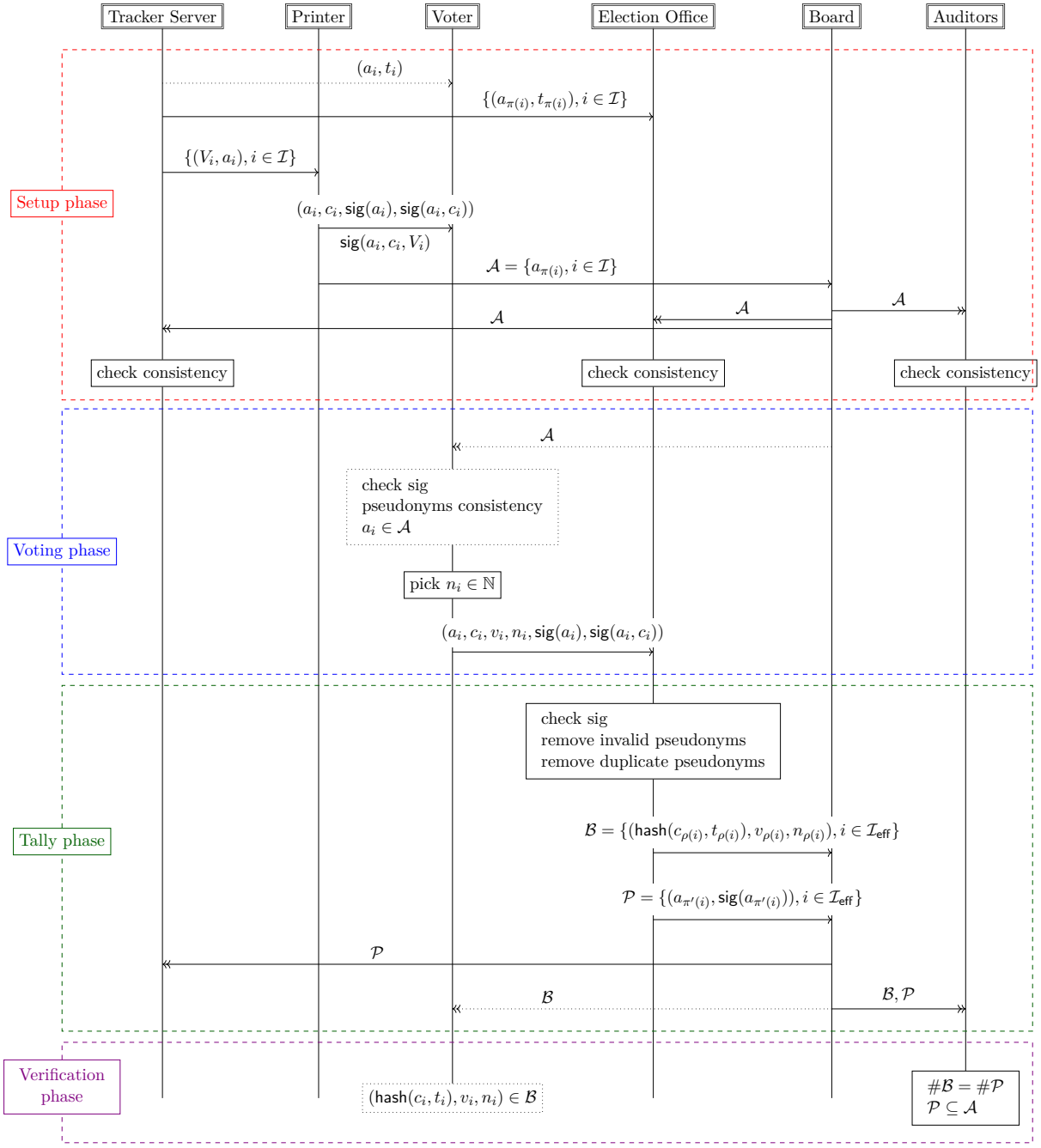


Figure 4.3: Vote&Check protocol. Dotted lines and boxes represent optional steps for the voter. The conscientious voters perform them, while the offline voters do not. Reading the Board is possible at any time, but we emphasize some natural moments for reading it with double-tipped arrows.

protect against targeted privacy breach. It also serves as a way for the voter to keep the value c_i for the verification phase. Finally, the Printer publishes the list $\mathcal{A} = (a_{\pi(i)}, i \in \mathcal{I})$ on the board, where π is the same sorting permutation as before. The other authorities, namely the Tracker Server and the Election Office, verify that this list matches their own view. The Auditors check that the list contains the correct number of items.

4.2.3 Voting phase

Each voter receives by post the two sheets of paper sent by the Printer: their paper ballot and the additional paper with just a signature. Each conscientious voter, with their verification device, also receives their verification material from the Tracker Server and verifies that the three signatures on the ballot and on the additional paper are correct and consistent, that their name appears (signed) on the additional sheet, and that the pseudonym on the ballot matches the one on the verification material and that this pseudonym is present in the list $\mathcal{A}$ from the board.

The voting procedure itself, done by both conscientious and offline voters involves just pen and paper. They pick a short nonce n_i and write it on their ballot. Then, they tick the box corresponding to their vote v_i and send the ballot to the Election Office via the post. Combining the handwritten and the electronic data contained in the QR-code, the ballot contains $(a_i, c_i, v_i, n_i, \mathsf{sig}(a_i), \mathsf{sig}(a_i, c_i))$.

The nonce n_i is chosen and written by the voter and must therefore be short. However, it must be large enough to ensure that a clash attack will go undetected with only a small (but non-negligible) probability. We typically suggest a 3-digit number.

4.2.4 Tally phase

The Election Office controls the signatures on the received ballots and discards those with incorrect ones. They also discard ballots with a pseudonym a_i that does not belong to $\mathcal{A}$, as seen on the board. Lastly, they discard every ballot using a pseudonym that has already been used. The Election Office is responsible for ensuring these three properties. For every remaining ballot $(a_i, c_i, v_i, n_i, \mathsf{sig}(a_i), \mathsf{sig}(a_i, c_i))$, the Election Office finds the corresponding couple (a_i, t_i) in the list received from the Tracker Server during setup, and inserts $(\mathsf{hash}(c_i, t_i), v_i, n_i)$ in the list $\mathcal{B}$ of valid votes and $(a_i, \mathsf{sig}(a_i))$ in the list $\mathcal{P}$ of used pseudonyms. To publish the result, the Election Office publishes the shuffled list of valid votes $\mathcal{B} = \{(\mathsf{hash}(c_{\rho(i)}, t_{\rho(i)}), v_{\rho(i)}, n_{\rho(i)})\}_i$ where ρ sorts the $\mathsf{hash}(c_i, t_i)$ in alphabetical order. We note $\mathcal{I}_{\mathsf{eff}}$ the subset of $\mathcal{I}$ containing these indexes. The Election Office also publishes the list of used pseudonyms $\mathcal{P} = \{(a_{\pi'(i)}, \mathsf{sig}(a_{\pi'(i)}))\}_i$, where, as for $\mathcal{A}$, π' sorts the a_i in alphabetical order.

4.2.5 Verification phase

Auditors verify that the lists of used pseudonyms and of valid votes have the same length, that each pseudonym is signed and belong to $\mathcal{A}$, and that $\mathcal{A}$ has a size equal to the number of registered voters. Each conscientious voter computes $\mathsf{hash}(c, t)$ from c and t received resp. from the Printer and Tracker Server and checks that it appears on $\mathcal{B}$ next to their vote v and nonce n.

4.3 Security claims

As usual in analyzing the security of voting systems, we assume that at least one of the Auditors is honest and will raise an alert if something goes wrong. Also, we allow several voters to be dishonest and to help the attacker in attacking privacy and individual verifiability of honest voters.

4.3.1 Privacy

Vote secrecy is guaranteed as long as at least two authorities among the Tracker Server, the Printer and the Election Office are honest. The key to this is that the information published on the board can not be tracked back to voters without data coming from both the Tracker Server and the Printer.

Not surprisingly, as soon as two authorities collude, they can break privacy, as we now explain briefly. If the Tracker Server and the Printer collude, they hold all the information that voters use to perform the verification: the Tracker Server knows (V_i, t_i), the Printer knows (V_i, c_i) and on the board stands $(\mathsf{hash}(c_i, t_i), v_i)$. Similarly, the Election Office knows (t_i, v_i) and (c_i, v_i) so they may collude either with the Tracker Server or the Printer to break the confidentiality.

Moreover, Vote&Check is subject to privacy breach by complaints as introduced in Section 3.2: if the Election Office removes a ballot for candidate A, it can then observe who complains: the complaining voter has voted for A. This can be scaled to several voters by removing several ballots for A. Instead, Vote&Check ensures privacy in the context of *anonymous complaints*, where we assume that complaining voters can anonymously contact a third party (a judge for example). In practice, a voter will lose privacy if they complain publicly, for example on a social network. But they will more reasonably be instructed to contact some dedicated authority that does not know which ballots have been removed.

4.3.2 Verifiability

Vote verifiability covers four main properties: *cast-as-intended*, i.e., the ballot cast by the voter contains their intended vote; *recorded-as-cast*, i.e., the ballot registered in the ballot box is the ballot cast by the voter; *tallied-as-recorded*, i.e., the result corresponds to the ballots in the ballot box; *eligibility verifiability*, i.e., the ballots only come from legitimate voters. We call *individual verifiability* the combination of the first two properties.

These properties are intuitive but they assume a particular setting (e.g., a global ballot box) and they may miss some attacks such as clash attacks [KTV12] where two voters agree on the same ballot. Moreover, they do not cover the case of offline voters, or more generally, the case of voters who do not verify, while it unfortunately corresponds to the most frequent case. Hence, a more general notion of *end-to-end verifiability* has been proposed [CGK+16]. It guarantees that the result of the election is the disjoint union of:

- all the votes of conscientious voters. This combines the *individual verifiability* property and *no clash attack*.
- a subset of votes of honest voters who did not verify their votes (called offline voters). Intuitively, an attacker may always drop such votes hence at best a subset of these votes will be counted.
- a set of votes corresponding to the corrupted voters (at most as the number of corrupted voters).

Election Office status	H	C	H	H	H	C	C	C
Printer status	H	H	C	H	C	H	C	C
Tracker Server status	H	H	H	C	C	C	H	C
Privacy (conscientious voter)	✔	✔	✔	✔	✘	✘	✘	✘
Privacy (offline voter)	✔	✔	✘	✔	✘	✘	✘	✘
Individual verifiability (conscientious voter)	✔	✔	✔	✔	✔	✔	✔	✔
Weak universal verifiability	✔	✔	✘	✔	✘	✔	✘	✘
Universal verifiability	✔	✘	✘	✔	✘	✘	✘	✘

Figure 4.4: **Vote&Check security properties** in considered threat models. A H means that the entity is honest, a C that the entity is corrupt.

End-to-end verifiability itself can be split into individual verifiability for conscientious voters and *universal verifiability*, where universal verifiability states that each ballot in the ballot box is either the last ballot cast by a conscientious voter, a ballot cast by an offline voter or a ballot from a dishonest voter, each voter being in correspondence with at most one ballot.

In Vote&Check some properties come for free: the voter *sees* their vote on their ballot hence they are guaranteed that their ballot contains their intended vote. Moreover, anyone can count the votes on the ballot box since they appear in clear.

Since every conscientious voter can verify that their vote stands on the board next to their tracker $\mathsf{hash}(c_i, t_i)$ and their anti-clash number n_i, then individual verifiability is guaranteed in Vote&Check, even if all authorities are dishonest. To obtain end-to-end verifiability, one must also control that no votes can be added for offline voters (or absentee voters). In Vote&Check, end-to-end verifiability is guaranteed only if both the Printer and the Election Office are honest. Let us explain why this is the case. If the Printer is compromised, since they have all the voting material, they can vote in place of absentees. Also, if the Election Office is corrupted, they can record every cast vote v_i into another vote v'_i. If the voter V_i does not perform the verification, this will be undetected. However, this attack no longer works if every voter that cast a ballot verifies.

In order to show that these are the only obstructions to end-to-end verifiability, we introduce a variant of universal verifiability, called *weak universal verifiability* that now tolerates that votes from offline voters can be modified. Then Vote&Check guarantees weak verifiability as long as the Printer is honest.

We recapitulate our security claims on Figure 4.4.

4.4 Discussion

Vote&Check achieves a better level of verifiability and privacy than STROBE, RemoteVote and SAFE Vote, while keeping a similar infrastructure: all protocols require a printer and a collecting authority (the Election Office). In Vote&Check, we introduce a Tracker Server, that replaces the set of decryption authorities used in STROBE, RemoteVote and SAFE Vote. Compared to Devillez et al, the security level is similar. However, we reduce significantly the number of independent authorities (from 4 + a set of decryption authorities to 3) and we recover full universal verifiability (no proxy).

Vote&Check remains however subject to several attacks.

Privacy breach by complaints. This attack seems unavoidable as soon as the votes are sent in clear, which is an important feature for usability. The Election Office can selectively remove

ballots that vote for a certain candidate A and see who complains. To circumvent this issue, one would need to provide a complaint mechanism that preserves vote privacy, e.g., encouraging voters to launch false complaints in order to hide true ones. But this also requires a robust accountability mechanism to avoid accepting wrong complaints. So in short, there is no easy solution to prevent privacy breach by complaints.

Weak eligibility. The printer in charge of printing and sending the material to voters may vote in place of absentees. To improve on this, one could use several printers that each sends a part of the voting material. Such an approach has cost and usability issues. Alternatively, voters may be requested to write down some authentication token, obtained for example through a mobile application. This however means that the solution is no longer purely paper based, which may cause some usability issue.

Alter votes of non verifying voters. The Election Office may always drop some ballots. If the corresponding voters do not check, this will remain undetected. This is indistinguishable from the case where the corresponding voters did not vote. However, in Vote&Check as well as all the other protocols under study, the Election Office may not only drop but also simply change a vote for another candidate that they prefer. Here, there could be a room for improvement, for example binding an authenticated secret data to each voting option. The Election Office would not know the secret associated to another vote. This should be carefully design to preserve vote privacy and usability.

Limitations. As many existing voting schemes, Vote&Check is not *accountable*: when a voter detects an issue, there is no mechanism to identify who misbehaved. Such mechanisms exist in some protocols (e.g., sElect [KMST16], Themis [BCC+22b]) but, they still have limitations: they only identify which authority misbehaved, assuming the voter behaves honestly. When considering cast-as-intended, a voter may lie by voting for candidate A and then pretending they have voted for B and complaining accordingly. Given that voters typically cannot apply any cryptographic mechanism (only their possibly corrupted device may perform such operations), it is hard to distinguish between an honest voter who has been attacked and a dishonest voter who wishes to diminish the trust in the election. Such an issue is very challenging and still largely unaddressed, even for Internet voting protocols.

Another limitation of Vote&Check is that it is subject to coercion attacks. Indeed, a coerced voter may be instructed to choose a particular value n to be written on their ballot and to vote for A. Then the coercer expects to see A, n on the Board. Resisting against vote selling is difficult in the context of postal voting. Indeed, a voter may typically provide their voting material to the coercer. One possible direction would be to design a protocol such that the voters can forge voting material, whose corresponding votes will, undetectably, be removed during the tally.

Altogether, and even if not perfect, the recent proposals for postal voting show that it is possible to significantly improve postal voting in terms of verifiability and vote privacy.

Chapter 5

Security analysis of Vote&Check

In this chapter, we present our security analysis of Vote&Check. We conduct the analysis of our protocol using the ProVerif tool presented in Section 2.1. This choice was motivated by the dedicated framework [CDC23] for proving end-to-end verifiability of voting protocols. For privacy, we had to handle the fact that complaints break privacy, as indicated in Section 3.2. Indeed, a compromised Election Office can figure out who cast a set of ballot by intentionally dropping them and listening who complains later. We instead show that our protocol preserves privacy provided that complaints can be made anonymously, for example through a trusted third party (a judge). For verifiability, Vote&Check suffers from limitations due to the postal voting setting. For example, the voters cannot perform cryptographic computations, and thus cannot sign their ballots, making possible for a corrupt Election Office to alter ballots. Therefore, we introduced a weaker and more fine-grained property than universal verifiability, *weak universal verifiability*. Finally, we had to circumvent a limitation of the voting framework. Even if it is meant to be as general as possible, the framework implicitly assumes that every ballot can be linked to its voter through a voter identifier. However, there is no such identifying data in Vote&Check. Instead, we show how to use the credential and the tracker, with a flexible association that can depend on the trust assumptions. We also had to circumvent the fact that ProVerif cannot easily reason on "else" branches and we introduced a new axiom, for which we provide a proof of correctness.

5.1 The verifiability framework

The ProVerif verifiability framework [CDC23] helps to analyse a voting protocol in different ways. First, it provides a fill-in-the-blank process to model every part of the protocol. In particular, it models precisely the tally phase that was usually abstracted because of the complexity for mechanical tools to count all the ballots. It also provides two turnkey queries and a proof that they imply end-to-end verifiability. For most protocol examples, the execution of ProVerif on the resulting modeling file would not terminate, so the framework comes with lemmas, secrecy assumptions and resolution strategy tuning to avoid infinite loops.

5.1.1 Processes

The verifiability framework is made of generic processes and protocol-dependent processes. The user must keep the former unchanged and specify the latter. For example, the (generic) `Voter` process calls the following (specific) `Voting` and `Final_Check` processes.

```
1  let Voter =
2    !
3      in(c_att, vote);
4      get voting_data_table(voter_index, voting_data) in
5        Voting(voter_index, vote, voting_data)
6      |
7        in(res_voting(voter_index), end_data);
8        in(c_att, is_last: boolean);
9        if is_last then
10       Final_Check(voter_index, vote, voting_data, end_data).
```

Not only the attacker schedules the execution of the protocol, but they can also control its flow through inputs on the dedicated channel `c_att`. For instance, on lines 3 and 8, the attacker chooses the vote that the voter will cast, and whether they will revote or perform the verification. Beside `Voting` and `Final_Check`, the user also chooses how the table `voting_data_table` is filled. This table contains the voting data of the voter, for example a blank ballot, a voting token, a signing key, etc. Later, the `Voting` process takes it as input.

5.1.2 Properties

Verifiability properties are implemented in the framework thanks to events and injective correspondence queries (see Chapter 2). The events are placed in the protocol to keep track of what happened during the execution. The event `Verified(V, v)` is triggered at the end of the verification process, for a voter V and a vote v. The event `Counted(v)` is executed during the tally phase, after the vote v is processed, and `Finish` is executed at the end of the tally phase, when all ballots are processed. As explained in Section 4.1.1, honest voters are either conscientious (they perform the verification procedure) or offline (they skip all optional steps). Each time the attacker generates a new voter V, they choose to make it either a honest conscientious voter, a honest offline voter or a malicious voter, this executes respectively the event `HV(V)`, `HNV(V)` or `Corrupt(V)`.

The authors of [CDC23] showed that proving end-to-end verifiability is equivalent to prove two queries:

- **Individual verifiability** requires that if a voter conducts the verification procedure, then their ballot will eventually be counted.

```
event(Finish) && inj-event(Verified(v_id,v)) ==> inj-event(Counted(v))
```

- **Universal verifiability** states that every counted vote v comes either from a honest voter that voted v or from a corrupt voter.

```
event(Finish) && inj-event(Counted(v)) ==>
     inj-event(HV(v_id)) && event(Verified(v_id,v))
  || inj-event(HNV(v_id)) && event(Voted(v_id,v))
  || inj-event(Corrupt(v_id))
```

In protocols with revote, the event `Voted` may be triggered several times per voter, whereas the `Verified` event is unique for each voter. In the case of the conscientious voters, we only want their last vote to be counted, the one that they will look for during the verification phase. On the other hand, offline voters do not perform the verification so any of their votes can be counted, hence the difference of handling.

5.2 Model

5.2.1 Agents

Every agent of the protocol is represented by one process in the model. We model the Election Office, the Printer, the Tracker Server, an unbounded number of voters and the postal service. The first three can either be honest or corrupt, depending on the threat model. The attacker controls all the voters, except the ones considered by the privacy and verifiability properties. Finally, the postal service is assumed honest.

```
1  let Voting(voter_index, vote, voting_data) =
2    let _ = only_once(OO_Voting(voter_index)) in (* the voter votes only once *)
3
4    (* In our protocol, voting data include:
5    - the physical key of the letterbox
6    - nonces a_tracker_serv and t from the tracker server
7    - the voter's favourite number n *)
8    let (voter_letterbox, a_tracker_serv, t, n) = voting_data in
9
10   (* the voter receives and opens their envelope *)
11   in(voter_letterbox, (a, c, sig1, sig2, sig3));
12
13   if check1(sig1, a, pk_printer) then
14   if check2(sig2, (a, c), pk_printer) then
15   if check3(sig3, (a, c, voter_index), pk_printer) then
16   if a = a_tracker_serv then (* Check that the identifier on the ballot is correct *)
17
18   event Voted(voter_index, vote);
19     out(deposit_letterbox, (a, c, sig1, sig2, vote, n)) (* Cast their ballot *)
20   |
21     out(res_voting(voter_index), (c, t, n)).
```

The specific process `Voting` includes all aspects related to Vote&Check. The voter receives their material, verifies that it is well formed and casts their vote. The `res_voting` channel passes the information from the `Voting` process to the `Final_Check` process. The line 2 ensures that every voter votes only once and will be detailed in Section 5.2.4.

5.2.2 Communication between agents

The communication between agents is modeled by *channels*. A channel can either be public or private. In the first case, the attacker has full control on the inputs and outputs on the channel, ie can intercept any message and inject any message they can build. In the second case, the attacker can block messages but cannot access their content, nor can inject messages: private channel are secret and authenticated. It is important to note that in the ProVerif semantics (see Section 2.1), input/output pairs are executed atomically. An immediate consequence is that an isolated input (resp. output) on a private channel is blocking, until an output (resp. input) on the same channel is reached somewhere else in the process. Also, each output message can be read only once. If we want several processes to access the same message on a private channel, we have to publish this exact amount of copies. These issues do not appear for public channel, since the attacker can read or inject messages to unlock the flow, or copy messages they want to replay.

In our protocol, voters receive and send messages over the post. Since we assume a honest post, we do not give the attacker access to these messages. However, an attacker can drop

anything in any mailbox. The postal channel is secret but is not authenticated. We choose to reflect this using the following process:

```
free c_att: channel.
free deposit_letterbox: channel [private].
!(in(c_att, x: bitstring); out(deposit_letterbox, x))
```

In our model, `c_att` is the public channel used to model the interactions between the attacker and the other agents. Due to the !, the third line can be replicated any number of times. It takes as input any message from the attacker, and outputs in on the voter's letterbox.

Conversly, the bulletin board is authenticated, public and append-only. We want the attacker to be able to read its messages, but not to inject ones. Thus, we also use a private channel with a dedicated process. For example, the `token_list` channel is the subpart of the bulletin board where the Printer publishes the valid voting tokens a_i.

```
free c_att: channel.
free token_list: channel [private].
!(in(token_list, x: bitstring); out(c_att, x); out(token_list, x))
```

Here, the third line takes as input any voting token and gives it to the attacker. Then, it publishes back on the `token_list` channel so that the honest users can access it again. Note that the message is outputted back on the channel, hence this does not affect the "only one in per out" rule.

5.2.3 Modeling corruption

Usually, when an entity is corrupt, we simply give access to the attacker to all their private data and channels:

```
let Printer(c_TrackingServer_to_Printer: channel, sk_Printer: private_key) =
  if printer_status = corrupt then
  out(c_att, c_TrackingServer_to_Printer);
  out(c_att, sk_Printer)
  else
  ...
```

However, in some cases, this construction is not precise enough. For example, in Vote&Check, even if the Election Office is corrupt, they cannot publish everything on the bulletin board. Indeed, auditors are inspecting the board and will report any irregularity. More precisely, they will check that the number of published ballot $(\mathsf{hash}(c,t),v,n)$ is the same as the number of published signatures $\mathsf{sig}(a,\mathsf{sk}_{\texttt{Printer}})$, and that those signatures are valid. To enforce it, the Printer publishes these signatures on the dedicated private channel `token_list`.

```
let ElectionOffice =
  if electionoffice_status = corrupt then
    ...
  else
    in(c_electionoffice_mailbox, (a, c, sig1, sig2, v, n)); (* Receives a ballot *)
  ...
    in(token_list, =a); (* Verifies that the token is valid *)
  ...
    insert ballot_box(ballot); (* Publishes the ballot *)
    out(ch_pub, (a, sig1)).
```

Everytime the honest Election Office receives a ballot, they also input a token from `token_list`. Per the Proverif semantics, each output can be used only once, so this token is "consumed" and will not be available for other ballots. Afterwards, they publishes the corresponding signature

so that the auditors can perform this check. If the Election Office is compromised, the attacker should be able to insert ballots in the ballot box, but at most one per valid signature they can exhibit.

```
let ElectionOffice =
  if electionoffice_status = corrupt then
    out(c_att, c_electionoffice_mailbox);
    out(c_att, c_trackerserver_to_electionoffice)
    !(
      in(token_list, a);
      in(c_att, sig: signature1);
      if check(sig, a, pk_printer) then
      in(c_att, ballot);
      insert ballot_box(ballot);
    )
  else
  ...
```

The channels c_trackerserver_to_electionoffice and c_electionoffice_mailbox are private channels the Election Office can access to communicate with the Tracker Server and to receive their ballots. Then, when dishonest, the Election Office can also insert ballot in the ballot box, but everytime they do, they must exhibit a valid signature and consume the corresponding token.

5.2.4 Control flow

In Vote&Check, some actions can be performed only once, or only once per voter. For example, the tracker server registers each voter only once. Also, each voter votes only once and verifies only once and Printer sends only one ballot to each voter. By default, if we do not specify, ProVerif overapproximates and will look for attacks that use actions any number of times. In our case, it would lead to a false attack. For example, if a voter can vote twice, and if the Printer can send the voting material twice, then the attacker can distinguish `Voter(Alice,0)` | `Voter(Bob,1)` | S and `Voter(Alice,1)` | `Voter(Bob,0)` | S by simply having Alice voting twice and observing the outcome. ProVerif precision can be improved using restrictions. Intuitively, only the traces that fulfil all the restrictions will be considered to verify the properties.

```
let TrackerServer(V: voter) =
  new st: timestamp; (* generates a fresh timestamp *)
  event OnlyOnce_TrackerServer(st, V);
  if tracker_server_status = corrupt then
  ... (* outputs its secret *)
  else
  ... (* generates and sends the tracker *)

restriction st1, st2: timestamp, V: voter;
  event(OnlyOnce_TrackerServer(st1, V)) &&
  event(OnlyOnce_TrackerServer(st2, V)) ==> st1 = st2.
```

Here, we forbid the Tracker Server to send two trackers to a single voter by drawing a fresh nonce st every time it registers a voter, and by stating through the restriction that a voter cannot be registered at two different timestamps.

5.3 Privacy

We model vote privacy using the definition from Section 1.2.1. Intuitively, vote privacy is preserved if an attacker cannot distinguish the case where Alice votes 0 and Bob votes 1 from

the case where the two votes are swapped [DKR07]. This can be written

$$\texttt{Voter(Alice,0)} \mid \texttt{Voter(Bob,1)} \mid S \approx_{\text{obs}} \texttt{Voter(Alice,1)} \mid \texttt{Voter(Bob,0)} \mid S$$

where S represents the overall system that runs in parallel with the processes of Alice and Bob. The relation $P \approx_{\text{obs}} Q$ is observational equivalence [AF01] that intuitively states that an adversary cannot distinguish P from Q. The privacy property can equivalently be written in ProVerif as follows

$$\texttt{Voter(Alice, choice[0,1])} \mid \texttt{Voter(Bob, choice[1,0])} \mid S$$

in a way that highlights the only few differences from the two processes. If ProVerif returns true, then the process instantiated with the left part of the choice operator is observationally equivalent to the process instantiated with the right part of the choice operator.

Anonymous complaints. As explained in Section 4.3.1, Vote&Check is subject to privacy breach by complaints: the Election Office or the postman in charge of delivering the cast ballots could destroy some ballots for a certain candidate A and observe who complains. We hence prove privacy under the assumption that voters complain *anonymously*. This is modelled by a "double complaint": in the same way that the vote of Alice is protected by Bob voting in another way, the privacy of a complaining Alice is protected by a complaining Bob voting in another way. Hence, instead of writing the verification in the voter process, we create a `Verification` process where Alice and Bob simultaneously check their ballots and a complaint is raised if any of the two checks fail, without letting the attacker knows which test failed.

```
1  let Voting(voter, vote) =
2    in(voter_letterbox(voter), (a, c, sig1, sig2, sig3)); (* The voter gets their ballot in their
         postbox *)
3    if check(sig1, a, pk(sk_Pri)) then
4    if check(sig2, (a, c), pk(sk_Pri)) then
5    if check(sig3, (a, c, voter), pk(sk_Pri)) then
6    in(trackerserver_to_voter(voter), (a', t)); (* Get the verification material from the Tracker
         Server *)
7    if a = a' then (* Check that the identifier is correct *)
8    new n;
9    out(deposit_letterbox, (a, c, sig1, sig2, vote, n)); (* Send their ballot *)
10   out(voter_to_verifier(voter), (hash(c, t), vote, n)).
11
12 let Verification =
13     in(voter_to_verifier(Alice), (hashA, vA, nA));
14     in(voter_to_verifier(Bob), (hashB, vB, nB));
15     in(bulletin_board, (hash0, v0, n0));
16     in(bulletin_board, (hash1, v1, n1));
17     if (hashA, vA, nA) = (hash0, v0, n0) &&
18         (hashB, vB, nB) = (hash1, v1, n1)
19       ||
20         (hashA, vA, nA) = (hash1, v1, n1) &&
21         (hashB, vB, nB) = (hash0, v0, n0)
22     then event Happy(Alice); event Happy(Bob)
23     else out(public, complaint).
```

After having voted, Alice and Bob passes their verification information $(\mathsf{hash}(c,t), v, n)$ to the fictive `Verification` process. This process looks at the Bulletin Board and uses the data received on channels `voter_to_verifier(Alice)` and `voter_to_verifier(Bob)` to perform the verifications. The disjunction in lines 18-21 handles the fact that the ballots of Alice and Bob may appear in any order on the Bulletin Board.

Offline voters. Voters may not want to contact the Tracker Server before casting their vote. In that case, they may be subject to a targeted privacy attack if the Printer is dishonest. Indeed, assume that the attacker wants to learn Alice's vote. A dishonest Printer could print official voting material with Alice's name on it but with Charlie's pseudonym `a'`. Then the dishonest voter Charlie could use his own verification mechanism, and in particular his tracker `t'` to find out Alice's vote that will be associated to `hash(c,t')`. We show however that this is the only additional risk w.r.t. privacy for offline voters. That is, we prove privacy for offline voters when the Printer is honest and either the Tracker Server or the Election Office is honest. Offline voters are easily modelled by removing all optional checks (process `Verification` and line 7 of process `Voting`).

Multiple permutations. In Vote&Check, vote privacy is not ensured through cryptographic mechanisms such as encryption. Instead, it relies on multiple shuffles:

- the ballots are implicitly shuffled by the postal services when the voters send back their ballots to the Election Office. This prevents the Election Office from learning who sent what;
- the Printer and the Tracker Server shuffle respectively the a_i and the (a_i, t_i) (possibly by sorting them alphabetically). This prevents the Election Office from linking the ballots to the actual identities;
- the Election Office publishes the shuffled set of valid ballots $\mathcal{B}$ and the shuffled set of valid signed voting tokens $\mathcal{P}$. They use a different shuffle, which prevents the Tracker Server and the Printer from linking the votes to the voting material.

All these shuffles need to be properly modelled to prove privacy. This can be easily done taking advantage of the non-determinism in ProVerif. For example, the following process receives two messages on a channel c and outputs them in some non-deterministic order:

$$\texttt{in(c,x}_1\texttt{);in(c,x}_2\texttt{); (out(c,x}_1\texttt{) | out(c,x}_2\texttt{))}.$$

An attacker does not know, *a priori*, if x_1 is output first or second.

The issue is that ProVerif actually does not prove observational equivalence. Instead, it proves *diff-equivalence* [BAF05], a stronger notion that checks, step by step that the two processes take exactly the same action. In particular, if ProVerif has to prove $P_1 \mid P_2 \approx Q_1 \mid Q_2$, it will instead try to prove that any action of P_1 can be mapped to an action of Q_1 (and not Q_2) and that any action of P_2 can be mapped to an action of Q_2 (and not Q_1). Intuitively, ProVerif chooses an arbitrary scheduler to resolve non-determinism when proving diff-equivalence. Even if this proof strategy is sound, it may lead to false attacks. One can try to solve this issue by asking ProVerif to prove $P_1 \approx_{\texttt{obs}} P_2 \approx_{\texttt{diff}} Q_2 \approx_{\texttt{obs}} Q_1$ instead, if one thinks that the resulting mapping will prevent reaching false attack. This *swapping* technique (Q_1 and Q_2 are swapped) was firstly introduced by Delaune et. al. [DRS08] and then proved sound by Blanchet and Smyth [BS16]. We isolated the two voter processes of Alice and Bob (for which privacy is proved) and applied *swaps* when necessary. We let the other voter processes unchanged (since they are not critical to prove vote secrecy and thus do not lead to false attacks).

5.4 Verifiability

Identifying public identifiers. Although the verifiability framework is meant to be as general as possible, we had to tweak our model to make it fit in. In particular, the framework

assumes that each voter can be associated to a *public identifier* and that this public identifier can then be extracted from a ballot. The public identifier is typically the signing key of the voter or another form of voting credential. However, in Vote&Check, there is no such public identifier. A natural candidate is the credential c provided by the Printer. However, there are immediately two issues. First, c cannot be read from a ballot of the form h, v, n since it is hidden by the hash. Second, when the Printer is dishonest, the credential can no longer be trusted. Indeed, a dishonest Printer could give the same credential to several voters, in order to try to perform a clash attack. Instead, one could reason on the tracker t provided by the Tracker Server. But again, the tracker cannot be trusted if the Tracker Server is compromised. Actually, when both the Printer and the Tracker Server are compromised, only the fresh nonce chosen by the voter still provide some individual verifiability.

To circumvent this issue, we proceed in two steps:

1. We define as public identifier all material that is used by the voter, namely $a, \mathsf{hash}(c, t), n$. Note that it also includes n, the nonce generated by the voter. This identifier is added in a table `public_identifier` by the framework.

2. We provide a dynamic association between ballots and public identifiers, depending on the honesty status of each party. Intuitively, this association is computed "magically", knowing all the private information of (honest) participants.

We believe that our approach is rather systematic and could be applied on other voting contexts, when the notion of public identifier is a blurry concept.

In Vote&Check, we compute this association through the function `get_ident_from_ballot` that associates an identifier to a ballot $a, \mathsf{hash}(c, t), v, n$. Note that a and $\mathsf{hash}(c, t), v, n$ are published separately on the bulletin board. When all parties behave honestly, `get_ident_from_ballot` returns $a, \mathsf{hash}(c, t), n$ as expected. When the Printer is honest and the other parties might be dishonest, given a ballot $a_1, \mathsf{hash}(c, t_1), v, n_1$, we look for a voter that used the credential c provided by the Printer, with an identifier of the form $a_2, \mathsf{hash}(c, t_2), v, n_2$ and we return $a_2, \mathsf{hash}(c, t_2), v, n_2$ since this ballot must have been built by the voter that received c, hence identifier by $a_2, \mathsf{hash}(c, t_2), v, n_2$. In case no such voter exist (for example, c has been given to a dishonest voter), then we return the identifier associated to a_1. This is our default case: any accepted ballot can be associated to one of the a's since the number of accepted ballots is bounded by the number of (signed) a's. This property is enforced by the Auditors.

```
1  letfun get_ident_from_ballot(ballot) =
2    let ballot_of_bit(a1, h1, v, n1) = ballot in
3    let c1 = getc(h1) in (* returns c such that h=hash(c,_) *)
4    let t1 = gett(h1) in (* returns t such that h=hash(_,t) *)
5
6    if printer_status = honest then (
7        get public_identifier(_, ident_of_triplet(a2, h2, n2)) suchthat c1 = getc(h2) in
8        ident_of_triplet(a2, h2, n2)
9        else
10     get public_identifier(_, ident_of_triplet(a2, h2, n2)) suchthat a1 = a2 in
11       ident_of_triplet(a2, h2, n2)
12       else dummy_ident (* this should never happen *)
13   )
14   else
15
16   ... (* all other corruption cases *)
17
18     else dummy_ident (* this should never happen *)
```

Since `get_ident_from_ballot` is not a concrete function used by the agents but rather a fictive function used only for modeling, we introduce private functions `getc` and `gett` that reverse the `hash` function.

Weak universal verifiability. Vote&Check achieves E2E verifiability, that is individual and universal verifiability, when both the Printer and the Election Office are honest. Unfortunately, universal verifiability no longer holds when the Election Office is dishonest. Indeed, when receiving a ballot from an offline voter, it could easily modify the candidate' name. Note that the Election Office does not know the origin of a ballot, so it has to bet that the corresponding voter will not check. We therefore consider a weaker version of universal verifiability, where an attacker is allowed to modify the votes of voters who do not verify. This notion of verifiability is considered in [KTV10a] for example.

```
event(Finish()) && inj-event(Counted(v)) ==>
  inj-event(HV(v_id)) && event(Verified(v_id,v))
  || inj-event(HNV(v_id)) && event(Voted(v_id,v')) (*v' instead of v*)
  || inj-event(Corrupt(v_id))
```

Vote&Check achieves weak universal verifiability as soon as the Printer is honest. Conversely, a dishonest Printer can always vote in the name of absentees and hence break weak universal verifiability.

Finally, individual verifiability is preserved in all cases: voters who verify are guaranteed that their votes will be counted. When all parties are corrupted, individual verifiability relies solely on the nonce chosen by the voters. However, if either the Printer or the Tracker Server is honest, we show that individual verifiability holds even if voters all use an empty nonce.

5.5 Increasing ProVerif precision

The verifiability framework [CDC23] comes with a library of lemmas that take care of proving individual and universal verifiability, two injective properties, thanks to a variety of counters and temporal constraints. However, we detected a small discrepancy in the framework, acknowledged by the authors. The framework distinguishes between three types of voters: conscientious voters, honest voters that do not verify (called here offline voters), and dishonest voters. However, this was inaccurately modelled by first generating a voter and then, later on, assigning it to one of the three types, at the adversary's choice. This actually led to a fourth type of voters: voters that were not yet assigned to any category but that the adversary could try to use to vote on their behalf. We corrected this issue by requiring that all voters are assigned to one of the three types.

Then a remaining issue is caused by our complex function `get_ident_from_ballot` that calls the table `public_identifier` multiple times, with many `else` branches. Due to its internal translation into first order logic, ProVerif does not properly capture the fact that, whenever a `else` branch is considered, the previous conditions in the `if` branches must be falsified. In particular, if some term t has been inserted in a table `tab` and if the term t satisfies some condition D that appears in a branch of the form `get tab(x) suchthat D ... else event E` then the event `E` can no longer be emitted in the trace. We prove this implication and we add as an axiom the conclusion of the implication, as soon as the assumptions can be proved by ProVerif (as a set of lemmas).

While ProVerif defines the operational semantics between configurations presented in Section 2.1, for sake of simplicity and understandability, we only consider abstract action traces

made of events and predicates that highlight specific actions. Mapping executions in ProVerif semantics to our abstract traces shall be straightforward.

5.5.1 Protocol

A protocol is defined by its (infinite) set of execution action traces. We note $\mathcal{F}_{\mathsf{ev}}$ a specific set of function symbols used to define events, and $\mathcal{T}bl$ a specific set of function symbols used to define tables. For our reasoning, we only need to consider some particular actions in a protocol execution.

More precisely, an *action trace* is a finite sequence of actions of the following form:

tr	:=	0	empty trace
	|	$\mathtt{event}(E(e(u_1,\ldots,u_n))).\mathsf{tr}$	event
	|	$\mathtt{insert}\ tbl(u_1,\ldots,u_n).\mathsf{tr}$	table insertion
	|	$\mathtt{getSucc}(D, tbl(u_1,\ldots,u_n)).\mathsf{tr}$	table get success
	|	$\mathtt{getFail}(D, tbl).\mathsf{tr}$	table get failure

where $u_1,\ldots,u_n \in \mathcal{T}(\Sigma_c,\mathcal{N}), e \in \mathcal{F}_{\mathsf{ev}}$ is an event symbol, $tbl \in \mathcal{T}bl$ is a table symbol, and $D \in \mathcal{T}(\Sigma, \mathcal{X} \cup \mathcal{N})$ is a term.

Intuitively, the term D represents the condition which appears in the ProVerif command `get tbl(x) suchthat D in P else Q`. Hence, D is a term that is expected to reduce to a boolean value.

An *execution action trace* is an action trace $\mathsf{tr} = \mathsf{tr}_1.\ldots.\mathsf{tr}_n$ such that for all $i \in \{1,\ldots,n\}$ we have:

- tr_i is an event; or
- tr_i is a table insertion; or
- $\mathsf{tr}_i = \mathtt{getSucc}(D, tbl(u_1,\ldots,u_n))$ and there exists an index j such that $j < i$ and $\mathsf{tr}_j = \mathtt{insert}\ tbl(u_1,\ldots,u_n)$ and $D\{x_1 \mapsto u_1,\ldots,x_n \mapsto u_n\} \Downarrow = \mathsf{true}$ where $\{x_1,\ldots,x_n\} = vars(D)$; or
- $\mathsf{tr}_i = \mathtt{getFail}(D, tbl)$ and for all indices $j < i$ if we have $\mathsf{tr}_j = \mathtt{insert}\ tbl(u_1,\ldots,u_n)$ then $D\{x_1 \mapsto u_1,\ldots,x_n \mapsto u_n\} \Downarrow = \mathsf{fail}$ where $\{x_1,\ldots,x_n\} = vars(D)$.

5.5.2 Queries

Even if ProVerif supports complex queries we only consider simple ones in our model. These last are expressive enough to state and prove our soundness result.

In what follows, we consider only six types of predicates:

- $\mathsf{p}{-}\mathsf{event}(e(t_1,\ldots,t_n))$ where $e \in \mathcal{F}_{\mathsf{ev}}$ and $t_1,\ldots,t_n \in \mathcal{T}(\Sigma_c,\mathcal{X})$,
- $\mathsf{p}{-}\mathsf{insert}(tbl(t_1,\ldots,t_n))$ where $tbl \in \mathcal{T}bl$ and $t_1,\ldots,t_n \in \mathcal{T}(\Sigma_c,\mathcal{X})$.
- $\mathsf{p}{-}\mathsf{getSucc}(tbl(t_1,\ldots,t_n))$ where $tbl \in \mathcal{T}bl$ and $t_1,\ldots,t_n \in \mathcal{T}(\Sigma_c,\mathcal{X})$.
- $\mathsf{p}{-}\mathsf{getFail}(D, tbl)$ where $tbl \in \mathcal{T}bl$ and $D \in \mathcal{T}(\Sigma,\mathcal{X})$.
- $u = v$ where $u, v \in \mathcal{T}(\Sigma,\mathcal{X})$.

- $i > j$ where $i, j \in \mathcal{V}_t$, $\mathcal{V}_t$ being a specific set of variables used to model timing annotations. It is important to note that these variables cannot appear in other predicates or events.

We define the satisfaction relation $\vdash$ as follows: given an execution action trace $\mathsf{tr} = \mathsf{tr}_1.\ldots.\mathsf{tr}_p$ and a grounding substitution σ, we write $\mathsf{tr}, \sigma \vdash \mathsf{p}{-}\mathsf{event}(e(t_1, \ldots, t_n))@i$ if $\mathsf{tr}_{i\sigma} = \mathtt{event}(e(t_1\sigma, \ldots, t_n\sigma))$. The satisfaction relation is defined similarly for $\mathsf{p}{-}\mathsf{insert}(\cdot)$, $\mathsf{p}{-}\mathsf{getSucc}(\cdot)$, and $\mathsf{p}{-}\mathsf{getFail}(\cdot, \cdot)$.

Note that the $\mathsf{p}{-}\mathsf{getSucc}(\cdot)$ predicate records only the table element used to pass the get. In contrast, to ease the formal development, the $\mathsf{p}{-}\mathsf{getFail}(\cdot, \cdot)$ keeps track of the condition D.

Encoding from ProVerif models If events are a standard command in ProVerif, it is not the case for the specific events that we have introduced. However, any ProVerif model can be easily annotated to correspond to the model described just before. To do so, in our model, any command `insert tbl(u1,...,un)` is preceded by a specific `event Insert(tbl(u1,...,un))` used to record it in the action trace. Similarly, an event `getSucc()` is placed at the very beginning of the positive branch of a `get` command to record the action. Also, a specific event `getFail()` is placed at the very beginning of the negative branch of a `get` command. However, unlike $\mathsf{p}{-}\mathsf{getSucc}(\cdot)$ that only record the entry used to pass the test, $\mathsf{p}{-}\mathsf{getFail}(\cdot, \cdot)$ predicate records the table but also the test applied to enter in the else branch. Unfortunately, ProVerif does not allow to encode such an element into an event. For sake on simplicity, we thus simply assume that the specific event is tagged with a fresh name to uniquely identify the `get` command that failed. This extra annotation is easy to implement. We give an example of how we annotate the `get_ident_from_ballot` function that we presented in Section 5.4.

```
1  letfun get_ident_from_ballot(ballot) =
2
3    let ballot_of_bit((a1, h1, v, n1)) = ballot in
4    let c1 = getc(h1) in
5    let t1 = gett(h1) in
6
7    new st: stamp;
8      event(Enter_GIFB_c(st,c));
9
10   if printer_status = honest then (* If the Printer is honest *)
11     get public_identifier( _, ident_of_triplet(a2, h2, n2)) suchthat c1 = getc(h2) in
12       event getSucc(ident_of_triplet(a2, h2, n2));
13       ident_of_triplet(a2, h2, n2)
14     else
15       event getFail(c1, st);
16       get public_identifier(_, ident_of_triplet(a2, h2, n2)) suchthat a1 = a2 in
17         event getSucc(ident_of_triplet(a2, h2, n2));
18         ident_of_triplet(a2, h2, n2)
19       else
20         dummy_ident
21   else
22   ... (* Other corruption cases *)
23
24   else dummy_ident.
```

First, we add the getSucc and getFail events that we raise when a table lookup is successful or not. In our model, we keep the name of the table implicit in the getSucc event since it is always about the same one. Similarly, we do not mention the destructor D in getFail. Then, we added an event on line to mark when get_ident_from_ballot is used. We will use this event in Section 5.5.3.

5.5.3 Soundness result

Intuitively, we state that if two events F_1 and F_2 guarantee that a term t has been successfully retrieved from a table *tbl* satisfying a condition D, then once F_1 and F_2 are executed in an action trace, it is no longer possible to take the else branch of a command of the form `get tbl(x) suchthat D in P else Q`.

Let $\mathcal{P}$ be a protocol, i.e. a set of execution action traces. We define some queries and an axiom:

- Let $\rho_1 = F_1@i_1 \wedge F_2@i_2 \Rightarrow \mathsf{p}{-}\mathsf{getSucc}(tbl(u_1,\ldots,u_n))@i_s$.
- Let $\rho_2 = \mathsf{p}{-}\mathsf{getSucc}(tbl(y_1,\ldots,y_n)) \Rightarrow D\{x_1 \mapsto y_1,\ldots,x_n \mapsto y_n\}\Downarrow = \mathsf{true}$.
- Let $\rho_3 = F_1@i_1 \wedge F_2@i_2 \Rightarrow i_1 > i_2$.
- Let $\rho_4 = F_1@i_1 \wedge F_2 \wedge \mathsf{p}{-}\mathsf{getFail}(D, tbl)@i_f \Rightarrow i_1 < i_f$.
- Let $ax = F_1@i_1 \wedge F_2 \wedge \mathsf{p}{-}\mathsf{getFail}(D, tbl)@i_f \Rightarrow \mathsf{false}$.

The first query means that if both F_1 and F_2 are executed, then a table access was succesful before. The second states that all elements in the table are well-formed. The third means that if both F_1 and F_2 happened, then F_2 happened before F_1. The last one states that if F_1 and F_2 are executed, the table access can only fail after F_1 was executed. Finaly, the axiom states that if F_1 and F_2 are executed, then the table access cannot fail.

Proposition 1. *If* $\mathcal{P} \vdash \rho_1 \wedge \rho_2 \wedge \rho_3 \wedge \rho_4$ *then* $\mathcal{P} \vdash ax$.

To apply this axiom, we instantiate F_1 and F_2 with events that depend of the corruption case. If, for example, the Printer is honest, we take $F_1 = \mathtt{event}(\mathsf{Enter_GIFB_c}(\mathrm{st},\mathrm{c}))$, an event raised everytime the `get_ident_from_ballot` function is called. We also take $F_2 = \mathtt{event}(\mathsf{Public_identifier}(i, \mathsf{ident_of_triplet}(a, \mathsf{hash}((c,t)), n)))$, raised when the voter with index i is being attributed the voter identifier $\mathsf{ident_of_triplet}(a, \mathsf{hash}((c,t)), n)$. In this case, we verify the queries $\rho_1, \rho_2, \rho_3, \rho_4$ using ProVerif. Then, per the axiom, the first get on line 11 will never fail.

Proof. Let $\mathsf{tr} \in \mathcal{P}$ be an execution action trace and σ a grounding substitution such that $\mathsf{tr}, \sigma \vdash F_1@i_1 \wedge F_2 \wedge \mathsf{p}{-}\mathsf{getFail}(D, tbl)@i_f$.

Since $\mathcal{P} \vdash \rho_1$ we know that there exists a grounding substitution σ' extending σ_α (σ_α is equal to σ up to an α-renaming of variables occurring in predicates and timing annotations) such that $\mathsf{tr}, \sigma' \vdash \mathsf{p}{-}\mathsf{getSucc}(tbl(u_1,\ldots,u_n))@i_s$. By definition of the satisfaction of the predicate, we have that $\mathsf{tr}_{i_s\sigma'} = \mathtt{getSucc}(_, tbl(u_1\sigma',\ldots,u_n\sigma'))$ and $i_s\sigma' \leq max(i_1^1\sigma', i_2^1\sigma') = max(i_1\sigma, i_2\sigma)$ (i_1^1 and i_2^1 being the timing annotation occurring in ρ_1). By definition of an execution action trace, we know that there exists $i_{\mathtt{insert}} \leq i_s\sigma'$ such that $\mathsf{tr}_{i_{\mathtt{insert}}} = \mathtt{insert}\ tbl(u_1\sigma',\ldots,u_n\sigma')$.

Moreover, because $\mathcal{P} \vdash \rho_2$ we have $\mathsf{tr}, \sigma'_\alpha \vdash \rho_2$ (where σ'_α is equal to σ', up to, as before, an α-renaming of variables occurring in predicates), and thus $D\{x_1 \mapsto u_1\sigma',\ldots,x_n \mapsto u_n\sigma'\}\Downarrow = \mathsf{true}$.

Let us show that $i_f\sigma' < i_1\sigma$. Applying the definition of the satisfaction relation to $\mathsf{tr}, \sigma \vdash \mathsf{p}{-}\mathsf{getFail}(D, tbl)@i_f$ and considering the table element $tbl(u_1\sigma',\ldots,u_n\sigma')$ we deduce that

$$
\begin{array}{lcll}
i_f\sigma' & < & i_{\mathtt{insert}} & \textit{(def. } \mathtt{getFail}(\cdot)\textit{)} \\
 & < & i_s\sigma' & \textit{(def. } \mathtt{getSucc}(\cdot)\textit{)} \\
 & < & max(i_1^1\sigma', i_2^1\sigma') & \textit{(def. } \Rightarrow\textit{)} \\
 & < & max(i_1\sigma, i_2\sigma) & \textit{(def of } \sigma'\textit{)} \\
 & < & i_1\sigma & \textit{(}max(i_1\sigma, i_2\sigma) = i_1\sigma \textit{ by } \rho_3\textit{)}
\end{array}
$$

We hence derive a contradiction with ρ_4.
We conclude that $tr, \sigma \vdash ax$, hence $\mathcal{P} \vdash ax$. □

5.6 Results

All our ProVerif files are available as supplementary material in [CDGL25]. The verifiability framework [CDC23] requires to use a special version of the ProVerif prover. We used the commit `cc4f8cde5` of the `improved_scope_lemma` branch, from the official development repository. All experiments were run on a standard laptop equipped with a 4-core Intel i7-8665U CPU, with 16 GB of RAM; the running times are given for this machine. The resulting analysis is displayed in Figure 5.1 and confirms our security claims. The common lemmas refer to the lemmas provided by the framework, meant to help proving verifiability, and our own lemmas needed to infer our new axiom.

Corruption scenario			**Privacy**		**Verifiability**			
Tracker server	Printer	Election office	Offline voters	Conscientious voters	Common lemmas	Universal	Weak univ.	Individual
H	H	H	✔ (12.3 s)	✔ (12.3 s)	✔ (9.0 s)	✔ (0.5 s)	✔ (0.6 s)	✔ (0.4 s)
C	H	H	✔ (15.7 s)	✔ (15.6 s)	✔ (9.5 s)	✔ (0.4 s)	✔ (0.5 s)	✔ (0.3 s)
H	C	H	✘	✔ (2.1 s)	✔ (26.4 s)	✘	✘	✔ (0.3 s)
H	H	C	✔ (0.4 s)	✔ (0.4 s)	✔ (39.6 s)	✘	✔ (3.9 s)	✔ (0.5 s)
H	C	C	✘	✘	✔ (6.2 s)	✘	✘	✔ (0.2 s)
C	H	C	✘	✘	✔ (45.0 s)	✘	✔ (2.9 s)	✔ (0.6 s)
C	C	H	✘	✘	✔ (32.3 s)	✘	✘	✔ (0.2 s)
C	C	C	✘	✘	✔ (8.1 s)	✘	✘	✔ (0.2 s)

Figure 5.1: Security properties. Each line gives a corruption scenarios, where H means that the entity is honest, and C means that it colludes with the attacker. For each property, the ✔ symbol means that it has been proven in ProVerif, in the given amount of time; the ✘ means that we know that there is an attack. The special ✔ symbol means that in this case, the individual verifiability holds only if different voters uses different n, which is weak, since n is short. Privacy holds no matter if n is short or long.

Vote&Check preserves vote privacy when no more than one authority is corrupt. It always provides individual verifiability: the votes of conscientious voters are counted. Full end-to-end verifiability, that is, votes of offline voters can only be dropped, requires an honest Printer and Election Office. If we relax to weak verifiability (votes of offline voters may be changed), then this property holds as soon as the Printer is honest. Indeed, the Printer possesses the entire voting material hence it can always vote for absentee voters.

5.7 Conclusion

Vote&Check is a relatively simple scheme compared to other protocols in the literature. However, its analysis required to define new security properties to account to take into account the specificity of the postal context: anonymous complains or weak universal verifiability. Usiung a tool allowed us to consider a lot of different corruption cases. Yet, even if we used a framework dedicated to the analysis of electronic voting, we still had to modify some processes, define a new query and even introduce a new lemma that we proved by hand. The security of Vote&Check partially relies on short nonces n, to avoid clash attacks. Therefore, the attacker could guess its value and win with a small but non-neglectable property. This weakness is hard to capture

in a ProVerif model. We study in the next section how to analyse more precisely this class of probabilistic protocols.

Part III

Properly proving probabilistic properties

Chapter 6

A framework for probabilistic protocols

Symbolic analysis allows to reason automatically on protocols. However, its high-confidence guarantees comes at the cost of some abstraction. For example, probabilistic choices are out of reach of the state-of-the-art provers. It is possible to model an agent drawing a nonce at random, but not taking a decision at random. The class of properties one can analyse is also limited. While the reachability properties of the form "if a ballot is accepted, then it comes from an honest voter" are feasible, it is difficult to prove "the attacker cannot forge more than n ballots with a probability greater than $\frac{1}{2^n}$". One can circumvent this issue with an ad-hoc construction [BCC+22b, CDGG23]. We propose a generic method to address those challenges. We begin this chapter by presenting the extension of applied pi-calculus to probabilistic protocols [CCK23]. Then, we define restricted classes of properties and protocols, on which we can easily reason. Finally, we propose a refinement of the model to enunciate a sufficient condition to bound the probability of victory of the adversary.

6.1 Syntax

We presented in Chapter 2 how to express non-randomised protocols in the applied pi-calculus. Since probabilistic processes are out-of-reach of this grammar, the authors of [CCK23] enrich it with a specific command $P +_p Q$, representing a probabilistic choice between two processes. Our model however deviates from [CCK23] for some aspects.

6.1.1 Messages

Public and private names, variables, constructors and destructors, may-fail terms, expressions, formulas, events and tables remain the same as in Chapter 2. We also consider $\mathcal{L} = \{\mathtt{sid}_1, \mathtt{sid}_2, \ldots\}$, a set of labels and $\mathcal{X}_\mathcal{L} \subseteq \mathcal{X}$, a subset of variables reserved for labels. Terms are built on names, variables and also labels. We note the set of terms $\mathbb{T}$. Additionally, given an expression D, we define the predicate Msg such that $\mathsf{Msg}(D) = \top$ if $D \Downarrow M$, where M is a term.

6.1.2 Process algebra

Protocols are modelled by processes and executions are formalised through an operational semantics $\mathtt{sem}_0$ over multisets of processes. The $\mathtt{syn}_0$ syntax for processes and extended processes

P, Q	$:=$		Processes
		0	null process
		$\texttt{let}\ x = D\ \texttt{in}\ P\ \texttt{else}\ Q$	conditional declaration
		$\texttt{new}\ n.P$	name restriction
		$\texttt{out}(N, M).P$	output
		$\texttt{in}(N, x).P$	input
		$P \mid Q$	parallel composition
		$P_1 +_{1/p} \ldots +_{1/p} P_p$	probabilistic choice
		$P_1 +^{\texttt{sid}}_{1/p} \ldots +^{\texttt{sid}}_{1/p} P_p$	labeled probabilistic choice
		$\texttt{event}(E(M_1, \ldots, M_k)).P$	event
		$\texttt{insert}\ t(M).P$	table insertion
		$\texttt{get}\ t(x)\ \texttt{suchthat}\ D\ \texttt{in}\ P\ \texttt{else}\ Q$	table access
V	$:=$		Extended processes
		P	process
		$\texttt{executed}(E(M_1, \ldots, M_k))$	event execution
		$\texttt{label}(w)$	labeling

where $k \in \mathbb{N}$, $M_1, \ldots, M_k, M, N$ are terms, D is an expression, n is a name, $x \in \mathcal{X}$ is a variable, $E \in \mathcal{F}_{Ev}$ is an event name, $t \in \mathcal{T}bl$ is a table name, $p \in \mathbb{N}^*$, $\texttt{sid} \in \mathcal{L} \cup \mathcal{X}_{\mathcal{L}}$ and $w \in (\mathcal{L} \times \mathbb{N})^*$.

Figure 6.1: Probabilistic protocols syntax

is defined in Figure 6.1. Note that there is no replication in $\texttt{syn}_0$.

We will present the semantics $\texttt{sem}_1$ later in this chapter. In $\texttt{sem}_1$, the extended process $\texttt{executed}(E(M_1, \ldots, M_k))$ records that the event E has been executed while $\texttt{label}(w)$ contains a list w of the form $(\texttt{sid}_1, i_1) :: \cdots :: (\texttt{sid}_n, i_n)$ that keeps track of all labeled probabilistic choices $\texttt{sid}_j$ that have been executed, with the corresponding branch i_j. Such extended processes $\texttt{label}(w)$ will not be used in our initial semantics.

For terms M and N and processes P and Q, we write $\texttt{if}\ M = N\ \texttt{then}\ P\ \texttt{else}\ Q$ as syntactic sugar for $\texttt{let}\ x = \texttt{If}(\texttt{Equals}(M, N))\ \texttt{in}\ P\ \texttt{else}\ Q$, as explained in Example 3.

We consider special event names choice and $\mathsf{inserted}_t$ for any $t \in \mathsf{Tab}$ that are reserved for our semantics and should not appear in processes.

We note $\mathcal{SP}$ (resp $\mathcal{SP}_{\mathsf{ext}}$) for the set of all *closed processes* (resp. *closed extended processes*), that is the set of processes (resp extended processes) generated by the previous grammar that do not contain free variables. We note $\mathcal{MP}$ (resp. $\mathcal{MP}_{\mathsf{ext}}$) for the set of all multisets over $\mathcal{SP}$ (resp. $\mathcal{SP}_{\mathsf{ext}}$).

Example 6. *We can express our running example in this syntax.*

```
1  Gamble =
2    (
3      in(c, x).
4      if x = 0 then
5      event(Happy(sid, x))
6    )
7  +^sid_1/2
8    (
9      in(c, x).
10     if x = 1 then
```

```
11      event(Happy(sid, x))
12    )
```

The Gamble *process draws* 0 *or* 1 *at random and compare it with an input from an other process. It can model a "heads or tails" protocol.*

6.2 A first semantics

In applied pi-calculus, a semantics is usually a transition system between the multisets of processes. However, we need to represent the probabilistic aspect of our extended processes. Therefore, [CCK23] considers distributions of probabilities over the multisets of extended processes instead.

Probability distributions Given a multiset $\mathcal{S}$, $\mathfrak{P}(\mathcal{S})$ denotes all finite subsets of $\mathcal{S}$ and $\mathcal{D}(\mathcal{S})$ denotes the set of all the finitely supported distributions D over $\mathcal{S}$. We note $\mathsf{supp}(D)$ its support, i.e. $\mathsf{supp}(D) = \{x \in \mathcal{S} \mid D(x) > 0\}$. Moreover, given two distributions $D, E \in \mathcal{D}(\mathcal{S})$, for $p, q \geq 0$ such that $p + q = 1$, we note $(p \cdot D + q \cdot E)(x) = p \cdot D(x) + q \cdot E(x)$. For sake of simplicity, if $q = 0$ we write $p \cdot D$ instead of $p \cdot D + 0 \cdot E$. Finally, we denote by δ_x the Dirac distribution on x.

Semantics Extended processes are executed through an operational semantics $\mathtt{sem}_0$ between a multiset of extended processes and a probability distribution over the multisets of extended processes. We denote by $\mathcal{P} \to_0 \mu$ a step in the operational semantics. Formally, each command executes as follows:

$$\mathcal{P} \uplus \{\!\{0\}\!\} \to_0 \delta_{\mathcal{P}} \qquad \text{(Null)}$$

$$\mathcal{P} \uplus \{\!\{\mathtt{new}\ n.P\}\!\} \to_0 \delta_{\mathcal{P} \uplus \{\!\{P\{n \mapsto n'\}\}\!\}} \quad \text{where } n' \in \mathcal{N}_{\mathsf{priv}} \text{ is fresh} \qquad \text{(New)}$$

$$\mathcal{P} \uplus \{\!\{\mathtt{let}\ x = D\ \mathtt{in}\ P\ \mathtt{else}\ Q\}\!\} \to_0 \delta_{\mathcal{P} \uplus \{\!\{P\{x \mapsto M\}\}\!\}} \quad \text{if } D \Downarrow M \text{ and } x \in \mathcal{X}_{\mathcal{L}} \Rightarrow M \in \mathcal{L} \qquad \text{(Let-T)}$$

$$\mathcal{P} \uplus \{\!\{\mathtt{let}\ x = D\ \mathtt{in}\ P\ \mathtt{else}\ Q\}\!\} \to_0 \delta_{\mathcal{P} \uplus \{\!\{Q\}\!\}} \quad \text{otherwise} \qquad \text{(Let-E)}$$

$$\mathcal{P} \uplus \{\!\{\mathtt{out}(N, M).P, \mathtt{in}(N, x).Q\}\!\} \to_0 \delta_{\mathcal{P} \uplus \{\!\{P, Q\{x \mapsto M\}\}\!\}} \quad \text{if } x \in \mathcal{X}_{\mathcal{L}} \Rightarrow M \in \mathcal{L} \qquad \text{(Comm)}$$

$$\mathcal{P} \uplus \{\!\{P \mid Q\}\!\} \to_0 \delta_{\mathcal{P} \uplus \{\!\{P, Q\}\!\}} \qquad \text{(Par)}$$

$$\mathcal{P} \uplus \{\!\{P_1 +_{1/p} \ldots +_{1/p} P_p\}\!\} \to_0 \sum_{i=1}^{p} 1/p \cdot \delta_{\mathcal{P} \uplus \{\!\{P_i\}\!\}} \qquad \text{(PChoice)}$$

$$\mathcal{P} \uplus \{\!\{P_1 +^{\mathtt{sid}}_{1/p} \ldots +^{\mathtt{sid}}_{1/p} P_p\}\!\} \to_0 \sum_{i=1}^{p} 1/p \cdot \delta_{\mathcal{P} \uplus \{\!\{P_i, \mathtt{executed}(\mathtt{choice}(\mathtt{sid}))\}\!\}} \qquad \text{(LPChoice)}$$

$$\mathcal{P} \uplus \{\!\{\mathtt{event}(E(M_1, \ldots, M_k)).P\}\!\} \to_0 \delta_{\mathcal{P} \uplus \{\!\{P, \mathtt{executed}(E(M_1, \ldots, M_k))\}\!\}} \qquad \text{(Event)}$$

$$\mathcal{P} \uplus \{\!\{\mathtt{insert}\ t(M).P\}\!\} \to_0 \delta_{p \uplus \{\!\{\mathtt{executed}(\mathsf{inserted}_t(M)), P\}\!\}} \qquad \text{(Insertion)}$$

$$\mathcal{P} \uplus \{\!\{\mathtt{get}\ t(x)\ \mathtt{suchthat}\ D\ \mathtt{in}\ P\ \mathtt{else}\ Q, \mathsf{inserted}_t(M)\}\!\} \to_0 \delta_{\mathcal{P} \uplus \{\!\{P\{x \mapsto M\}, \mathsf{inserted}_t(M)\}\!\}} \quad \text{if } D\sigma \Downarrow \mathsf{true} \text{ and } x\sigma = M \qquad \text{(Reading-1)}$$

$$\mathcal{P} \uplus \{\!\{\mathtt{get}\ t(x)\ \mathtt{suchthat}\ D\ \mathtt{in}\ P\ \mathtt{else}\ Q\}\!\} \to_0 \delta_{\mathcal{P} \uplus \{\!\{Q\}\!\}} \quad \text{if } \forall \sigma, \forall M, \mathsf{inserted}_t(M) \in \mathcal{P} \Rightarrow D\sigma \not\Downarrow \mathsf{true} \text{ or } x\sigma \neq M \qquad \text{(Reading-2)}$$

Figure 6.2: Semantics $\to_0$

We may write $\to$ instead of $\to_0$ when the chosen semantics is clear from the context. This semantics allows us to define a transition system.

Convenient representation To reason about the operational semantics, we introduce a convenient representation of it using transition systems. In the following, we call state a multiset of extended processes.

Definition 9 (NPTS)**.** *A* non-deterministic probabilistic transition system *is a pair* $(\mathcal{S}, \mathsf{trans})$ *where:*

- $\mathcal{S}$ *is a set of states;*
- $\mathsf{trans} : \mathcal{S} \to \mathfrak{P}(\mathcal{D}(\mathcal{S}))$ *is a transition function: for each state* $s \in \mathcal{S}, \mathsf{trans}(s)$ *is a set of finitely supported distributions over* $\mathcal{S}$*.*

The operational semantics $\mathtt{sem}_0$ previously defined can be represented by the NPTS $\mathsf{N}^o = (\mathcal{MP}, \mathsf{trans}^o)$ where $\mathsf{trans}^o(s) = \{D \mid s \to_0 D\}$. It is important to note that N^o contains all the possible multisets of extended processes and all their executions, and therefore is an infinite transition system.

To define reachability properties, we first need to resolve non-determinism in N^o. Informally, this step consists in defining a scheduler for the attacker. More precisely, it consists in extracting a probabilistic transition system from an NPTS. The resulting transition system is called a *reactive probabilistic transition system* (RPTS).

Definition 10 (RPTS)**.** *A* reactive probabilistic transition system *is a pair* $(\mathcal{S}, \mathsf{trans})$ *where:*

- $\mathcal{S}$ *is a set of states;*
- $\mathsf{trans} : \mathcal{S} \to (\mathcal{D}(\mathcal{S}) \cup \{\star\})$ *is a transition function: for each state* $s \in \mathcal{S}$*,* $\mathsf{trans}(s)$ *is either a finitely supported distribution over* $\mathcal{S}$ *or* $\star$*, a specific state used to model deadlocks.*

Definition 11 (Resolution)**.** *Let* $\mathsf{N} = (\mathcal{S}, \mathsf{trans})$ *be a NPTS. A* non-randomised resolution *for* N *is a RPTS* $R = (\mathcal{S}, \mathsf{trans}')$ *such that* $\forall s \in \mathcal{S}, \mathsf{trans}'(s) \in \mathsf{trans}(s) \cup \{\star\}$.

We note $\mathcal{R}(\mathsf{N}^0)$ the set of non-randomised resolutions for N^0. Once a resolution is chosen, the execution of the protocol only depends on the probabilistic choices.

Before defining the reachability properties in a non-deterministic probabilistic transition system, we first define the probability of reaching a state in a target set $\mathcal{T}$ from a state s in a reactive probabilistic transition system.

Definition 12. *Let* $R = (\mathcal{S}, \mathsf{trans})$ *be a RPTS,* $\mathcal{T} \subseteq \mathcal{S}$ *a set of states and* $s \in \mathcal{S}$*. For every* $n \in \mathbb{N}$*, we define the probability of reaching* $\mathcal{T}$ *from* s *in at most* n *steps as:*

$$\mathsf{RProb}_R^{\leq 0}(s, \mathcal{T}) = \begin{cases} 1 & \text{if } s \in \mathcal{T} \\ 0 & \text{otherwise} \end{cases}$$

$$\mathsf{RProb}_R^{\leq n+1}(s, \mathcal{T}) = \begin{cases} 1 & \text{if } s \in \mathcal{T} \\ 0 & \text{if } s \notin \mathcal{T} \wedge \mathsf{trans}(s) = \star \\ \Sigma_{u \in \mathsf{supp}(D)} D(u) \cdot \mathsf{RProb}_R^{\leq n}(u, \mathcal{T}) & \text{if } s \notin \mathcal{T} \wedge \mathsf{trans}(s) = D \end{cases}$$

We define the probability of reaching $\mathcal{T}$ *from* s *in* R *as:*

$$\mathsf{RProb}_R(s, \mathcal{T}) = \mathsf{lim}_{n \to +\infty} \mathsf{RProb}_R^{\leq n}(s, \mathcal{T}).$$

This limit exists since $\mathsf{RProb}_R^{\leq n}(s, \mathcal{T})$ *grows with* n *and is bounded by 1.*

Finally, we define the probability of reaching $\mathcal{T}$ from s in N^o as:

$$\mathsf{RProb}_{\mathsf{N}^o}(s, \mathcal{T}) = \mathsf{sup}\{\mathsf{RProb}_R(s, \mathcal{T}) \mid R \in \mathcal{R}(\mathsf{N}^o)\}.$$

Definition 12 is extended to a set of states as follows: the probability of reaching $\mathcal{T}$ from $\mathcal{S}$ in N^o is:

$$\mathsf{RProb}_{\mathsf{N}^o}(\mathcal{S}, \mathcal{T}) = \mathsf{sup}\{\mathsf{RProb}_{\mathsf{N}^o}(s, \mathcal{T}) \mid s \in \mathcal{S}\}.$$

We only consider states made of a multiset of finite (i.e., without replication) extended processes. Moreover, if we measure the size of a process as its depth as a tree, each step of the semantics reduces the size of the state. Therefore, we only reason about finite execution traces. Hence, for all states $s \in \mathcal{MP}$ and target set $\mathcal{T}$, there exists $n_{\mathsf{lim}} \in \mathbb{N}$ such that $\mathsf{lim}_{n \to +\infty} \mathsf{RProb}_R^{\leq n}(s, \mathcal{T}) = \mathsf{RProb}_R^{\leq n_{\mathsf{lim}}}(s, \mathcal{T})$.

From here, we deviate from [CCK23]. In our analysis, we only consider a subclass of reachability properties.

Definition 13. *Let $\mathcal{T}_f \subseteq \mathcal{MP}$, let ϕ be a formula. We say that $(\mathcal{T}_f, \phi)$ is a k-*simple *property with respect to the event* `happy` *if $\mathcal{T}_f = \{s \in \mathcal{MP} \mid \phi_{\mathsf{nd}}(s) \wedge \exists \mathtt{sid}_1, \ldots, \mathtt{sid}_k \in \mathcal{L}$ all distinct, $\exists M_1, \ldots, M_k \in \mathbb{T}, \forall i \in \{1, \ldots, k\}, \phi(u_i) \wedge \mathtt{executed}(\mathtt{happy}(\mathtt{sid}_i, M_i)) \in s\}$ where*

$$\phi_{\mathsf{nd}}(s) \equiv \forall \mathtt{sid} \in \mathcal{L}, (\mathtt{executed}(\mathsf{choice}(\mathtt{sid})) \in s \Leftrightarrow \exists M \in \mathbb{T}, \mathtt{executed}(\mathtt{happy}(\mathtt{sid}, M)) \in s).$$

Intuitively, in our framework, each `sid` represents a different user. Also, k is the number of users the attacker has to trick in order to win. The event $\mathtt{happy}(\mathtt{sid}, M)$ is triggered at the end of the process modeling the user indexed by `sid`. For example, in a voting protocol, `sid` can represent a voter and M their ballot and their vote. In this case, we trigger $\mathtt{happy}(\mathtt{sid}, (x, v))$ at the end of the verification process, with x the encrypted vote they cast and v their voting intention. The goal of the attacker is to flip votes without the affected voters noticing it, so we pose $\phi(x, v) =$ "$\mathtt{dec}(x) \neq v$". Hence, the property is satisfied as soon as the attacker tricked k voters.

Example 7. *We consider a simplified version of the Benaloh audit. Each voter enters their vote in their (corrupted) voting device, received a ciphertext and either cast it or audit it. A voter that audits their ciphertext will not cast any ballot.*

```
1  Voter(vote, sid) =
2    out(c, vote).
3    in(c, cipher).
4      (
5        in(c, rand).
6        if cipher = aenc(vote, rand, pkE) then
7        event Happy(sid, (vote, cipher))
8      )
9      +^sid_1/2
10     (
11       event Cast(cipher).
12       event Happy(sid, (vote, cipher))
13     )
```

Then, we consider the multiset of processes $\{\!\{\mathtt{Voter}(v_i, \mathtt{sid}_i), i \in \{1, \ldots, n\}\}\!\}$, with n a natural number. Our security property is defined by the ϕ we mentioned: $\phi(x, v) =$ "$\mathtt{dec}(x) \neq v$".

Finally, the ϕ_{nd} predicate represents the fact that once the attacker launched the process of a user, this user must be able to achieve their task. Otherwise, in our example, they would be able to start any number of voter processes, abort the ones that take the audit branch and only finishes the ones that take the cast branch. This behavior would lead to a probability of winning close to 1 but is not realistic.

Attacker. The attacker is modelled by a multiset of closed processes $\mathcal{P}_{\mathsf{att}}$, called *attacker process*, such that $names(\mathcal{P}_{\mathsf{att}}) \subseteq \mathcal{N}_{\mathsf{pub}}$ and $\mathcal{P}_{\mathsf{att}}$ does not contain events, table insertions, table accesses nor annotated probabilistic choices. We call $\mathcal{A}$ the set of attackers.

When analysing a security property, one needs to consider any initial state that satisfies some requirements. The protocol is represented by a multiset of processes, as well as the attacker.

Definition 14. *Let $\mathcal{P}$ be a multiset of extended processes. We call initial state s_{init} any multiset of processes $\mathcal{P}_{\mathsf{init}} = \mathcal{P} \uplus \{\!\{P_{\mathsf{att}}\}\!\}$, where P_{att} is an attacker process. We note $\mathcal{S}^0_{\mathsf{init}}(\mathcal{P})$ the (infinite) set of initial states. In particular, an initial state does not contain any extended process of the form* `executed()` *or* `label()`*. Then, for a protocol $\mathcal{P}$ and a reachability property $\mathcal{T}_f$, the probability for the attacker to win is $\mathsf{RProb}_{N^0}(\mathcal{S}^0_{\mathsf{init}}(\mathcal{P}), \mathcal{T}_f)$.*

6.3 Refining CCK model

In the next chapter, we will show that *k-simple* properties are implied by a completely deterministic property that we call deterministic security. Introducing this property however requires to refine our setting. To reason about the probabilistic choices, we introduce a new semantics $\mathtt{sem}_1$ that keeps track of the chosen branches. The rules of $\to_1$ are the same than $\to_0$ except that the rule (LPChoice) for labeled probabilistic choice is replaced by the (APChoice) rule:

$$\mathcal{P} \uplus \{\!\{P_1 +^{\mathtt{sid}}_p \ldots +^{\mathtt{sid}}_p P_p, \mathtt{label}(w)\}\!\} \to_1 \sum_{i=1}^{p} 1/p \cdot \delta_{\mathcal{P} \uplus \{\!\{P_i, \mathtt{executed}(\mathsf{choice}(\mathtt{sid})), \mathtt{label}(w::(\mathtt{sid},i))\}\!\}}$$

We may write $\to$ instead of $\to_1$ when the chosen semantics is clear from the context. This new semantics defines a NPTS. For all $s \in \mathcal{MP}, \mathsf{trans}^1(s) = \{D, s \to_1 D\}$. Then $N^1 = (\mathcal{MP}, \mathsf{trans}^1)$ is a NPTS that represents $\mathtt{sem}_1$. Thereby, we also have a notion of RPTS, reachability property and resolution. We note $\mathcal{R}(N^1)$ the set of non-randomised resolutions for N^1.

Definition 15 (Initial states)**.** *Let $\mathcal{P}$ be a multiset of extended processes. The set of initial states for $\mathcal{P}$ and $\to_1$ is denoted $\mathcal{S}^1_{\mathsf{init}}(\mathcal{P})$ and is the set of states of the form $\mathcal{P} \uplus \{\!\{P_{\mathsf{att}}\}\!\} \uplus \{\!\{\mathtt{label}(\epsilon)\}\!\}$ where P_{att} is an attacker process.*

Any extended process accessible from an initial state contains exactly one extended process of the form $\mathtt{label}(w)$ for some w.

Let $(\mathcal{T}, \phi)$ be a k-simple security property. This notion of reachability properties works for both $\mathtt{sem}_0$ and $\mathtt{sem}_1$. Moreover, the probability of reaching a target set is at most the same in $\mathtt{sem}_0$ as in $\mathtt{sem}_1$.

Proposition 2. *Let $(\mathcal{T}, \phi)$ be a k-simple security property. Let $\mathcal{P}$ be a traceable protocol. We have $\mathsf{RProb}_{N^0}(\mathcal{S}^0_{\mathsf{init}}(\mathcal{P}), \mathcal{T}) \leqslant \mathsf{RProb}_{N^1}(\mathcal{S}^1_{\mathsf{init}}(\mathcal{P}), \mathcal{T})$.*

Proof. We recall that for $i \in \{0,1\}, \mathsf{RProb}_{N^i}(\mathcal{S}^i_{\mathsf{init}}, \mathcal{T}) = \sup\{\mathsf{RProb}_R(s,\mathcal{T}) \mid R \in \mathcal{R}(N^i), s \in \mathcal{S}^i_{\mathsf{init}}\}$.
We also have, on the one hand, $\mathcal{S}^0_{\mathsf{init}}(\mathcal{P}) = \{\mathcal{P} \uplus \{P_{\mathsf{att}}\} \mid \mathcal{P}_{\mathsf{att}} \in \mathcal{A}\}$, and on the other hand, $\mathcal{S}^1_{\mathsf{init}}(\mathcal{P}) = \{\mathcal{P} \uplus \{P_{\mathsf{att}}\} \uplus \{\mathtt{label}(\epsilon)\} \mid \mathcal{P}_{\mathsf{att}} \in \mathcal{A}\}$.
Since $\mathcal{P}$ is a multiset of processes (and not of extended processes) and $\mathcal{P}_{\mathsf{att}}$ is an attacker process, states in $\mathcal{S}^0_{\mathsf{init}}(\mathcal{P})$ do not contain elements of the form $\mathtt{label}(w)$, and elements of $\mathcal{S}^1_{\mathsf{init}}(\mathcal{P})$ contains exactly one occurence of $\mathtt{label}(w)$, for $w = \epsilon$. Thus, we have a natural correspondence between the sets of initial states. For $s \in \mathcal{MP}$, if $s \in \mathcal{S}^0_{\mathsf{init}}(\mathcal{P})$, we have $s \uplus \{\mathtt{label}(\epsilon)\} \in \mathcal{S}^1_{\mathsf{init}}(\mathcal{P})$. Also, if $s \in \mathcal{S}^1_{\mathsf{init}}(\mathcal{P})$, then $s \setminus \{\mathtt{label}(\epsilon)\} \in \mathcal{S}^0_{\mathsf{init}}(\mathcal{P})$.

Let $s_0 \in \mathcal{S}^0_{\mathsf{init}}(\mathcal{P})$ and $R_0 = (\mathcal{S}_0, \mathsf{trans}_0) \in \mathcal{R}(N^0)$.
We will build $s_1 \in \mathcal{S}^1_{\mathsf{init}}(\mathcal{P})$ and $R_1 = (\mathcal{S}_1, \mathsf{trans}_1) \in \mathcal{R}(N^1)$ such that $\mathsf{RProb}_{R_0}(s_0,\mathcal{T}) = \mathsf{RProb}_{R_1}(s_1,\mathcal{T})$.
We pose $s_1 = s_0 \uplus \{\mathtt{label}(\epsilon)\}$.
We consider $\mathcal{S}'_0$, the subset of $\mathcal{S}_0$ of states that do not contain elements of the form $\mathtt{label}(w)$.
We pose $\mathcal{S}_1 = \{s \uplus \{\mathtt{label}(w)\} \mid s \in \mathcal{S}'_0, w \in (\mathcal{L} \times \mathbb{N})^*\}$.
For $s' = s \uplus \{\mathtt{label}(w)\} \in \mathcal{S}_1$, we define $\mathsf{trans}_1(s')$ as follows.
If $\mathsf{trans}_0(s) = \star$, then we pose $\mathsf{trans}_1(s') = \star$.
Otherwise, $\mathsf{trans}_0(s) \in \mathcal{D}(\mathcal{S}_0)$. More precisely, $\exists m \in \mathbb{N}^*, \exists u_1, \ldots, u_m \in \mathcal{S}_0, \mathsf{trans}_0(s) = \sum_{i=1}^m 1/m \cdot \delta_{u_i}$. For all $i \in \{1,\ldots,m\}$, for semantic reason, we have $u_i \in \mathcal{S}'_0$, so $u_i \uplus \{\mathtt{label}(w)\} \in \mathcal{S}_1$ and for all $\mathtt{sid} \in \mathcal{L}, u_i \uplus \{\mathtt{label}(w :: (\mathtt{sid}, i))\} \in \mathcal{S}_1$.
If D executes a probabilistic choice labeled by $\mathtt{sid}$, then we pose $\mathsf{trans}_1(s') = \sum_{i=1}^m 1/m \cdot \delta_{u_i \uplus \{\mathtt{label}(w::(\mathtt{sid},i))\}}$. Otherwise, we pose $\mathsf{trans}_1(s') = \sum_{i=1}^m 1/m \cdot \delta_{u_i \uplus \{\mathtt{label}(w)\}}$.

Now that R_1 is defined, we show that $\mathsf{RProb}_{R_0}(s_0,\mathcal{T}) = \mathsf{RProb}_{R_1}(s_1,\mathcal{T})$.
We proceed by induction. For all $n \in \mathbb{N}$, we pose A_n:

$$\forall s \in \mathcal{S}'_0, \forall w \in (\mathcal{L} \times \mathbb{N})^*, \mathsf{RProb}^{\leq n}_{R_0}(s,\mathcal{T}) = \mathsf{RProb}^{\leq n}_{R_1}(s \uplus \{\mathtt{label}(w)\},\mathcal{T})$$

Since $(\mathcal{T},\phi)$ is a k-simple property, we have $\mathcal{T} = \{s \in \mathcal{MP} \mid \phi_{\mathsf{nd}}(s) \wedge \exists \mathtt{sid}_1, \ldots, \mathtt{sid}_k \in \mathcal{L}$ all distinct, $\exists M_1, \ldots, M_k \in \mathbb{T}, \forall i \in \{1,\ldots,k\}, \phi(u_i) \wedge \mathtt{executed}(\mathtt{happy}(\mathtt{sid}_i, M_i)) \in s\}$. The membership of one or more labels to a state s does not change wether $s \in \mathcal{T}$ or not. In particular, we have $\forall s \in \mathcal{S}'_0, \forall w \in (\mathcal{L} \times \mathbb{N})^*, s \in \mathcal{T} \Leftrightarrow s \uplus \{\mathtt{label}(w)\} \in \mathcal{T}$, so A_0 holds.

Let $n \in \mathbb{N}$, assume A_n. Let $s \in \mathcal{S}'_0$ and $w \in (\mathcal{L} \times \mathbb{N})^*$. We pose $s' = s \uplus \mathtt{label}(w)$. If $\mathsf{trans}_0(s) = \star$, then $\mathsf{RProb}^{\leq n+1}_{R_0}(s,\mathcal{T}) = 0$ and $\mathsf{trans}_1(s') = \star$ so $\mathsf{RProb}^{\leq n+1}_{R_1}(s',\mathcal{T}) = 0$.
Otherwise, $\exists m \in \mathbb{N}^*, \exists u_1, \ldots, u_m \in \mathcal{S}'_0, \mathsf{trans}_0 = \sum_{i=1}^m 1/m \cdot \delta_{u_i}$. We have $\mathsf{RProb}^{\leq n+1}_{R_0}(s,\mathcal{T}) = \sum_{i=1}^m 1/m \cdot \mathsf{RProb}^{\leq n}_{R_0}(u_i,\mathcal{T})$.
If $\mathsf{trans}_0(s)$ executes a probabilistic choice labeled by $\mathtt{sid} \in \mathcal{L}$, then $\mathsf{trans}_1(s') = \sum_{i=1}^m 1/m \cdot \delta_{u_i \uplus \{\mathtt{label}(w::(\mathtt{sid},i))\}}$. In this case, $\mathsf{RProb}^{\leq n+1}_{R_1}(s',\mathcal{T}) = \sum_{i=1}^m 1/m \cdot \mathsf{RProb}^{\leq n}_{R_1}(u_i \uplus \{\mathtt{label}(w) :: (\mathtt{sid}, i)\},\mathcal{T})$. By the induction hypothesis, for all $i \in \{1,\ldots,m\}$, since $u_i \in \mathcal{S}'_0$, we have $\mathsf{RProb}^{\leq n}_{R_0}(u_i,\mathcal{T}) = \mathsf{RProb}^{\leq n}_{R_1}(u_i \uplus \{\mathtt{label}(w :: (\mathtt{sid}, i))\},\mathcal{T})$, so $\mathsf{RProb}^{\leq n+1}_{R_1}(s',\mathcal{T}) = \sum_{i=1}^m 1/m \cdot \mathsf{RProb}^{\leq n}_{R_0}(u_i,\mathcal{T}) = \mathsf{RProb}^{\leq n+1}_{R_0}(s,\mathcal{T})$, so A_{n+1} holds.
Otherwise, $\mathsf{trans}_1(s') = \sum_{i=1}^m 1/m \cdot \delta_{u_i \uplus \{\mathtt{label}(w)\}}$. We proceed as well. We have $\mathsf{RProb}^{\leq n+1}_{R_1}(s',\mathcal{T}) = \sum_{i=1}^m 1/m \cdot \mathsf{RProb}^{\leq n}_{R_1}(u_i \uplus \{\mathtt{label}(w)\},\mathcal{T})$. By the induction hypothesis, for all $i \in \{1,\ldots,m\}$, we have $\mathsf{RProb}^{\leq n}_{R_1}(u_i \uplus \{\mathtt{label}(w)\},\mathcal{T}) = \mathsf{RProb}^{\leq n}_{R_0}(u_i,\mathcal{T})$. Finally, $\mathsf{RProb}^{\leq n+1}_{R_0}(s,\mathcal{T}) = \sum_{i=1}^m 1/m \cdot \mathsf{RProb}^{\leq n}_{R_0}(u_i,\mathcal{T})$, so $\mathsf{RProb}^{\leq n+1}_{R_0}(s,\mathcal{T}) = \mathsf{RProb}^{\leq n+1}_{R_1}(s',\mathcal{T})$ and so A_{n+1} also holds.

We have $s_0 \in \mathcal{S}'_0$. Hence, $\forall n \in \mathbb{N}, \mathsf{RProb}^{\leq n}_{R_0}(s_0,\mathcal{T}) = \mathsf{RProb}^{\leq n}_{R_1}(s_1,\mathcal{T})$, so $\mathsf{RProb}_{R_0}(s_0,\mathcal{T}) = \mathsf{RProb}_{R_1}(s_1,\mathcal{T})$.

For all $s_0 \in \mathcal{S}^0_{\mathsf{init}}(\mathcal{P})$ and $R_0 \in \mathcal{R}(N^0)$, there exists $s_1 \in \mathcal{S}^1_{\mathsf{init}}(\mathcal{P})$ and $R_1 \in \mathcal{R}(N^1)$ such that $\mathsf{RProb}_{R_0}(s_0, \mathcal{T}) = \mathsf{RProb}_{R_1}(s_1, \mathcal{T})$. Hence, $\mathsf{RProb}_{N^0}(\mathcal{S}^0_{\mathsf{init}}, \mathcal{T}) \leqslant \mathsf{RProb}_{N^1}(\mathcal{S}^1_{\mathsf{init}}, \mathcal{T})$. □

Definition 16 (Traceable protocol)**.** *Let $\mathcal{P}$ be a multiset of processes. We say that $\mathcal{P}$ is a* traceable *protocol w.r.t. the event* happy *if $\mathcal{P}$ does not contain any* new *or* event(choice(sid))*, and if every initial state $s_0 \in \mathcal{S}^1_{\mathsf{init}}(\mathcal{P})$ satisfies the two following properties:*

1. *For all resolution $R \in \mathcal{R}(N^1)$, for all state s_1 such that $\mathsf{RProb}_R(s_0, \{s_1\}) > 0$, for all $\mathtt{sid} \in \mathcal{L}$, $\neg\{\!\{\mathtt{executed}(\mathsf{choice}(\mathtt{sid})), \mathtt{executed}(\mathsf{choice}(\mathtt{sid}))\}\!\} \subseteq s_1$.*
2. *For all resolution $R \in \mathcal{R}(N^1)$, for all label $\mathtt{sid} \in \mathcal{L}$, for all sequence $s_0 \dots, s_n$ such that $\forall i \in \{0, \dots, n-1\}, s_{i+1} \in \mathsf{supp}(\mathsf{trans}^1(s_i))$, if there exists $M \in \mathbb{T}$ and $\mathtt{sid} \in \mathcal{L}$ such that $\mathtt{executed}(\mathtt{happy}(\mathtt{sid}, M)) \in s_n$, then $\mathtt{executed}(\mathsf{choice}(\mathtt{sid})) \in s_n$.*

The first condition means that it is impossible to execute two different probabilistic choices labeled by the same name sid. The second one means that the events $\mathtt{happy}(\mathtt{sid}, M)$ cannot be executed without passing the event $\mathsf{choice}(\mathtt{sid})$.

Deterministic security

Compared to π-calculus, the main complexity of our framework is the computation of the probability of winning for the attacker. Fortunately, we can reason only about traces and not about probability by keeping track of the chosen branches in probabilistic choices. In Example 7, for each voter, either the attacker chooses to cheat or to comply with the protocol. If they cheat, they win with probability 1/2 but will also be stuck unable to win with probability 1/2. If they comply, they will not get stuck but will not get closer to victory either. We generalise this idea in the *deterministic security* property: if, for every probabilistic choice, either one branch will eventually be blocking or no branch brings the attacker closer to victory, we say that the protocol is deterministically secure and show that the chances of the attacker to win are bounded. This idea requires however to amend our semantics.

Let $\mathcal{P}$ be a traceable protocol, $s_0 \in \mathcal{S}^1_{\mathsf{init}}(\mathcal{P})$ be an initial state for $\mathcal{P}$ and $R \in \mathcal{R}(N^1)$ be a resolution. Let $s_1, \dots, s_n \in \mathcal{MP}$ such that $\forall i \in \{0, \dots, n-1\}, s_{n+1} \in \mathsf{supp}(\mathsf{trans}(s_i))$ and such that $\mathtt{executed}(\mathtt{happy}(\mathtt{sid}, M)) \in s_n$ for some $\mathtt{sid} \in \mathcal{L}$ and $M \in \mathbb{T}$. Then, by the second point of Definition 16, $\mathtt{executed}(\mathsf{choice}(\mathtt{sid})) \in s_n$ and by the first point of Definition 16, there is a unique $\mathtt{executed}(\mathsf{choice}(\mathtt{sid})) \in s_n$. Let i be the index of the trace such that s_i executes $\mathtt{executed}(\mathsf{choice}(\mathtt{sid}))$. Let $\beta \in \{1, \dots, p\}$ be the index branch that was taken when executing $s_i \to s_{i+1}$. When executing $\mathtt{event}(\mathtt{happy}(\mathtt{sid}, M))$, we label happy by β and write $\mathtt{happy}^\beta(\mathtt{sid}, M)$. This semantics amendment is only valid for traceable protocols, so we will only consider those ones in the following.

Definition 17 (Label renaming)**.** *Let $w \in (\mathcal{L} \times \mathbb{N})^*$. We define the renaming $\sigma_w : \mathcal{L} \to \mathcal{L}$ associated to this word as follow. Let $\mathtt{sid} \in \mathcal{L}$. If $\forall n \in \mathbb{N}, (\mathtt{sid}, n) \notin w$, we pose $\sigma_w(\mathtt{sid}) = \mathtt{sid}_w$. Otherwise, let w_1 be the shortest word such that $w = w_1 :: (\mathtt{sid}, n) :: w_2$. We pose $\sigma_w(\mathtt{sid}) = \mathtt{sid}_{w_1}$.*

Definition 18. *Let $\mathcal{P}$ be a traceable protocol. Let $(\mathcal{T}_f, \phi)$ be a* k-simple *property.*

We say that $\mathcal{P}$ is p-deterministically secure if for all initial state $s \in \mathcal{S}^1_{\mathsf{init}}(\mathcal{P})$, for all resolution $R \in \mathcal{R}(N^1)$, for all label $\mathtt{sid} \in \mathcal{L}$, for all states $s_1, \dots, s_p \in \mathcal{MP}$ and terms $M_1, \dots, M_p \in \mathbb{T}$, if $\sigma_{w_1}(\mathtt{sid}) = \dots = \sigma_{w_p}(\mathtt{sid})$, then we have:

$$\bigwedge_{j \in \{1,\dots,p\}} \left(\mathsf{RProb}_R(s, s_j) > 0 \wedge \mathtt{happy}^j(\mathtt{sid}, M_j) \in s_j\right) \Rightarrow \bigwedge_{j \in \{1,\dots,p\}} \neg\phi(M_j).$$

where w_j is the sequence such that $\mathtt{label}(w_j) \in s_j$.

Example 8. *To illustrate the need for label renaming, we consider the following processes and protocol:*
$P_0 = \mathtt{if}\ x = 0\ \mathtt{then}\ \mathtt{event}(\mathtt{happy}(\mathtt{sid}, \emptyset))$.
$P_1 = \mathtt{if}\ x = 1\ \mathtt{then}\ \mathtt{event}(\mathtt{happy}(\mathtt{sid}, \emptyset))$.
$Q_0 = \mathtt{out}(c, 0)$.
$Q_1 = \mathtt{out}(c, 1)$.
Then the protocol $\mathtt{in}(c, x); (P_0 +^{\mathtt{sid}}_{1/2} P_1) | (Q_0 +^{\mathtt{sid}'}_{1/2} Q_1)$ *is only deterministically secure with the renaming of the* `sid`. *Indeed, it is possible to find two different executions where* $\mathtt{happy}^0(\mathtt{sid})$ *and* $\mathtt{happy}^1(\mathtt{sid})$ *are executed, but with different labeling (*$\mathtt{sid}' \to 0$ *in the first case,* $\mathtt{sid}' \to 1$ *in the second).*

Deterministc attacker The attacker process definition is simple, but could be improved. Indeed, in [CCK23] and in our model as well, the attacker "knows" the result of the probabilistic choices, even if they are made secretly by an honest user. They can exploit this information to artificially break a security property. Thus, considering several attacker processes in parallel may lead to unrealistic attacks, as we illustrate in Example 9.

Example 9. *Let us consider the* Gamble *process again. We recall that* $\mathsf{Gamble} = P_0 +^{\mathtt{sid}}_{1/2} P_1$, *with* $P_i = \mathtt{in}(c, x).\mathtt{if}\ x = i\ \mathtt{then}\ \mathtt{happy}(\mathtt{sid})$, *for* $i = 0$ *or* 1. *We define* $\mathcal{T}_f$ *as a simple property by posing* $\phi = \top$. *Intuitively, the attacker has to guess the random bit that the process just drawn in order to win. Their probability of success should therefore be exactly* $1/2$. *However, the attacker process* $\mathcal{A} = \mathtt{out}(c, 0) | \mathtt{out}(c, 1)$ *wins with probability* 1. *Indeed, there exists a resolution* R *such that* $\mathsf{RProb}_R(\mathcal{P} \uplus \{\!\{\mathcal{A}\}\!\}, \mathcal{T}_f) = 1$, *as shown on Figure 6.3. There is only one probabilistic*

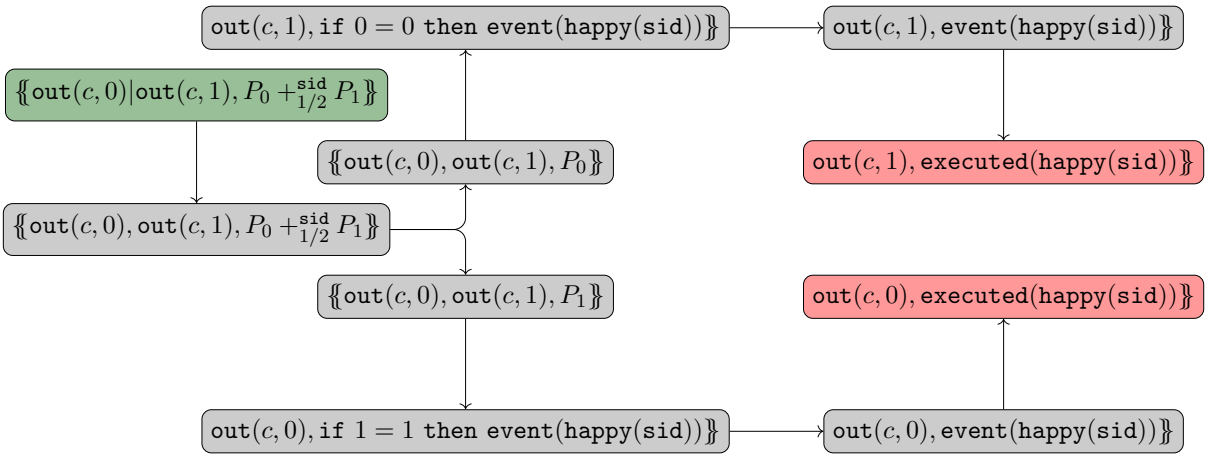


Figure 6.3: Nondeterministic attack

choice in this tree, and the two branches of this choice offers a probability of winning of 1 *to the attacker: they both lead to a winning (red) state with only non-randomised transitions. Hence, this attacker wins with probability* 1. *For this reason, we only consider a restricted class of attackers: deterministic attackers.*

We will call *deterministic attacker* the attacker processes that do not contain parallel composition, and we call $\mathcal{A}_{\mathtt{det}}$ the set of deterministic attackers. This restricted attacker class

is however still subject to false attacks. Intuitively, the parallel composition that was in the attacker process can be hidden in a honest process.

Example 10. *We use the* Gamble *process again one more time. We consider the process* $\mathtt{out}(c_{\mathsf{pub}}, 0)|\mathtt{out}(c_{\mathsf{pub}}, 1)|\mathsf{Gamble}$. *We consider the same security property as in Example 9. The probability of success of the adversary should also be* $1/2$. *However, the attacker process* $\mathcal{A} = \mathtt{in}(c_{\mathsf{pub}}, x); \mathtt{out}(c, x)$ *wins with probability* 1 *for the same reason as in the previous example.*

Chapter 7

A sufficient condition for probabilistic security

The goal of this chapter is to state and prove a theorem to reduce the study of probabilistic properties on probabilistic protocols to the study of a deterministic property on probabilistic protocols. Using the formalism introduced in the previous chapter, we use a deterministic sufficient condition called *deterministic security* for a probabilistic protocol to be secure. This result only holds for simple security properties on traceable protocols. While the simple hypothesis is purely syntactic, checking the traceable hypothesis requires to consider all possible executions of the protocol and is not as straightforward. Thus, we also present an algorithm relying on the ProVerif tool to check that a given protocol is traceable.

7.1 From probabilistic to deterministic

Theorem 1. *Let $(\mathcal{T}_f, \phi)$ be a k-*simple *property. Let $\mathcal{P}$ be a traceable protocol.*

If P is p-deterministically secure, then $\mathsf{RProb}_{N^1}\left(\mathcal{S}^1_{\mathsf{init}}(\mathcal{P}), \mathcal{T}_f\right) \leqslant \left(1 - \frac{1}{p}\right)^k$.

We introduced a new formalism to analyse probabilistic protocols. One can write a protocol in our syntax, as well as a reachability property it should satisfy. Then, Definition 12 gives the probability for an attacker to break this reachability property. However, such a probability is difficult to calculate in practice. To overcome this obstacle, Theorem 1 gives a bound on this probability for subclasses of protocols and properties.

7.2 Overview of the proof

We consider the execution tree of the protocol for each initial state and each resolution. We provide an example of such a tree on Figure 7.1, where the attacker has to trick $k = 2$ voters in order to win. Each labeled probabilistic choice has $p = 3$ branches so the attacker wins with probability at most $\frac{4}{9}$. Indeed, we can control that this result holds in our example by computing this probability by hand. After the choices sc_2, sc_3 and sc_4, two of the branches lead directly to a winning leaf while the third one is blocking. Therefore, we have $\mathsf{RProb}_R(sc_i, \mathcal{T}_f) = \frac{2}{3}$ for $i = 2, 3, 4$. The state s_1 can lead either to sc_2 or sc_3, so if also offers a probability of win of 2/3. Finally, the state sc_1 branches to s_1, sc_4 or a blocking state, so by Definition 12, $\mathsf{RProb}_R(sc_1, \mathcal{T}_f) = \frac{1}{3}\mathsf{RProb}_R(s_1, \mathcal{T}_f) + \frac{1}{3}\mathsf{RProb}_R(sc_4, \mathcal{T}_f) = \frac{4}{9}$. The probability of winning from the initial state s_0 is thus also $\frac{4}{9}$.

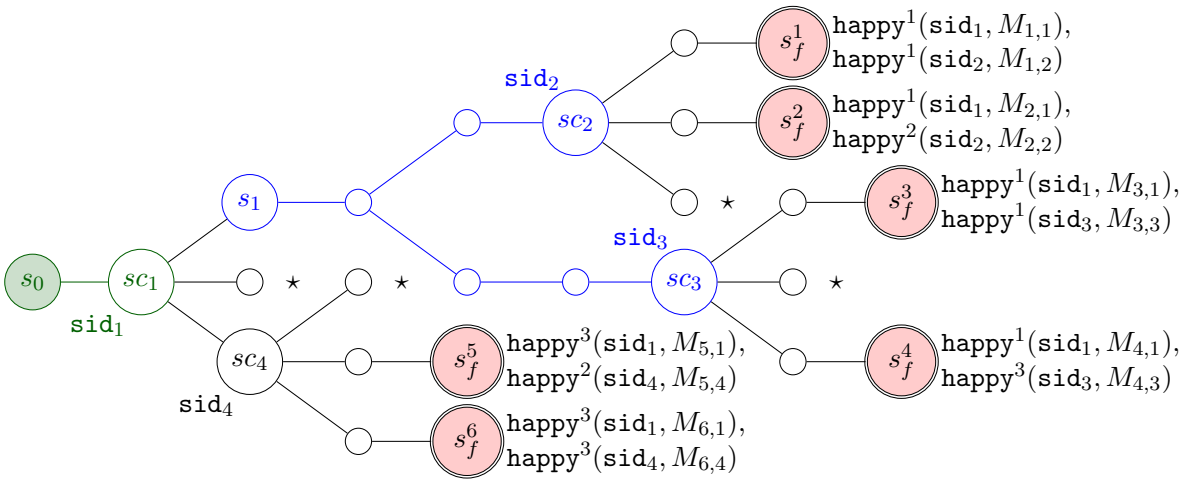


Figure 7.1: Example of an execution tree. The initial state is filled in green, the winning leaves in red.

The proof of Theorem 1 relies on two main ideas. First, the attacker has to pass at least k probabilistic choices to reach a winning leaf from an initial state. Second, each time the attacker passes a probabilistic choice, their probability of winning decreases by at least a factor $\left(1 - \frac{1}{p}\right)$. Per the simple property definition, each winning leaf s_f contains at least k elements of the form $\texttt{executed}(\texttt{happy}(\texttt{sid}_i, M_i))$, for distinct $\texttt{sid}_1, \ldots, \texttt{sid}_k$, and such that $\phi(M_i)$. We call Fooled the set of such sid. The winning leaves are encircled in the figure, and the relevant happy are specified beside. Intuitively, they are the session identifier for which the attacker will cheat in order to win. Then, per the simple property definition again, by the ϕ_{nd} property, each winning leaf also contains the corresponding $\texttt{executed}(\mathsf{choice}(\texttt{sid}_i))$. However, no initial state s_0 contains executed() elements, so the $\mathsf{choice}(\texttt{sid}_i)$ must have been executed between s_0 and s_f.

We call SChoice the states executing a probabilistic choice labeled by a $\texttt{sid} \in$ Fooled. On Figure 7.1, those states are $sc_1, \ldots, sc_4$. Using the deterministic security property, we show that at least one of the branch starting from every SChoice offers no chance of winning to the attacker. Thus, the probability of winning from an SChoice is at most $\frac{p-1}{p}$.

We call *degree* of a state the lowest number of SChoice that the attacker has to cross to reach a winning leaf from this state. The degree of an initial state is at least k, and we show that for a state of degree d, the probability of winning for the attacker is bounded by $\left(1 - \frac{1}{p}\right)^d$. For this matter, we consider the *bush* of a state, the subtree having this state for root, with children corresponding to the protocol execution, and stoping when reaching a SChoice or a deadlock. The bush of the initial state is represented in green on the above figure, and the bush of s_1 in blue. We prove a probability conservation in bushes, i.e., that the probability of winning from the root is the sum on all leaves of the probability of winning from this leaf multiplied by the probability of reaching this leaf. Then, we show that the leaves' children are at least of degree $d - 1$. We conclude using the induction hypothesis and the fact that at least one of each leaf's children is eventually blocking.

7.3 Arsenal

We introduce some formalism before proving Theorem 1.

For a state $s \in \mathcal{MP}$ and a label $\mathtt{sid} \in \mathcal{L}$, we write $\mathtt{sid} \mathrel{\mathring{\in}} s$ if there exist an index β and a term M such that $\phi(M)$ and $\mathtt{executed}(\mathtt{happy}^{\beta}(\mathtt{sid}, M)) \in s$. It means that the attacker has tricked the user labeled by $\mathtt{sid}$. For a state $s \in \mathcal{MP}$, we define $\mathsf{Fooled}(s)$, the set of the session identifiers for which the attacker will cheat or has cheated.

Definition 19 (Fooled)**.** *Let $(\mathcal{T}_f, \phi)$ be a k-*simple *property. For $s \in \mathcal{MP}$, we define $\mathsf{Fooled}(s)$ recursively:*

- *if $s \in \mathcal{T}_f$, $\mathsf{Fooled}(s) = \{\mathtt{sid} \in \mathcal{L}, \mathtt{sid} \mathrel{\mathring{\in}} s\}$.*
- *otherwise, if $\mathsf{trans}(s) = \star$, $\mathsf{Fooled}(s) = \emptyset$.*
- *otherwise, $\mathsf{trans}(s) = D \in \mathcal{D}(\mathcal{S})$. Then, $\mathsf{Fooled}(s) = \bigcup_{s' \in \mathsf{supp}(D)} \mathsf{Fooled}(s')$.*

Then, we define the *degree* of a state s, that represents the minimum number of times the attacker has to cheat to reach a target from s.

Definition 20 (Degree)**.** *Let $(\mathcal{T}_f, \phi)$ be a k-*simple *property. Let $(\mathcal{S}, \mathsf{trans}) \in \mathcal{R}(N^1)$. For $s \in \mathcal{MP}$, we define $\deg_{R,\mathcal{T}_f}(s)$ recursively:*

- *if $s \in \mathcal{T}_f$, then $\deg_{R,\mathcal{T}_f}(s) = 0$.*
- *otherwise, if $\mathsf{trans}(s) = \star$, then $\deg_{R,\mathcal{T}_f}(s) = \infty$.*
- *otherwise, $\mathsf{trans}(s) = D \in \mathcal{D}(\mathcal{S})$. If D executes a probabilistic choice labeled by $\mathtt{sid} \in \mathsf{Fooled}(s)$, $\deg_{R,\mathcal{T}_f}(s) = 1 + \min\{\deg_{R,\mathcal{T}_f}(s'), s' \in \mathsf{supp}(D)\}$. Otherwise, $\deg_{R,\mathcal{T}_f}(s) = \min\{\deg_{R,\mathcal{T}_f}(s'), s' \in \mathsf{supp}(D)\}$.*

Definition 21 (Proof tree)**.** *Let $(\mathcal{T}_f, \phi)$ be a k-*simple *property. Let $R \in \mathcal{R}(N^1)$. Let $s \in \mathcal{MP}$. We call proof tree and note $\pi_{R,s,\mathcal{T}_f}$ the tree defined inductively as following:*

- *if $\mathsf{trans}(s) = \star$ or if $s \in \mathcal{T}_f$, $\pi_{R,s,\mathcal{T}_f}$ is the tree with s as only node.*
- *otherwise, $\mathsf{trans}(s) = D \in \mathcal{D}(\mathcal{S})$ and $\pi_{R,s,\mathcal{T}_f}$ is the tree where s is the root and for every $s' \in \mathsf{supp}(D)$, $\pi_{R,s',\mathcal{T}_f}$ is attached by its root to s.*

If π is a proof tree, we note $\mathtt{W-leaf}(\pi) \subseteq \mathcal{T}_f$ (for winning leaves) the set of accepting states reached in π: $\mathtt{W-leaf}(\pi_{R,s,\mathcal{T}_f}) = \{s_f \in \mathcal{T}_f \mid \mathsf{RProb}_R(s, \{s_f\}) > 0\}$.

Definition 22 (SChoice)**.** *Let $R = (\mathcal{S}, \mathsf{trans}) \in \mathcal{R}(N^1)$. Let $(\mathcal{T}_f, \phi)$ be a k-*simple *property. Let $s \in \mathcal{S}$ be a state such that $\deg_{R,\mathcal{T}_f}(s) \geqslant 1$. Let $s_f \in \mathtt{W-leaf}(\pi_{R,s,\mathcal{T}_f})$. Let $s_1, \ldots, s_n$ be the unique path from s to s_f.*

Since $\deg_{R,\mathcal{T}_f}(s) \geqslant 1$, there exist $i \in \{1, \ldots, n\}$ and $\mathtt{sid} \in \mathcal{L}$ such that $\mathsf{trans}(s_i)$ executes the probabilistic choice corresponding to $\mathtt{sid} \mathrel{\mathring{\in}} s_f$.

We note $\mathsf{SChoice}(s_f, \pi_{R,s,\mathcal{T}_f})$ the state s_i closest to the root such that $\mathsf{trans}(s_i)$ executes a probabilistic choice corresponding to a $\mathtt{sid} \mathrel{\mathring{\in}} s_f$.

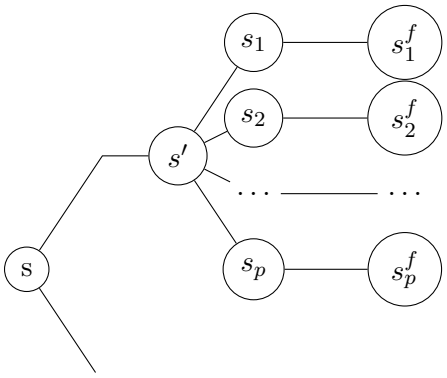
Definition 23 (Bush)**.** *Let $R = (\mathcal{S}, \mathsf{trans}) \in \mathcal{R}(N^1)$. Let $(\mathcal{T}_f, \phi)$ be a k-*simple *property. Let $s \in \mathcal{S}$ be a state such that $\deg_{R,\mathcal{T}_f}(s) \geqslant 1$.*

We define the bush $B_{R,s,\mathcal{T}_f}$, a tree defined inductively from $\pi_{R,s,\mathcal{T}_f}$ as follows:

- *s is the root of $B_{R,s,\mathcal{T}_f}$.*
- *if $\mathsf{trans}(s) = \star$ or if there exists $s_f \in \mathtt{W} - \mathtt{leaf}(\pi_{R,s,\mathcal{T}_f})$ such that $s = \mathsf{SChoice}(s_f, \pi_{R,s,\mathcal{T}_f})$, then s does not have any children.*
- *Otherwise, $\mathsf{trans}(s) = D \in \mathcal{D}(\mathcal{S})$ and s children are the bushes $B_{R,s',\mathcal{T}_f}$ for $s' \in \mathsf{supp}(D)$.*

Proposition 3. *Let $R = (\mathcal{S}, \mathsf{trans}) \in \mathcal{R}(N^1)$. Let $(\mathcal{T}_f, \phi)$ be a k-*simple *security property. Let $\mathcal{P}$ be a traceable p-deterministically secure protocol. Let $s \in \mathcal{S}$ such that $\deg_{R,\mathcal{T}_f}(s) \geqslant 1$ and $\exists s_0 \in \S^1_{\mathsf{init}}(\mathcal{P})$ such that $\mathsf{RProb}_R(s_0, s) > 0$. Let $s_f \in \mathtt{W} - \mathtt{leaf}(\pi_{R,s,\mathcal{T}_f})$. Let $D = \mathsf{trans}(\mathsf{SChoice}(s_f, \pi_{R,s,\mathcal{T}_f}))$. Then, $D \in \mathcal{D}(\mathcal{S})$ and $\exists s_1, \ldots, s_p, \mathsf{supp}(D) = \{s_1, \ldots, s_p\}$. Also, there exists $j \in \{1, \ldots, p\}$ such that $\mathsf{RProb}_R(s_j, \mathcal{T}_f) = 0$.*

Proof. Let $s' = \mathsf{SChoice}(s_f, \pi_{R,s,\mathcal{T}_f})$. Let $D = \mathsf{trans}(s')$ such that D executes a probabilistic choice corresponding to $\mathtt{sid} \mathring{\in} s_f$. Then, by Definition 13, there exist $M \in \mathbb{T}$ and $\beta \in \{1, \ldots, p\}$ such that $\phi(M)$ and $\mathtt{executed}(\mathtt{happy}^{\beta}(\mathtt{sid}, M)) \in s_f$. Also, we have $\mathsf{supp}(D) = \{s_1, \ldots, s_p\}$. By contradiction, assume that for all $j \in \{1, \ldots, p\}, \mathsf{RProb}_R(s_j, \mathcal{T}_f) > 0$. If it is the case, for all $j \in \{1, \ldots, p\}$, there exists $s_j^f \in \mathcal{T}_f$ such that $\mathsf{RProb}_R(s_j, s_j^f) > 0$.



For syntactic reason, for all j, we have $\mathtt{executed}(\mathsf{choice}(\mathtt{sid})) \in s_j^f$: events are persistent in our model, and $\mathtt{executed}(\mathsf{choice}(\mathtt{sid})) \in s_j$. Since $s_j^f \in \mathcal{T}_f$, we have $\phi_{\mathsf{nd}}(s_j^f)$ so $\exists \beta_j, \exists M_j$ such that $\mathtt{executed}(\mathtt{happy}^{\beta_j}(\mathtt{sid}, M_j)) \in s_j^f$. Then, per our event labeling, we have $\beta_j = j$.

For all $j \in \{1, \ldots, p\}$, we have $\mathsf{RProb}_R(s, s_j^f) > 0$ and $\mathtt{executed}(\mathtt{happy}^{j}(\mathtt{sid}, M_j)) \in s_j^f$. We also have $\mathsf{RProb}_R(s, s^f) > 0$ and $\mathtt{executed}(\mathtt{happy}^{\beta}(\mathtt{sid}, M)) \in s^f$ such that $\phi(M)$ holds.

Remember that for each reachable s there is exactly one extended process $\mathtt{label}(w)$ such that $\mathtt{label}(w) \in s$. Let w_j^f be the labeling sequence such that $\mathtt{label}(w_j^f) \in s_j^f$ and w_f such that $\mathtt{label}(w_f) \in s_f$. Let us show that $\sigma_{w_1^f}(\mathtt{sid}) = \ldots = \sigma_{w_p^f}(\mathtt{sid}) = \sigma_{w_f}(\mathtt{sid})$. For this, we consider w' and $\mathtt{sid}$. For semantic reason, for all j, there exists t_j such that $w_j^f = w' :: (\mathtt{sid}, j) :: t_j$. Also, $\exists t, w_f = w' :: (\mathtt{sid}, \beta) :: t$. Since $\mathcal{P}$ is traceable, $\mathsf{choice}(\mathtt{sid})$ is executed at most once in every trace, hence $\mathtt{sid}$ does not appear in w', t_j, and t'. Thus $\sigma_{w_f}(\mathtt{sid}) = \sigma_{w_1^f}(\mathtt{sid}) = \ldots = \sigma_{w_p^f}(\mathtt{sid}) = \mathtt{sid}_{w'}$.

We have $\forall j \in \{1, \ldots, p\}, \mathsf{RProb}_R(s, s_j^f) > 0$ and $\mathtt{executed}(\mathtt{happy}^{j}(\mathtt{sid}, M_j)) \in s_j^f$. We also have $\mathsf{RProb}_R(s, s^f) > 0$ and $\mathtt{executed}(\mathtt{happy}^{\beta}(\mathtt{sid}, M)) \in s^f \wedge \phi(M)$. Finally, $\sigma_{w_f}(\mathtt{sid}) = \sigma_{w_1^f}(\mathtt{sid}) = \ldots = \sigma_{w_p^f}(\mathtt{sid})$. Since $\mathsf{RProb}_R(s_0, s) > 0$, we have $\mathsf{RProb}_R(s_0, s^f) > 0$ and $\forall j \in \{1, \ldots, p\}, \mathsf{RProb}_R(s_0, s_j^f) > 0$, with s_0 an initial state. This contradicts the p-deterministic security property. Thus, $\exists j \in \{1, \ldots, p\}, \mathsf{RProb}_R(s_j, \mathcal{T}_f) = 0$. □

Proposition 4. *Let $R = (\mathcal{S}, \mathsf{trans}) \in \mathcal{R}(N^1)$. Let $(\mathcal{T}_f, \phi)$ be a k-*simple *security property. For all $s \in \mathcal{S}$ such that $\deg_{R,\mathcal{T}_f}(s) \geqslant 1$, the leaves of the bush $B_{R,s,\mathcal{T}_f}$ are states $l \in \mathcal{S}$ such that $\mathsf{trans}(l) = \star$ or $l = \mathsf{SChoice}(s_f, \pi_{R,s,\mathcal{T}_f})$ for $s_f \in \mathtt{W} - \mathtt{leaf}(\pi_{R,s,\mathcal{T}_f})$. Moreover, those leaves are not in $\mathcal{T}_f$.*

Proof. We prove this property by induction on the height of $B_{R,s,\mathcal{T}_f}$. For $n \in \mathbb{N}$, we note $P(n)$: for all $s \in \mathcal{S}$ such that $\deg_{R,\mathcal{T}_f}(s) \geqslant 1$, if the height of the tree $B_{R,s,\mathcal{T}_f}$ is at most n, then the leaves of $B_{R,s,\mathcal{T}_f}$ are not in $\mathcal{T}_f$ and are states $l \in \mathcal{S}$ such that $\mathsf{trans}(l) = \star$ or $\exists s_f \in \mathcal{T}_f$, such that $l = \mathsf{SChoice}(s_f, \pi_{R,s,\mathcal{T}_f})$.

Let $s \in \mathcal{S}$ of degree at least 1. If $B_{R,s,\mathcal{T}_f}$ is reduced to its root s, then s is its only leaf and either $\mathsf{trans}(s) = \star$ or $\exists s_f \in \mathcal{T}_f, s = \mathsf{SChoice}(s_f, \pi_{R,s,\mathcal{T}_f})$. Also, if $s \in \mathcal{T}_f$, then $\deg_{R,\mathcal{T}_f}(s) = 0$ so $s \notin \mathcal{T}_f$. Hence $\mathcal{P}(1)$ holds.

Let $n \in \mathbb{N}^*$. Assume $P(i)$ holds for all $i \in \{1, \dots, n\}$. Let $s \in \mathcal{S}$ of degree at least 1 such that the height of $B_{R,s,\mathcal{T}_f}$ is $n+1$. Let $B = B_{R,s,\mathcal{T}_f}$. Since B is not of height 1, it is not reduced to its root s. Hence, $\exists D \in \mathcal{D}(\mathcal{S}), \mathsf{trans}(s) = D$ and children of s are copies of $B_{R,l,\mathcal{T}_f}$ for $l \in \mathsf{supp}(D)$. Also, D does not execute a probabilistic choice corresponding to a $\mathtt{sid} \in \mathsf{Fooled}(s)$, otherwise B would be reduced to its root.
By the definition of the degree, $\deg_{R,\mathcal{T}_f}(s) = \min\{\deg_{R,\mathcal{T}_f}(s'), s' \in \mathsf{supp}(D)\}$. Hence, we have $\forall s' \in \mathsf{supp}(D), \deg_{R,\mathcal{T}_f}(s') \geqslant \deg_{R,\mathcal{T}_f}(s) \geqslant 1$. Moveover, for all $s' \in \mathsf{supp}(D)$, since the height of B is at most $n+1$, the height of $B_{R,s',\mathcal{T}_f}$ is at most n. By $P(n)$, for all $s' \in \mathsf{supp}(D)$, the leaves of $B_{R,s',\mathcal{T}_f}$ are states $l \in \mathcal{S} \setminus \mathcal{T}_f$ such that either $\mathsf{trans}(l) = \star$ or $\exists s_f \in \mathcal{T}_f$, such that $l = \mathsf{SChoice}(s_f, \pi_{R,s',\mathcal{T}_f})$.
Let l be a leaf of B. There exists $s' \in \mathsf{supp}(D)$ such that l is a leaf of $B_{R,s',\mathcal{T}_f}$. Hence, $l \notin \mathcal{T}_f$ and either $\mathsf{trans}(l) = \star$ or $\exists s_f \in \mathcal{T}_f$ such that $l = \mathsf{SChoice}(s_f, \pi_{R,s,\mathcal{T}_f})$. Assume $\mathsf{trans}(l) \neq \star$. Let $s_f \in \mathcal{T}_f$ such that $l = \mathsf{SChoice}(s_f, \pi_{R,s',\mathcal{T}_f})$. Since B is not reduced to its root, D does not execute a probabilistic choice corresponding to a $\mathtt{sid} \in \mathsf{Fooled}(s)$ so $l = \mathsf{SChoice}(s_f, \pi_{R,s,\mathcal{T}_f})$. The leaves of B are states $l \in \mathcal{S} \setminus \mathcal{T}_f$ such that $\mathsf{trans}(l) = \star$ or $\exists s_f \in \mathcal{T}_f$ such that $l = \mathsf{SChoice}(s_f, \pi_{R,s,\mathcal{T}_f})$. Thus, $P(n+1)$ holds. □

Proposition 5. *Let $R = (\mathcal{S}, \mathsf{trans}) \in \mathcal{R}(N^1)$. Let $(\mathcal{T}_f, \phi)$ be a k-*simple *security property. Let $s \in \mathcal{S}$ such that $\deg_{R,\mathcal{T}_f}(s) \geqslant 1$. Let L be the set of leaves of $B_{R,s,\mathcal{T}_f}$. Then, $\mathsf{RProb}_R(s, \mathcal{T}_f) = \sum_{l \in L} \mathsf{RProb}_R(s, \{l\}) \mathsf{RProb}_R(l, \mathcal{T}_f)$.*

Proof. We prove this property by induction of the height of $B_{R,s,\mathcal{T}_f}$. For $n \in \mathbb{N}$, we note $P(n)$: for all $s \in \mathcal{S}$ such that $\deg_{R,\mathcal{T}_f}(s) \geqslant 1$, if the height of the tree $B_{R,s,\mathcal{T}_f}$ is at most n, then $\mathsf{RProb}_R(s, \mathcal{T}_f) = \sum_{l \text{ leaf of } B_{R,s,\mathcal{T}_f}} \mathsf{RProb}_R(s, \{l\}) \mathsf{RProb}_R(l, \mathcal{T}_f)$.

Base case: let $s \in \mathcal{S}$ of degree at least 1 such that the height of $B_{R,s,\mathcal{T}_f}$ is at most 1. Then, $B_{R,s,\mathcal{T}_f}$ is reduced to its root s and s is its only leaf. Also, $s \in \{s\}$ so $\mathsf{RProb}_R(s, \{s\}) = 1$. Thus, $\mathsf{RProb}_R(s, \mathcal{T}_f) = \sum_{l \text{ leaf of } B_{R,s,\mathcal{T}_f}} \mathsf{RProb}_R(s, \{l\}) \mathsf{RProb}_R(l, \mathcal{T}_f)$ and $P(1)$ holds.

Induction step: let $n > 1$, assume $P(i)$ holds for $i \in \{1, \dots, n\}$. Let $s \in \mathcal{S}$ of degree at least 1 such that the height of $B_{R,s,\mathcal{T}_f}$ is at most $n+1$. Let $B = B_{R,s,\mathcal{T}_f}$. If the height of B is at most n, we conclude using $P(n)$. Otherwise, B is not reduced to its root. Then, $s \notin \mathcal{T}_f$ and $\mathsf{trans}(s) \neq \star$. Let $D = \mathsf{trans}(s)$. We have $\mathsf{RProb}_R(s, \mathcal{T}_f) = \sum_{s' \in \mathsf{supp}(D)} \mathsf{RProb}_R(s', \mathcal{T}_f) D(s')$.
The bush B has for children the bushes $B_{R,s',\mathcal{T}_f}$ for $s' \in \mathsf{supp}(D)$. Also, since the height of B

is $n+1$, for $s' \in \mathsf{supp}(D)$, the height of $B_{R,s',\mathcal{T}_f}$ is at most n so by $P(n)$, $\mathsf{RProb}_R(s',\mathcal{T}_f) = \sum_{l \text{ leaf of } B_{R,s',\mathcal{T}_f}} \mathsf{RProb}_R(s',\{l\})\mathsf{RProb}_R(l,\mathcal{T}_f)$. Because of the tree structure of B, the sets of leaves of $B_{R,s',\mathcal{T}_f}$ for $s' \in \mathsf{supp}(D)$ form a partition of the set of leaves of B.

$$\begin{aligned}
\mathsf{RProb}_R(s,\mathcal{T}_f) &= \sum_{s' \in \mathsf{supp}(D)} \mathsf{RProb}_R(s',\mathcal{T}_f)D(s') \\
&= \sum_{s' \in \mathsf{supp}(D)} D(s') \sum_{l \text{ leaf of } B_{R,s',\mathcal{T}_f}} \mathsf{RProb}_R(s',\{l\})\mathsf{RProb}_R(l,\mathcal{T}_f) \\
&= \sum_{s' \in \mathsf{supp}(D)} \left(\sum_{l \text{ leaf of } B_{R,s',\mathcal{T}_f}} \mathsf{RProb}_R(s,\{l\})\mathsf{RProb}_R(l,\mathcal{T}_f) \right) \\
&= \sum_{l \text{ leaf of } B} \mathsf{RProb}_R(s,\{l\})\mathsf{RProb}_R(l,\mathcal{T}_f)
\end{aligned}$$

□

Proposition 6. *Let $R = (\mathcal{S}, \mathsf{trans}) \in \mathcal{R}(N^1)$. Let $(\mathcal{T}_f, \phi)$ be a k-*simple *security property. Let $\mathcal{P}$ be a traceable protocol. Let $s_0 \in \mathcal{S}^1_{\mathsf{init}}(\mathcal{P})$ be an initial state. Then, $\deg_{R,\mathcal{T}_f}(s_0) \geqslant k$.*

Proof. We define a sequence of states recursively. If s_n is defined, then:

- if $s_n \in \mathcal{T}_f$ of if $\mathsf{trans}(s_n) = \star$, we do not define s_{n+1}.
- otherwise, let s_{n+1} be the element of $\mathsf{supp}(\mathsf{trans}(s_n))$ of minimal degree.

The sequence (s_n) is a trace for $\to_1$ so it is finite: there exists $N \in \mathbb{N}$ such that s_N is defined but s_{N+1} is not.

For all $n \in \mathbb{N}$ such that s_n and s_{n+1} are defined, two cases are possible. We have either $\deg_{R,\mathcal{T}_f}(s_n) = 1 + \deg_{R,\mathcal{T}_f}(s_{n+1})$ if $\mathsf{trans}(s_i)$ executes a probabilistic choice labeled by $\mathtt{sid} \in \mathsf{Fooled}(s_i)$, or $\deg_{R,\mathcal{T}_f}(s_n) = \deg_{R,\mathcal{T}_f}(s_{n+1})$ otherwise. Thus, we have $\deg_{R,\mathcal{T}_f}(s_0) \geqslant \deg_{R,\mathcal{T}_f}(s_N)$. Also, since s_{N+1} is not defined, either $\mathsf{trans}(s_N) = \star$ or $s_N \in \mathcal{T}_f$. In the first case, $\deg_{R,\mathcal{T}_f}(n_N) = \infty$ so $\deg_{R,\mathcal{T}_f}(s_0) = \infty$. In the second case, since $(\mathcal{T}_f, \phi)$ is a *k-simple* security property, $\exists \mathtt{sid}_1, \ldots, \mathtt{sid}_k \in \mathcal{L}$ all distinct, $\exists u_1, \ldots, u_k$ such that $\forall i \in \{1,\ldots,k\}, \phi(u_i)$ and $\mathtt{executed}(\mathtt{happy}(\mathtt{sid}_i, u_i)) \in \mathcal{P}_N$. For all $i \in \{1,\ldots,k\}$, since $\mathtt{sid}_i \in \mathsf{Fooled}(s_N)$, we have for all $n \in \{0,\ldots,N\}, \mathtt{sid}_i \in \mathsf{Fooled}(s_n)$. Moreover, since $\mathcal{P}$ is a traceable protocol, there exists $n_1 < \ldots < n_k \in \{0,\ldots,N\}$ such that for all $i \in \{1,\ldots,k\}, \mathsf{trans}(s_{n_i})$ executes a probabilistic choice labelled by $\mathtt{sid}_k$. Thus, we have $\deg_{R,\mathcal{T}_f}(s_{n_k}) = \deg_{R,\mathcal{T}_f}(s_{n_k+1})$. By summing inequalities, $\deg_{R,\mathcal{T}_f}(s_0) \geqslant k$. □

7.4 Proof

Proof. We prove here the Theorem 1 by induction. Let $R \in \mathcal{R}(N^1)$. Let $\mathcal{H}(d) = \forall s \in \mathcal{MP}$, if $\deg_{R,\mathcal{T}_f}(s) \geqslant d, \mathsf{RProb}_R(s,\mathcal{T}_f) \leqslant \left(1 - \frac{1}{p}\right)^d$.
Base case ($d = 0$): RProb_R is bounded by 1.
Induction step: let $d \geqslant 1$. Assume the result holds for $d-1$. Let s be a state such that

$\deg_{R,\mathcal{T}_f}(s) \geqslant d$. Let us show that $\mathsf{RProb}_R(s,\mathcal{T}_f) \leq \left(1-\frac{1}{p}\right)^d$.
Let $B = B_{R,s,\mathcal{T}_f}$. First, by proposition 5, we have an expression of $\mathsf{RProb}_R(s,\mathcal{T}_f)$:

$$\mathsf{RProb}_R(s,\mathcal{T}_f) = \sum_{l \texttt{ leaf of } B} \mathsf{RProb}_R(s,\{l\})\mathsf{RProb}_R(l,\mathcal{T}_f).$$

Second, by proposition 4, for all leaves l, either $\mathsf{trans}(l) = \star$ and $l \notin \mathcal{T}_f$ or $\exists s_f \in \mathcal{T}_f$ such that $\mathsf{trans}(l)$ executes the probabilistic choice corresponding to $\texttt{sid} \mathrel{\mathring{\in}} s_f$. In the latter case, l has p children $l_1,\ldots,l_p$. By proposition 3, $\exists i_0 \in \{1,\ldots,p\}$ such that $\mathsf{RProb}_R(l_{i_0},\mathcal{T}_f) = 0$. Also, for $i \in \{1,\ldots,p\}, i \neq i_0$, $\deg_{R,\mathcal{T}_f}(l_i) \geqslant d-1$ (otherwise, $\deg_{R,\mathcal{T}_f}(s) < d$). By the induction hypothesis, for all $i \in \{1,\ldots,p\}, \mathsf{RProb}_R(l_i,\mathcal{T}_f) \leqslant \left(1-\frac{1}{p}\right)^{d-1}$. We can now calculate $\mathsf{RProb}_R(l,\mathcal{T}_f)$:

$$\begin{aligned}
\mathsf{RProb}_R(l,\mathcal{T}_f) &= \sum_{i=1}^{p} \frac{1}{p}\mathsf{RProb}_R(l_i,\mathcal{T}_f) && (7.1)\\
&= \frac{1}{p}\mathsf{RProb}_R(l_{i_0},\mathcal{T}_f) + \sum_{i\neq i_0} \frac{1}{p}\mathsf{RProb}_R(l_i,\mathcal{T}_f) && (7.2)\\
&\leqslant \frac{1}{p}\sum_{i\neq i_0}\left(1-\frac{1}{p}\right)^{d-1} && (7.3)\\
&\leqslant \left(1-\frac{1}{p}\right)^{d} && (7.4)
\end{aligned}$$

Finally, we can calculate $\mathsf{RProb}_R(s,\mathcal{T}_f)$:

$$\begin{aligned}
\mathsf{RProb}_R(s,\mathcal{T}_f) &= \sum_{l \text{ leaf of } B} \mathsf{RProb}_R(s,\{l\})\mathsf{RProb}_R(l,\mathcal{T}_f) && (7.5)\\
&\leqslant \sum_{l \texttt{ leaf of } B} \left(1-\frac{1}{p}\right)^{d} \mathsf{RProb}_R(l,\mathcal{T}_f) && (7.6)\\
&\leqslant \left(1-\frac{1}{p}\right)^{d} \sum_{l \texttt{ leaf of } B} \mathsf{RProb}_R(l,\mathcal{T}_f) && (7.7)\\
&\leqslant \left(1-\frac{1}{p}\right)^{d} && (7.8)
\end{aligned}$$

This conclude the induction step.

By Proposition 6, for all $s \in \mathcal{S}^1_{\mathsf{init}}(\mathcal{P}), \deg_{R,\mathcal{T}_f}(s) \geqslant k$ so by $\mathcal{H}(k), \mathsf{RProb}_R(s,\mathcal{T}_f) \leqslant \left(1-\frac{1}{p}\right)^k$. Thus, $\mathsf{RProb}_R(\mathcal{S}^1_{\mathsf{init}}(\mathcal{P})t,\mathcal{T}_f) \leqslant \left(1-\frac{1}{p}\right)^k$. Since this result holds for all resolution $R \in \mathcal{R}(N^1)$, $\mathsf{RProb}_{N^1}(\mathcal{S}^1_{\mathsf{init}}(\mathcal{P}),\mathcal{T}_f) \leqslant \left(1-\frac{1}{p}\right)^k$ □

7.5 Checking the traceable hypothesis

In this section, we present how to automatically verify the traceable hypothesis using the tool ProVerif. In order to ease the translation between probabilistic processes and ProVerif processes, we restrict ourselves to probabilistic processes without `new`. This does not limit the expressivity of probabilistic processes: since we do not have replications, fresh names can be replaced by private names in $\mathcal{N}_{\mathsf{priv}}$.

7.5.1 Mini-ProVerif semantics

In order to automatically prove the deterministic security property, we translate our protocols into a subset of the ProVerif language. We call π-process a process generated by the grammar from the figure 7.2. It differs from the syntax presented in 2 to ease the correspondence with the probabilistic protocols syntax.

P, Q	$:=$		π-Processes
		0	null process
		$\texttt{let}\ x = D\ \texttt{in}\ P\ \texttt{else}\ Q$	conditional declaration
		$\texttt{out}(N, M); P$	output
		$\texttt{in}(N, x); P$	input
		$P \mid Q$	parallel composition
		$P + Q$	non deterministic choice
		$\texttt{event}(E(M_1, \ldots, M_k)); P$	event
		$\texttt{insert}\ t(M); P$	table insertion
		$\texttt{get}\ t(x)\ \texttt{suchthat}\ D\ \texttt{in}\ P\ \texttt{else}\ Q$	table access
V	$:=$		Extended π-processes
		P	π-process
		$\texttt{executed}(E(M_1, \ldots, M_k))$	event execution
		$\texttt{label}(w)$	labeling

Figure 7.2: Mini-ProVerif syntax

A π-process is closed if it contains no free variable. Let $\mathcal{MP}^{\pi}$ (resp $\mathcal{MP}^{\pi}_{\mathsf{ext}}$) be the set of all multisets of closed π-processes (resp. closed extended π-processes).

Definition 24 (Configuration)**.** *We call* configuration *a multiset of extended π-processes.*

The semantics of π-processes is defined by a reduction relation $\rightarrow_{\pi}$ between configurations, presented in figure 7.3. We may write $\rightarrow$ instead of $\rightarrow_{\pi}$ when the transition system is clear from the context. This grammar is similar to the ProVerif one exposed in Chapter 2, but has a few differences. First, the syntax is more expressive since it offers the non-deterministic choice $P + Q$. This is not an issue, since it can be reproduced in the ProVerif grammar using a parallel composition, and a restriction forbidding to take the two branches in the same trace. We chose to include the non-deterministic choice in our syntax to be as close as possible to the probabilistic process one from Chapter 6. It also keeps the labeling $\texttt{label}(w)$ from our probabilistic protocol language. To obtain a ProVerif process from an extended π-process, one can simply remove those labels. Second, the event facts $\texttt{executed}(ev)$ and the table facts $\texttt{executed}(\mathsf{inserted}_t(M))$ are stored directly in the multiset of extended processes instead of having processes, executed events and table facts separated in different sets of the state. Again, the reason of this choice is to mimic the probabilistic process grammar, and the correspondence with the real ProVerif grammar is straightforward. Finally, the π-processes syntax does not contain replication nor name restriction. Indeed, we will want to lift Mini-ProVerif attackers to probabilistic attackers. Considering only attackers without replication is sound, since all ProVerif traces are finite: if there exists an attack trace caused by an infinite attacker, the same trace can be executed by a finite attacker. Then, removing the `new` is also sound: without replication, we can use free name of $\mathcal{N}_{\mathsf{priv}}$ instead of drawing a fresh name. We may also want to consider an unbounded number

of honest process. In this case, instead of writing $!P$, we will write $P|\ldots|P$, with n copies of P, with n a free parameter. If we can prove our property for all values of n, then this property also holds for an unbounded number of copies of P.

$$\mathcal{P} \uplus \{\!\{0\}\!\} \to_\pi \mathcal{P} \qquad \text{(Null process)}$$
$$\mathcal{P} \uplus \{\!\{\texttt{let } x = D \texttt{ in } P \texttt{ else } Q\}\!\} \to_\pi \mathcal{P} \uplus \{\!\{P\{x \mapsto M\}\}\!\} \quad \text{if } D \Downarrow M \qquad \text{(Conditional)}$$
$$\mathcal{P} \uplus \{\!\{\texttt{let } x = D \texttt{ in } P \texttt{ else } Q\}\!\} \to_\pi \mathcal{P} \uplus \{\!\{Q\}\!\} \quad \text{otherwise}$$
$$\mathcal{P} \uplus \{\!\{\texttt{out}(N, M); P, \texttt{in}(N, x); Q\}\!\} \to_\pi \mathcal{P} \uplus \{\!\{P, Q\{x \mapsto M\}\}\!\} \qquad \text{(Communication)}$$
$$\mathcal{P} \uplus \{\!\{P \mid Q\}\!\} \to_\pi \mathcal{P} \uplus \{\!\{P, Q\}\!\} \qquad \text{(Parallel)}$$
$$\mathcal{P} \uplus \{\!\{P + Q\}\!\} \to_\pi \mathcal{P} \uplus \{\!\{P\}\!\} \qquad \text{(Choice-left)}$$
$$\mathcal{P} \uplus \{\!\{P + Q\}\!\} \to_\pi \mathcal{P} \uplus \{\!\{Q\}\!\} \qquad \text{(Choice-right)}$$
$$\mathcal{P} \uplus \{\!\{\texttt{event}(ev(M)); P\}\!\} \to_\pi \mathcal{P} \uplus \{\!\{P, \texttt{executed}(ev(M))\}\!\} \qquad \text{(Event)}$$
$$\mathcal{P} \uplus \{\!\{\texttt{insert } t(M); P\}\!\} \to_0 \mathcal{P} \uplus \{\!\{\texttt{executed}(\mathsf{inserted}_t(M)), P\}\!\} \qquad \text{(Insertion)}$$
$$\mathcal{P} \uplus \{\!\{\texttt{get } t(x) \texttt{ suchthat } D \texttt{ in } P \texttt{ else } Q, \texttt{executed}(\mathsf{inserted}_t(M))\}\!\}$$
$$\to_\pi \mathcal{P} \uplus \{\!\{\texttt{executed}(\mathsf{inserted}_t(M)), P\{x \mapsto M\ \}\!\} \quad \text{if } D\sigma \Downarrow \mathsf{true} \text{ and } x\sigma = M \qquad \text{(Reading-1)}$$
$$\mathcal{P} \uplus \{\!\{\texttt{get } t(x) \texttt{ suchthat } D \texttt{ in } P \texttt{ else } Q\}\!\}$$
$$\to_\pi \mathcal{P} \uplus \{\!\{Q\}\!\} \quad \text{if } \forall \sigma, \texttt{executed}(\mathsf{inserted}_t(x\sigma)) \in \mathcal{P} \Rightarrow D\sigma \not\Downarrow \mathsf{true} \qquad \text{(Reading-2)}$$

Figure 7.3: Mini-ProVerif semantics

Let C_0 be a configuration. We call *trace of* C_0 any finite sequence $C_0 \to_\pi C_1 \to_\pi \ldots \to_\pi C_n$. We write $C_0 \to_\pi^* C_n$ if there exist configurations $C_1, \ldots, C_{n-1}$ such that $C_0 \to_\pi C_1 \to_\pi \ldots \to_\pi C_{n-1} \to_\pi C_n$.

Definition 25 ($\mathcal{N}_{\mathsf{pub}}$-adversary)**.** *Let $\mathcal{A}$ be a multiset of processes. If $\mathcal{A}$ does not contain event actions nor table actions and if $fn(\mathcal{A}) \subseteq \mathcal{N}_{\mathsf{pub}}$, then $\mathcal{A}$ is a $\mathcal{N}_{\mathsf{pub}}$-adversary.*

Given a multiset of π-processes $\mathcal{P}$, we denote by $\mathsf{Tr}(\mathcal{P})$ the set of traces of $\mathcal{P} \uplus \mathcal{A}$ for $\mathcal{A}$ a $\mathcal{N}_{\mathsf{pub}}$-adversary. The set of initial states of a multiset of π-processes $\mathcal{P}$ is $\mathcal{S}^\pi_{\mathsf{init}}(\mathcal{P})$, the set of $\mathcal{P} \uplus \mathcal{A}$ for $\mathcal{A}$ a $\mathcal{N}_{\mathsf{pub}}$-adversary.

In what follows, we may write ev instead of $\texttt{executed}(\mathsf{ev})$ to ease readability.

We also have a grammar for queries :

ϕ	::=	Formula	
	$M = N$		Equality
	$M \neq N$		Disequality
ψ	::=	Conclusion	
	$\top$		true
	$\bot$		false
	$\psi \wedge \psi'$		Conjunction
	$\psi \vee \psi'$		Disjunction
	ϕ		Formula
ev	::=	Event	
	$E(M_1, \ldots, M_k)$		$E \in Ev$
q	::=	Correspondence query	
	$\mathsf{ev}_1@i_1 \wedge \ldots \wedge \mathsf{ev}_n@i_n \wedge \phi \Rightarrow \psi$		$i_1, \ldots, i_n \in \mathbb{N}$
	$\mathsf{ev}_1@i_1 \wedge \ldots \wedge \mathsf{ev}_n@i_n \wedge \phi \Rightarrow \mathsf{ev}$		$i_1, \ldots, i_n \in \mathbb{N}$

Let $T = \mathcal{P}_0 \to_\pi \mathcal{P}_1 \to_\pi \ldots \to_\pi \mathcal{P}_n$ be a trace. We say that T satisfies the event $\mathsf{ev}@t$ and write $T \vdash \mathsf{ev}@t$ if there exists a multiset of processes $\mathcal{P}$ and a process P such that $\mathcal{P}_t = \mathcal{P} \uplus \{\!\{\texttt{event}(ev(M)); P\}\!\}$ and $\mathcal{P}_{t+1} = \mathcal{P} \uplus \{\!\{P, \texttt{executed}(ev(M))\}\!\}$.

Definition 26 (Satisfiability)**.** *Let $q = (\mathsf{ev}_1@i_1 \wedge \ldots \wedge \mathsf{ev}_n@i_n \wedge \phi \Rightarrow a)$ be a correspondence query, where a is a formula ψ or an event ev. Let $T = \mathcal{P}_0 \rightarrow_\pi \mathcal{P}_1 \rightarrow_\pi \ldots \rightarrow_\pi \mathcal{P}_n$ be a trace. We say that T satisfies q and write $T \models q$ if whenever $T \vdash \mathsf{ev}_1\sigma@i_1, \ldots, \mathsf{ev}_n\sigma@i_n$ and $\phi\sigma$ holds for some substitution σ, then there exists σ' extending σ, that is, $\phi\sigma'$ holds and $\mathsf{ev}_j\sigma' = \mathsf{ev}_j\sigma$ for any $j \in \{1, \ldots, n\}$, and such that $\psi\sigma'$ holds if $a = \psi$ and such that $T \vdash \mathsf{ev}\sigma'$ if $a = \mathsf{ev}$.*

Let $\mathcal{P}$ be a multiset of processes. We say that $\mathcal{P}$ satisfies q and write $\mathcal{P} \models q$ if all traces $T \in \mathsf{Tr}(\mathcal{P})$, we have $T \models q$. By extension to sets of queries, we write $T \models S$ (resp $\mathcal{P} \models S$) if $T \models q$ (resp $\mathcal{P} \models q$) for any $q \in S$.

Definition 27 (Restriction)**.** *Let $\mathcal{P}$ be a multiset of processes. Let $\mathcal{Q}$ and $\mathcal{R}$ be two sets of correspondence queries. We say that $\mathcal{P}$ satisfies $\mathcal{Q}$ with respect to the restrictions $\mathcal{R}$ and we write $\mathcal{P} \models_{\mathcal{R}} \mathcal{Q}$ if for all traces $T \in \mathsf{Tr}(\mathcal{P})$, if $T \models \mathcal{R}$, then $T \models \mathcal{Q}$.*

7.5.2 Checking traceability

We introduce a transformation τ from probabilistic processes to π-processes that replaces each probabilistic choice by a non deterministic choice and leaves the rest unchanged.

- $\tau(P_1 +_{1/p} \ldots +_{1/p} P_p) = \tau(P_1) + \cdots + \tau(P_p)$
- $\tau(P_1 +^{\mathtt{sid}}_{1/p} \ldots +^{\mathtt{sid}}_{1/p} P_p) = \mathtt{event}(\mathsf{choice}(\mathtt{sid})); (\tau(P_1) + \cdots + \tau(P_p))$
- $\tau(0) = 0$
- $\tau(P \mid Q) = \tau(P) \mid \tau(Q)$
- $\tau(\mathtt{let}\ x = D\ \mathtt{in}\ P\ \mathtt{else}\ Q) = \mathtt{let}\ x = D\ \mathtt{in}\ \tau(P)\ \mathtt{else}\ \tau(Q)$
- $\tau(\mathtt{event}(E(M_1, \ldots, M_k)).P) = \mathtt{event}(E(M_1, \ldots, M_k)); \tau(P)$
- etc.

We extend τ on extended processes as follow. We define $\tau(V)$ as $\emptyset$ if $V = \mathtt{label}(w)$ or if $V = \mathtt{executed}(\mathsf{ev})$, and as $\tau(P)$ if $V = P$ is a process without `new`. Finally, we extend τ to multiset of extended processes: $\tau(\mathcal{V}) = \biguplus_{V \in \mathcal{V}} \{\!\{\tau(V)\}\!\}$.

Any move of P can be mimicked by a move of $\tau(P)$.

Proposition 7. *Let $s, s' \in \mathcal{MP}$ such that s does not contain any* `new` *and such that $s' \in \mathsf{supp}(\mathsf{trans}^1(s))$ then $\tau(s) \rightarrow^*_\pi \tau(s')$.*

Proof. The semantics $\rightarrow_\pi$ exactly mimics $\rightarrow_0$ except for the two rules PChoice and LPChoice.

- PChoice rule: Assume $s' \in \mathsf{supp}(\mathsf{trans}^1(s))$ and the PChoice has been executed, that is $s = \mathcal{P} \uplus \{\!\{P_1 +_{1/p} \ldots +_{1/p} P_p\}\!\}$ and $s' = \mathcal{P} \uplus \{\!\{P_i\}\!\}$ for some $1 \leq i \leq p$. Then $\tau(s) = \tau(\mathcal{P}) \uplus \{\!\{\tau(P_1) + \cdots + \tau(P_p)\}\!\}$ and $\tau(s') = \tau(\mathcal{P}) \uplus \{\!\{\tau(P_i)\}\!\}$. Then $\tau(s) \rightarrow^*_\pi \tau(s')$ by applying several (Choice-left) and (Choice-right) rules of $\rightarrow_\pi$.
- LPChoice rule: Assume $s' \in \mathsf{supp}(\mathsf{trans}^1(s))$ and the LPChoice has been executed, that is $s = \mathcal{P} \uplus \{\!\{P_1 +^{\mathtt{sid}}_{1/p} \ldots +^{\mathtt{sid}}_{1/p} P_p\}\!\}$ and $s' = \mathcal{P} \uplus \{\!\{P_i, \mathtt{executed}(\mathsf{choice}(\mathtt{sid}))\}\!\}$ for

some $1 \leq i \leq p$. Then $\tau(s) = \tau(\mathcal{P}) \uplus \{\!\{\mathtt{event}(\mathsf{choice}(\mathtt{sid})); (\tau(P_1) + \cdots + \tau(P_p))\}\!\}$ and $\tau(s') = \tau(\mathcal{P}) \uplus \{\!\{\tau(P_i)\mathtt{executed}(\mathsf{choice}(\mathtt{sid}))\}\!\}$. We have

$$\tau(s) \rightarrow_\pi \tau(\mathcal{P}) \uplus \{\!\{\tau(P_1) + \cdots + \tau(P_p), \mathtt{executed}(\mathsf{choice}(\mathtt{sid}))\}\!\} \rightarrow_\pi^* P_i$$

by applying the Event rule and then several (Choice-left) and (Choice-right) rules of $\rightarrow_\pi$.

□

We consider two queries that correspond to the two conditions of traceability for a process

- $q_{\mathsf{one-choice}} = \mathsf{choice}(\mathtt{sid})@t_1 \& \& \mathsf{choice}(\mathtt{sid})@t_2 \Rightarrow t_1 = t_2$
- $q_{\mathsf{Happy} \Rightarrow \mathsf{choice}} = \mathtt{happy}(\mathtt{sid}) \Rightarrow \mathsf{choice}(\mathtt{sid})$

Proposition 8. *Let $\mathcal{P}$ be a multiset of processes that does not contain any* `new` *and such that for all* $\mathtt{sid} \in \mathcal{L}, \mathsf{choice}(\mathtt{sid}) \notin \mathcal{P}$. *If $\tau(\mathcal{P}) \models q_{\mathsf{one-choice}}, q_{\mathsf{Happy} \Rightarrow \mathsf{choice}}$, then $\mathcal{P}$ is a traceable protocol.*

Proof. Assume $\mathcal{P}$ is not a traceable protocol. Then, $\mathcal{P}$ does not satisfy at least one of the two conditions of Definition 16. Let us show that $\tau(\mathcal{P}) \not\models q_{\mathsf{one-choice}}$ or $\tau(\mathcal{P}) \not\models q_{\mathsf{Happy} \Rightarrow \mathsf{choice}}$.

If the first condition is false, there exists a resolution $R \in \mathcal{R}(N^1)$, an initial state $s \in \mathcal{S}^1_{\mathsf{init}}(\mathcal{P})$, a state $s' = (w', \mathcal{P}') \in \mathcal{MP}$, and a label $\mathtt{sid} \in \mathcal{L}$ such that $\mathsf{RProb}_R(s, \{s'\}) > 0$ and $\{\!\{\mathsf{choice}(\mathtt{sid}), \mathsf{choice}(\mathtt{sid})\}\!\} \subseteq \mathcal{P}'$. Then, there exists a sequence of states $s_0, \ldots, s_n$ such that $s_0 = s, s_n = s'$, and for all $i \in \{0, \ldots, n-1\}, s_{i+1} \in \mathsf{supp}(\mathsf{trans}^1(s_i))$. By Proposition 7, for all $i \in \{0, \ldots, n-1\}, \tau(s_i) \rightarrow_\pi^* \tau(s_{i+1})$ is a valid trace. Since s is an initial state, s does not contain $\mathsf{choice}(\mathtt{sid})$. Also, we have $\{\!\{\mathsf{choice}(\mathtt{sid}), \mathsf{choice}(\mathtt{sid})\}\!\} \subseteq s_n$ so there exists $i_1 < i_2$ such that $\mathsf{choice}(\mathtt{sid}) \notin s_{i_1}$ and $\mathsf{choice}(\mathtt{sid}) \in s_{i_1+1}$, and $\{\!\{\mathsf{choice}(\mathtt{sid}), \mathsf{choice}(\mathtt{sid})\}\!\} \not\subseteq s_{i_2}$ and $\{\!\{\mathsf{choice}(\mathtt{sid}), \mathsf{choice}(\mathtt{sid})\}\!\} \subseteq s_{i_2+1}$. By Proposition 7, the traces $\tau(s_{i_1}) \rightarrow_\pi^* \tau(s_{i_1+1})$ and $\tau(s_{i_2}) \rightarrow_\pi^* \tau(s_{i_2+1})$ both execute the event $\mathsf{choice}(\mathtt{sid})$. Hence, the trace $T = \tau(s_0) \rightarrow_\pi^* \tau(s_n)$ verifies $T \vdash \mathsf{choice}(\mathtt{sid})@i_1, \mathsf{choice}(\mathtt{sid})@i_2$, with $i_1 \neq i_2$. Since $s \in \mathcal{S}^1_{\mathsf{init}}(\mathcal{P})$ is an initial state, there exists an attacker $\mathcal{P}_{\mathsf{att}}$ such that $s = (\epsilon, (\mathcal{P} | \mathcal{P}_{\mathsf{att}}))$. $\mathcal{P}_{\mathsf{att}}$ is an attacker so it does not contain any event nor `new` nor labelled probabilistic choice, and $fn(\mathcal{P}_{\mathsf{att}}) \subseteq \mathcal{N}_{\mathsf{pub}}$. Thus, $\tau(\mathcal{P}_{\mathsf{att}})$ is a $\mathcal{N}_{\mathsf{pub}}$-adversary, hence $T \in \mathsf{Tr}(\mathcal{P})$ and $\mathcal{P} \not\models q_{\mathsf{one-choice}}$.

If the second condition is false, there exists an initial state $s \in \mathcal{S}^1_{\mathsf{init}}(\mathcal{P})$, a resolution $R \in \mathcal{R}(N^1)$, a label $\mathtt{sid} \in \mathcal{L}$, a term M and a sequence of states $s_0, \ldots, s_n$ such that $s_0 = s, s' = s_n, \mathtt{happy}(\mathtt{sid}, M) \in s_n$ and $\mathsf{choice}(\mathtt{sid}) \notin s_n$. Hence $\mathtt{happy}(\mathtt{sid}, M) \in \tau(s_n)$ and $\mathsf{choice}(\mathtt{sid}) \notin \tau(s_n)$. Therefore $T = \tau(s_0) \rightarrow_\pi \ldots \rightarrow_\pi \tau(s_n) \vdash \mathtt{happy}(\mathtt{sid}, M)$ and $T \not\vdash \mathsf{choice}(\mathtt{sid})$. Moreover, similarly to the first case, we have $T \in \mathsf{Tr}(\mathcal{P})$. Thus, $\mathcal{P} \not\models q_{\mathsf{Happy} \Rightarrow \mathsf{choice}}$. □

7.6 Conclusion

This new grammar allows to express probabilistic protocols in a very similar way to π-calculus. We introduced the deterministic security property, to ease the analyse of probabilistic protocols by implying simple probabilistic properties by a deterministic property. However, this reduction is only valid for a reduced class of protocols, the traceable protocols. We provide a translation of probabilistic processes to a grammar useable by the ProVerif tool, and we use it to automatically check the belonging of a given protocol to this class. We strongly believe that this idea and a similar transformation can be used for the deterministic security property. In the next chapter, we give some case studies of probabilistic protocols of the literature illustrating the range but also the limits of the traceable protocols class.

Chapter 8

Case study analysis

In this chapter, we will show how our approach applies to different probabilistic protocols from the literature. We consider two different voting protocols, for which there are probabilistic choices in the control flow: the STROBE protocol [Ben21], as well as the BeleniosCaI protocol [CDGG23]. After introducing and modeling those protocols, we will present a proof that these models satisfy the traceable hypothesis. Then, we will present ongoing work that uses the same methodology in order to prove that the models also satisfy the deterministic security property.

8.1 Presentation of the protocols

8.1.1 Modeling probabilistic voting protocols

For both STROBE and BeleniosCaI, we focus on the cast-as-intended property. Indeed, it is the property that is guaranteed by a probabilistic mechanism. Our goal is to model this situation in the language described in Chapter 6. We want to bound the property that the attacker flips at least k votes without being detected. In short, each voter will be associated to a session identifier `sid`. The event $\texttt{happy}(\texttt{sid}, M)$ will be triggered when the voter labeled by `sid` successfully finishes their verification procedure. Hence, the security of the protocol will consist of a k-simple property, in the sense of Definition 13. This property holds in all corruption scenarios: it is true not only against a dishonest ballot provider (for STROBE, the printer; for Belenios CaI, the voting device) but also against a compromised voting server. Hence, we do not model those processes: the attacker is free to simulate them using their own process. However, the bulletin board is audited. Thus, we write a `Board` process to ensure that the ballot box contents respect the specification. We assume a private channel `public_board`, that represents the ballots included in the ballot box. The attacker can read the content of this channel, but not write anything.

8.1.2 STROBE

We already presented the STROBE protocol in Chapter 3. We briefly recall the main points of this voting protocol. The goal of STROBE is to provide verifiability against a dishonest printer. To do so, it uses a singular mechanism: the printer provides each voter with two ballots. The voter will use one of them to vote and will audit the other. If this choice is made uniformly at random, then, intuitively, the voter will detect any cheating attempt with probability at least 1/2. The rest of the protocol is standard: the election key is generated by a set of trustees,

there is a public board to publish the recorded ballots and some verification data, and the tally is either performed after a verifiable shuffle or via homomorphic encryption.

Ballots. In STROBE, each voter gets two ballots from the printer, similar to the one presented in Figure 8.1. Each ballot contains a long code (eight bytes). Additionally, the ballot contains a shortcode (one byte) for every voting option. To cast their vote, the voter simply picks one of their ballots, crosses the box for their choice, and posts it to the polling station.

Alice	☐	Q4
Bob	☐	D6
Charlie	☐	L7
None	☐	E1
1960-857c-c5db-3939		

Figure 8.1: Ballot in STROBE

Setup. To generate a ballot, the printer first encrypts the identity matrix using the election's public key pk_E. Each line of the matrix corresponds to a voting option. Then, they compute the shortcode of each line by truncating its hash. The structure of the encryption is displayed on Figure 8.2.

Candidate	Encryption	Code
Alice	$(\mathsf{aenc}(1), \mathsf{aenc}(0), \mathsf{aenc}(0), \mathsf{aenc}(0))$	Q4
Bob	$(\mathsf{aenc}(0), \mathsf{aenc}(1), \mathsf{aenc}(0), \mathsf{aenc}(0))$	D6
Charlie	$(\mathsf{aenc}(0), \mathsf{aenc}(0), \mathsf{aenc}(1), \mathsf{aenc}(0))$	L7
None	$(\mathsf{aenc}(0), \mathsf{aenc}(0), \mathsf{aenc}(0), \mathsf{aenc}(1))$	E1

Figure 8.2: Structure of the encryption

Part of this data is made public to allow tallying the ballots and performing the verifications. For each ballot, the lines are reordered such that the short codes increase. Then, the ballot's long code is computed by hashing the permuted encrypted matrix. The ballots are paired such that there is no short code collision between the ballots of a pair. The printer publishes on the bulletin board the information shown in Figure 8.3: for each ballot, the permuted encrypted matrix, the sorted short codes, the longcode and the longcode of the ballot twin. Finally, the printer sends a pair of ballots to each voter by post.

Tallying and verifying. The polling station collects all cast ballots. To record one, they simply publish its longcode and the shortcode of the chosen option. At the end of the election, the trustees compute the result using the ciphertexts corresponding to the chosen shortcodes. All ballots that are not cast are spoilt: the printer must reveal the randomness used to encrypt them, as well as a copy of each ballot. The auditors ensure that the information published on the board is consistent. Each voter verifies that their chosen shortcode has been recorded properly and that their audit ballot is displayed on the board.

Encryption	Code
$(\mathsf{aenc}(0), \mathsf{aenc}(1), \mathsf{aenc}(0), \mathsf{aenc}(0))$	D6
$(\mathsf{aenc}(0), \mathsf{aenc}(0), \mathsf{aenc}(0), \mathsf{aenc}(1))$	E1
$(\mathsf{aenc}(0), \mathsf{aenc}(0), \mathsf{aenc}(1), \mathsf{aenc}(0))$	L7
$(\mathsf{aenc}(1), \mathsf{aenc}(0), \mathsf{aenc}(0), \mathsf{aenc}(0))$	Q4

Ballot code: 1960-857c-c5db-3939
Code of the twin: 1768-0ad1-5757-96fe

Figure 8.3: Information published on the board

Modeling STROBE. We give here a modeling of STROBE in the $\mathtt{syn}_0$ syntax from Chapter 6. We consider a reduced version of the protocol, with only two voting options: v_0 and v_1. We consider in our equational theory:

- a `short_hash` function, that represents the truncation of a hash.
- a `make_ballot` function, that constructs a ballot out of its components: $\mathtt{make_ballot}(vote1, short_code1, vote2, short_code2, long_code)$.
- a `make_data` function that constructs the public data associated to a ballot: $\mathtt{make_data}(encryption1, short_code1, encryption2, short_code2, long_code)$

For the last two functions, there exists a destructor to retrieve any of their subterms.

We also consider several constants, used to tidy the bulletin board: `credentials`, for the list of the valid longcodes, `verification_data`, for the data published by the printer during the setup, `recorded`, for the list of recorded ballots, `longcode_to_spoil` for the unused longcodes, and `spoilt`, for the list of spoilt ballots.

```
1  Voter(vote, sid) =
2    in(c_att, (ballotL, ballotR)).                (* get the ballots from the dishonest printer *)
3    Voting(vote, ballotL, ballotR, sid) +1/2^sid Voting(vote, ballotR, ballotL, sid).
4
5  Voting(vote, voting_ballot, audit_ballot, sid) =
6    let make_ballot(=v0, short_code0, =v1, short_code1, longcode) = voting_ballot in
7    let verification_short_code = [if vote = v0 then short_code0 else short_code1] in
8    out(c_att, (voting_ballot, vote)).                                    (* cast the vote *)
9    in(board, (=recorded, =verification_short_code, =longcode)).    (* check the vote is recorded
         properly *)
10   in(board, (=spoilt, =audit_ballot)).
11   event(happy(sid, (voting_ballot, vote))).
```

The voter first receives their ballots, cast their vote, and then performs the verification: they check that their vote has been recorded properly (line 9) and that their other ballot is correctly spoilt (line 10).

```
1  Board =
2    (
3      in(c_att, (data_L, data_R)).                (* get the verification data from the printer *)
4      let make_data(x_a_L, short_code_a_L, x_b_L, short_code_b_L, longcode_L) = data_L in
5      let make_data(x_a_R, short_code_a_R, x_b_R, short_code_b_R, longcode_R) = data_R in
6
7      (* Consistency checks *)
8      if short_code_a_L = short_hash(x_a_L) then
9      if short_code_b_L = short_hash(x_b_L) then
```

```
10    if short_code_a_R = short_hash(x_a_R) then
11    if short_code_b_R = short_hash(x_b_R) then
12    if longcode_L = hash(x_a_L, x_b_L) then
13    if longcode_R = hash(x_a_R, x_b_R) then
14
15    out(board, (credential, longcode_L, longcode_R)).
16    out(board, (verification_data, data_L, data_R))
17  )
18  |
19  (
20    in(c_att, (shortcode, longcode)).            (* receive a ballot from the polling station *)
21
22    (* check if the longcode is a valid credential *)
23    in(board, (=credential, longcode_L, longcode_R)).
24    if longcode = longcode_L ∨ longcode = longcode_R then
25
26    (* publish the ballot *)
27    out(board, (recorded, shortcode, longcode)).
28
29    let audit_longcode = [if longcode = longcode_L then longcode_R else longcode_R] in
30    out(board, (longcode_to_spoil, audit_longcode))
31  )
32  |
33  (
34    (* get the randomness from the attacker *)
35    in(c_att, (r0, r1)).
36
37    (* recompute the encryptions *)
38    let x0 = aenc(v0, r0, pkE) in
39    let x1 = aenc(v1, r1, pkE) in
40
41    (* the attacker sorts the shortcodes *)
42    in(c_att, b: bool).
43    let x_a = if b then x0 else x1 in
44    let x_b = if b then x1 else x0 in
45    let short_code_a = short_hash(x_a) in
46    let short_code_b = short_hash(x_b) in
47
48    let longcode = hash(x_a, x_b) in
49    in(board, (=longcode_to_spoil, =longcode)).
50
51    (* control the consistency with the verification data *)
52    in(board, =verification_data, data_L, data_R).
53    in(c_att, b': bool).
54    let audit_data = if b' then data_L else data_R in
55    if audit_data = make_data(x_a, short_code_a, x_b, short_code_b, longcode) then
56
57    out(board, (spoilt, make_ballot(v0, short_code0, v1, short_code1, longcode))).
58  )
```

The board, scrutinised by the auditors, performs many actions. They receive the setup data from the printer, audit it and publish it on lines 2 to 17. From line 19 to line 31, they process the data obtained from the ballot box. Finally, the board publishes and controls the audit data published by the printer between the lines 33 and 58.

The victory condition of the attacker is defined by ϕ:

$$\begin{aligned}\phi(\mathsf{bal}, v_i) = \quad & \text{“}\mathsf{bal} = \mathsf{make_ballot}(v_0, sc_0, v_1, sc_1, \mathtt{longcode}) \\ & \wedge \mathtt{longcode} = \mathsf{hash}(x_0, x_1) \\ & \wedge x_i \neq \mathsf{aenc}(v_i, r, \mathsf{pk}_E)\text{”}\end{aligned}$$

It means that the voter tried to vote for the option v_i using the ballot `bal`, but this ballot encodes another option instead.

8.1.3 BeleniosCaI

BeleniosCaI [CDGG23] is another protocol that aims to provide cast-as-intended verifiability against a compromised voting device without losing receipt freeness. It relies on a so-called cast-and-audit mechanism: each voter will receive only one ballot, send it to the server, and then perform some verification. Like for STROBE, this verification grants verifiability only up to a certain property. Indeed, only half of the ballot, chosen by the voter, will be audited. It is the other half that ensures vote privacy by remaining secret.

BeleniosCaI involves different parties: the *voting server* that collects the ballot, the *decryption authorities* that collectively generate the election's public key pk_E, each *voter* and their *voting device*. As usual, there is also a public board where the recorded ballots are published, and *auditors* verifying that its content respects the specification. The ballots are authenticated by each voter's public key `pk`.

Ballots. In this simplified version of a protocol, we consider that each ballot contains only one vote, rather that a vector of votes. The vote v is encrypted by the voting device. The device also draws a random nonce a and compute $b = v + a$. Those nonces will serve later for verification. A ballot is formed by a triplet (M, π, σ), where $M = (V, A, B)$, V, A and B are the encryptions of v, a and b, π is a zero-knowledge proof of well-formedness and σ is the signature of (M, π). More precisely, π implies that V, A and B encrypt v', a' and v' such that v' is a valid vote and that $v' + a' = b'$.

However, the ProVerif language does not allow for summing integers. Since this zero-knowledge proof is not mandatory to show that the BeleniosCaI is traceable, we chose not to model it. It would however be necessary to prove other properties like deterministic security. In [CDGG23], modular arithmetic is modelled via restrictions, not supported in $\mathtt{syn}_0$.

Voting and auditing. The voter starts the voting phase by entering their signing key and their vote in their voting device. The device sends the ballot to the voting server, which does not publish it for now. Instead, it sends a nonce `chal` and the ballot hash via a channel independent of the voting device. The voter transmits the nonce to their device, hence starting the audit phase. The device reveals a and b to the voter, who checks that $v + a = b$. Then, the voter chooses at random a or b, and requests the corresponding side to the device by sending $\delta = L$ for a or R for b. The device sends δ and the corresponding randomness r to the server. The server recomputes a or b using r, and publishes on the board the ballot, the result of the computation, δ and r. The voter finished the audit phase by checking that the hash of their ballot appears on the board and that a or b is correct and corresponds to their request.

```
1  Voter(vote, sk, sid) =
2
3    (* enter the signing key and the vote in the device *)
4    out(c_att, (sk, vote)).
5
6    (* get the ballot hash from the voting device *)
7    in(c_att, h_1).
8
9    (* get the ballot hash and the challenge from the server *)
10   in(c_att, (h_2, chal)).
```

```
11
12    if h_1 = h_2 then
13
14    (* submit the challenge to the voting device *)
15    out(c_att, chal).
16
17    (* the voting device displays the nonces *)
18    in(c_att, (a, b)).
19
20    (* probabilistic part *)
21    Verification(L, a, vote, h_1, sid) +^{sid}_{1/2} Verification(R, b, vote, h_1, sid)
```

```
1  Verification(side, audit_plain, vote, h_1, sid) =
2
3    (* tell the device the side to spoil *)
4    out(c_att, side).
5
6    (* get the result of the challenge on the board *)
7    in(board, (h_3, _, audit_plain', side', _)).
8
9    (* control that the ballot is recorded properly *)
10   if h_1 = h_3 then
11
12   (* control that the challenge succeed *)
13   if audit_plain = audit_plain' then
14   if side = side' then
15
16   event(happy(sid, (vote, h_1))).
```

BeleniosCaI requires an encryption function that allows to decrypt using the randomness, like the ElGamal scheme. We model it by the constructor $\mathsf{aenc}(m, r, \mathsf{pk})$, for the encryption of the message m with the randomness r and the public key pk. The first associated destructor is $\mathsf{adec}(x, \mathsf{sk})$, with the rewriting rule $\mathsf{adec}(\mathsf{aenc}(m, r, \mathsf{pk}(\mathsf{sk})), \mathsf{sk}) \to m$. The decryption with the randomness is given by the other destructor $\mathtt{adec_rand}(x, r, \mathsf{pk})$, with the rewriting rule $\mathtt{adec_rand}(\mathsf{aenc}(m, r, \mathsf{pk}), r, \mathsf{pk}) \to m$.

```
1  Board(pk_voter) =
2    in(c_att, (ballot, side, delta, rand))
3    let (M, pi, sigma) = ballot in
4
5    (* verify that the signature is correct *)
6    if check_sig(sigma, (M, pi), pk_voter) then
7
8    let (V, A, B) = M in
9    let audit_cipher = [if delta = L then A else B] in
10
11   (* decrypt A or B using the randomness *)
12   let audit_plain = adec_rand(audit_cipher, rand, pkE) in
13
14   out(board, (hash(ballot), ballot, audit_plain, delta, rand)).
```

The victory condition of the attacker is defined by ϕ:

$$\phi(v_i, h) = \text{“}h = \mathsf{hash}(ballot) \wedge ballot = (M, \pi, \sigma) \wedge M = (X, A, B) \wedge X \neq \mathsf{aenc}(v_i, r, \mathsf{pk}_E)\text{”}$$

Again, the attacker wins if the voter tried to vote v_i with a ballot that contains another vote.

8.2 Checking traceability

We apply the method of Section 7.5.2 to the protocols of Section 8.1. Translating the protocols from the probabilistic protocols syntax to the ProVerif syntax is direct and only requires one minor change. Since the + operator does not exist in ProVerif, we used parallel composition instead, and encode the impossibility of executing both sides with a restriction.

```
1 restriction st: stamp, t1, t2: time; event(OnlyOneSide(st))@t1 && event(OnlyOneSide(st))@t2
2   ==> t1 = t2.
3
4 P + Q =
5   new st: timestamp;
6   (event OnlyOneSide(st); P)
7   |
8   (event OnlyOneSide(st); Q)
```

This approach allowed us to prove that our case studies are traceable. In both cases, the computation lasts for less than one second.

STROBE	✔
BeleniosCaI	✔

The ProVerif models will be available with the final version of the thesis.

8.3 Towards deterministic security

Starting from this section, the work we present is still ongoing. In particular, the statements we make are more sketches than formally proven theorems. Our goal is to provide an automated method to check if a protocol verifies the p-deterministic security property using the ProVerif tool. For the sake of simplicity, we only consider the $p = 2$ case. Thus, we consider the subset of the probabilistic processes language that does not involve probabilistic choices with more than two branches. Formally, we restrict the rules $P_1 +_p \ldots +_p P_n$ and $P_1 +_p^{\mathtt{sid}} \ldots +_p^{\mathtt{sid}} P_n$ to $P +_{1/2} Q$ and to $P +_{1/2}^{\mathtt{sid}} Q$. Moreover, from now on, we assume each labeled probabilistic action $P +_{1/2} Q$ to be also labeled by some constant α. Input actions $\mathtt{in}(c, x)$ may also be labeled. Given a probabilistic process P, we will assume that its labeled actions have unique labels (no label is used more than once). Intuitively, honest processes will be labeled while attacker processes will not.

We recall the deterministic security property, restricted to the case where $p = 2$.

Definition 28. *Let $\mathcal{P}$ be a traceable protocol. Let $(\mathcal{T}_f, \phi)$ be a k-*simple *property.*

We say that $\mathcal{P}$ is 2-deterministically secure if for all initial state $s \in \mathcal{S}^1_{\mathsf{init}}(\mathcal{P})$, for all resolution $R \in \mathcal{R}(N^1)$, for all label $\mathtt{sid} \in \mathcal{L}$, for all states $s_0, s_1 \in \mathcal{MP}$ and terms $M_0, M_1 \in \mathbb{T}$, if $\sigma_{w_0}(\mathtt{sid}) = \sigma_{w_1}(\mathtt{sid})$, if $\mathsf{RProb}_R(s, s_0) > 0$ and $\mathsf{RProb}_R(s, s_1) > 0$, then we have:

$$\mathtt{happy}^0(\mathtt{sid}, M_0) \in s_0 \wedge \mathtt{happy}^1(\mathtt{sid}, M_j) \in s_1 \Rightarrow \neg\phi(M_0) \wedge \neg\phi(M_1).$$

where w_0, w_1 are the sequences such that $\mathtt{label}(w_j) \in s_j$.

To check for deterministic security, we need intuitively to explore two branches of the execution of the process P. So intuitively, we want to run $\tau(P) \mid \tau(P)$. However, we also need to encode the labeling $\sigma_w(\mathtt{sid})$. Recording the list of probabilistic choices that have been passed would be difficult in the ProVerif language, so we chose to overapproximate instead. We let

the adversary freely chose any renaming of $\mathtt{sid}$, and, then, we constrain its behavior through restrictions. Moreover, we particularise a special session label $\mathtt{sid}^*$ that the attacker must use when breaking deterministic security. In this way, the query for checking deterministic-security will be defined as follows:

$$q_{\mathsf{det-security}} = \mathtt{happy}^0(\mathtt{sid}^*, u_0) \;\&\&\; \mathtt{happy}^1(\mathtt{sid}^*, u_1) \Rightarrow \neg\phi(u_0) \;\&\&\; \neg\phi(u_1)$$

To express those two branches, we define two transformations τ_1 and τ_2, one for each branch to be explored, as follows. Let $\delta \in \{1, 2\}$. We consider a table Tlabel_δ that will store the associations between the $\mathtt{sid}$ and the values chosen by the adversary for the explored trace δ. We also consider a table $\mathsf{Tglobal}$ that keeps track on whether the attacker has already used $\mathtt{sid}^*$ or not. $\mathsf{Tglobal}$ will initially contain the value 0. The value 1 will be added to the table as soon as the attacker uses $\mathtt{sid}^*$. Moreover, for each table $t \in \mathsf{Tab}$, we now consider two tables t_1 and t_2, one for each explored trace.

- $\tau_\delta(\mathtt{in}^\alpha(c, x).P) = \mathtt{get}\ \mathsf{Tglobal}(b)\ \mathtt{in}\ \mathtt{in}(c, x); \mathtt{event}(\mathtt{received}(\delta, b, \alpha, x)); \tau_\delta(P)\ \mathtt{else}\ 0$
  We record the message that has been received in trace δ for action α.
- $\tau_\delta(\mathtt{in}(c, x).P) = \mathtt{in}(c, x); \tau_\delta(P)$
  When the input action is not labeled, we simply propagate the transformation.
- $\tau_\delta(P +^{\mathtt{sid}}_{\alpha,1/2} Q) = \mathtt{in}(c_{\mathsf{att}}, \mathtt{sid}');\ \mathtt{if}\ \mathtt{sid}' = \mathtt{sid}^*\ \mathtt{then}\ \mathtt{insert}\ \mathsf{Tglobal}(1).R\ \mathtt{else}\ R$
  where $R = (\mathtt{event}(\mathsf{choice}(\delta, \alpha, \mathtt{sid}, \mathtt{sid}', 0)); \mathtt{insert}\ \mathsf{Tlabel}_\delta(\mathtt{sid}, \mathtt{sid}', 0); \tau_\delta(P)$
  $+\ \mathtt{event}(\mathsf{choice}(\delta, \alpha, \mathtt{sid}, \mathtt{sid}', 1)); \mathtt{insert}\ \mathsf{Tlabel}_\delta(\mathtt{sid}, \mathtt{sid}', 1); \tau_\delta(Q))$
- $\tau_\delta(P +_{1/2} Q) = \tau_\delta(P) + \tau_\delta(Q)$
- $\tau_\delta(\mathtt{insert}\ t(u).P) = \mathtt{insert}\ t_\delta(u); \tau_\delta(P)$
- $\tau_\delta(\mathtt{get}\ t(x)\ \mathtt{suchthat}\ \psi(x)\ \mathtt{in}\ P\ \mathtt{else}\ Q) = \mathtt{get}\ t_\delta(x)\ \mathtt{suchthat}\ \psi(x)\ \mathtt{in}\ \tau_\delta(P)\ \mathtt{else}\ \tau_\delta(Q)$
- $\tau_\delta(\mathtt{event}(\mathtt{happy}(\mathtt{sid}, u)).P) = \mathtt{get}\ \mathsf{Tlabel}_\delta(= \mathtt{sid}, \mathtt{sid}', \beta)\ \mathtt{in}\ \mathtt{happy}^\beta(\mathtt{sid}', u); \tau_\delta(P)\ \mathtt{else}\ 0$
- $\tau_\delta(\mathtt{event}(E(x_1, \ldots, x_k)).P) = \mathtt{event}(E(x_1, \ldots, x_k)); \tau_\delta(P)$ if $E \neq \mathtt{happy}$
- $\tau_\delta(P \mid Q) = \tau_\delta(P) \mid \tau_\delta(Q)$
- $\ldots$ (the transformation is just propagated)

We also consider several restrictions:

$$r_{\mathsf{sid-injective}} = \quad \mathsf{choice}(\delta, \alpha, \mathtt{sid}, \tilde{\mathtt{sid}}, X) \;\wedge \mathsf{choice}(\delta, \alpha', \mathtt{sid}', \tilde{\mathtt{sid}}, Y) \Rightarrow \mathtt{sid} = \mathtt{sid}' \wedge \alpha = \alpha'$$

$$r_{\mathsf{rcv}} = \quad \mathtt{received}(1, 0, \alpha, x_1) \wedge \mathtt{received}(2, 0, \alpha, x_2) \Rightarrow x_1 = x_2$$

$$\begin{aligned} r_{\mathsf{one-fork}} = \quad & \mathsf{choice}(\delta_1, \alpha, \mathtt{sid}, \mathtt{sid}', X) \\ \wedge \quad & \mathsf{choice}(\delta_2, \alpha, \mathtt{sid}, \mathtt{sid}'', Y) \\ \wedge \quad & \mathtt{sid}' \neq \mathtt{sid}^* \\ \wedge \quad & X \neq Y \\ \wedge \quad & \delta_1 \neq \delta_2 \\ \Rightarrow \quad & \mathtt{sid}' \neq \mathtt{sid}'' \wedge \mathtt{sid}'' \neq \mathtt{sid}^* \end{aligned}$$

Restriction $r_{\mathsf{sid-injective}}$ means that the attacker cannot rename two different labels identically. It reflects that in $\mathtt{sem}_1$, $\mathtt{sid}$ is renamed in $\mathtt{sid}_w$ for a certain w, so two different session identifiers $\mathtt{sid}$ and $\mathtt{sid}'$ cannot be renamed the same way.

Restriction r_{rcv} makes sure that before the attacker uses $\mathtt{sid}^*$, the messages that are input in the traces explored by τ_1 and τ_2 are the same. Note that once the attacker uses $\mathtt{sid}^*$, the table $\mathsf{Tglobal}$ will still contain 0 but will also contain 1. It will be up to the attacker to not restrict itself by picking 0 in the table while it could pick 1.

The last restriction, $r_{\mathsf{one-fork}}$, implies that for any probabilistic choice α associated with some $\mathtt{sid}$, for which the attacker did not choose $\mathtt{sid}^*$, if τ_1 and τ_2 take two different directions X and Y, then different labelings $\mathtt{sid}'$ and $\mathtt{sid}''$ have been used, since the prefix w must differ.

Conjecture 1. *Let $\mathcal{P}$ be a traceable protocol such that each probabilistic action and each input action is uniquely labeled. Let $(\mathcal{T}_f, \phi)$ be a simple property. If it holds that*

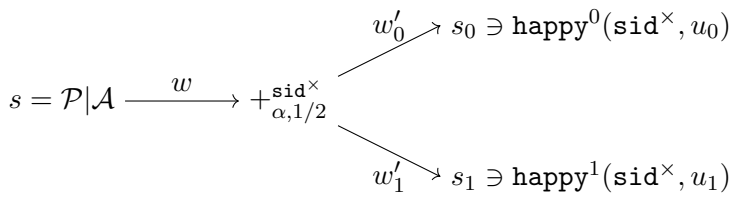
$$\tau_1(\mathcal{P}) \mid \tau_2(\mathcal{P}) \mid \mathtt{insert}\ \mathsf{Tglobal}(0) \models_{r_{\mathsf{sid-injective}}, r_{\mathsf{rcv}}, r_{\mathsf{one-fork}}} q_{\mathsf{det-security}},$$

then $\mathcal{P}$ is deterministically secure.

To show this result, we can, as well as for Proposition 8, proceed by contraposition. If a traceable protocol $\mathcal{P}$ is not deterministically secure, then there exists an attacker process $\mathcal{A}$, a resolution R, a session identifier $\mathtt{sid}^\times$, and two states s_0 and s_1 such that:

- $\mathsf{RProb}_R(s, s_0) > 0$ and $\mathsf{RProb}_R(s, s_1) > 0$,
- $\mathtt{happy}^0(\mathtt{sid}^\times, u_0) \in s_0$ and $\mathtt{happy}^1(\mathtt{sid}^\times, u_1) \in s_1$,
- $\sigma_{w_0}(\mathtt{sid}^\times) = \sigma_{w_1}(\mathtt{sid}^\times)$, with $\mathtt{label}(w_0) \in s_0$ and $\mathtt{label}(w_1) \in s_1$,
- $\phi(u_0)$ holds or $\phi(u_1)$ holds.

Then, we call w the longest common prefix to w_0 and to w_1. This means that the trace to s_0 and the trace to s_1 just split when executing a probabilistic choice after w. We pose $\mathtt{sid}^* = \sigma_w(\mathtt{sid}^\times)$, where $\mathtt{sid}^\star$ is the label of this probabilistic choice.



We translate every step of $s \to^* s_0$ in a step of a trace in $\to_\pi$ starting from $\tau_1(\mathcal{P})$, and proceed as well with $s \to^* s_1$. Combining those traces, we reach from $\tau_1(\mathcal{P}) \mid \tau_2(\mathcal{P}) \mid \mathtt{insert}\ \mathsf{Tglobal}(0)$ a state through a trace tr that contradicts $q_{\mathsf{det-security}}$. Before executing the probabilistic choice labeled by $\mathtt{sid}^\times$, $s \to^* s_0$ and $s \to^* s_1$ coincide, so $tr \vdash r_{\mathsf{rcv}}$. The renaming in $\mathtt{sem}_1$ is injective, so $tr \vdash r_{\mathsf{sid-injective}}$. Finally, if a probabilistic choice labeled by $\mathtt{sid}$ is taken after the $\mathtt{sid}^\times$ branching in $s \to^* s_0$ and $s \to^* s_1$, then the two labels w''_0 and w''_1 will be different: one contains $(\mathtt{sid}^\times, 0)$ while the other contains $(\mathtt{sid}^\times, 1)$. Thus, $\mathtt{sid}$ is renamed differently, so $tr \vdash r_{\mathsf{one-fork}}$.

8.4 Perspectives

8.4.1 Modeling short strings

Another source of non-negligible probabilities in protocols is the use of short strings, generally to ensure usability. For example, One-Time Password (OTP) is an authentication protocol

in which the client sends a 6-digit password to the server, that is the truncation of a regular cryptographic hash function. Similarly, in the SwissPost system, the verifiability is ensured through return codes that an attacker could guess. Even if they would only succeed with low probability, it is not impossible. In the formal analysis of the system [Pos24a], the possibility for the attacker to guess a return code is not considered. One possible way to take it into account is to treat short strings as random values. Then, when an agent needs to pick a short string, we can represent it through a probabilistic choice. For example, for a 4-digit string, instead of considering the process $P(x)$, where x is the short string, we consider the process $P(0000) +_{1/10000} \ldots +_{1/10000} P(9999)$. However, as we showed in Example 9, the attacker can adapt their strategy depending on the result of the probabilistic choices. This is not an issue for STROBE or BeleniosCaI, because the attacker has to commit on their choice before the probabilistic choice is made. It is not the case in OTP or SwissPost, and hence would require adapting either the model or the semantics.

8.4.2 Different flavours of deterministic security

In this section, we discuss the notion of deterministic security. We first recall its definition:

Definition 29. *Let $\mathcal{P}$ be a traceable protocol. Let $(\mathcal{T}_f, \phi)$ be a k-*simple *property.*

We say that $\mathcal{P}$ is p-deterministically secure if for all initial state $s \in \mathcal{S}^1_{\mathsf{init}}(\mathcal{P})$, for all resolution $R \in \mathcal{R}(N^1)$, for all label $\mathtt{sid} \in \mathcal{L}$, for all states $s_1, \ldots, s_p \in \mathcal{MP}$ and terms $M_1, \ldots, M_p \in \mathbb{T}$, if $\sigma_{w_1}(\mathtt{sid}) = \ldots = \sigma_{w_p}(\mathtt{sid})$, then we have:

$$\bigwedge_{j\in\{1,\ldots,p\}} \left(\mathsf{RProb}_R(s, s_j) > 0 \wedge \mathtt{happy}^j(\mathtt{sid}, M_j) \in s_j\right) \Rightarrow \bigwedge_{j\in\{1,\ldots,p\}} \neg\phi(M_j).$$

where w_j is the sequence such that $\mathtt{label}(w_j) \in s_j$.

Intuitivelly, it requires that if the user would be able to finish the protocol and reach a valid state without complaining in all possible branches of the probabilistic choice, then the attacker is not able to cheat. Conversly, it tolerates the attacker to be able to cheat without detection in all but one branches. This class of protocols is broad, but it gives a mediocre bound of the probability of winning for the attacker of $\frac{p-1}{p}$, with p the number of possible branches. For example, in the OTP protocol, the short code is a 6-digit number. Thus, the probability for the attacker to guess it is 1/100000. However, in our setting, the Theorem 1 provides only a bound of $1 - 1/100000$. Since the attacker can win in only one branch, instead of all but one branch, we can provide another notion of deterministic security:

Definition 30. *Let $\mathcal{P}$ be a traceable protocol. Let $(\mathcal{T}_f, \phi)$ be a k-*simple *property.*

We say that $\mathcal{P}$ is (m,p)-deterministically secure if for all initial state $s \in \mathcal{S}^1_{\mathsf{init}}(\mathcal{P})$, for all resolution $R \in \mathcal{R}(N^1)$, for all label $\mathtt{sid} \in \mathcal{L}$, for all states $s_1, \ldots, s_p \in \mathcal{MP}$ and terms $M_1, \ldots, M_p \in \mathbb{T}$, for all distinct indices $j_1, \ldots, j_m$, if $\sigma_{w_1}(\mathtt{sid}) = \ldots = \sigma_{w_p}(\mathtt{sid})$, then we have:

$$\bigwedge_{j\in\{j_1,\ldots,j_m\}} \left(\mathsf{RProb}_R(s, s_j) > 0 \wedge \mathtt{happy}^j(\mathtt{sid}, M_j) \in s_j\right) \Rightarrow \bigwedge_{j\in\{j_1,\ldots,j_m\}} \neg\phi(M_j).$$

where w_j is the sequence such that $\mathtt{label}(w_j) \in s_j$.

This notion of deterministic security would allow us to prove a better bound on the probability of winning of the attacker.

8.4.3 Limitation

A famous probabilistic protocol that we did not mention in this part so far is the *Benaloh challenge* [Ben07]. Similarly as the voting protocols of Section 8.1, the goal of this mechanism is to provide cast-as-intended verifiability without undermining receipt freeness. However, the main difference is that it is not a cast-and-audit mechanism anymore, but rather an audit-or-cast mechanism. We give on Figure 8.4 a description as automata of these two paradigms.

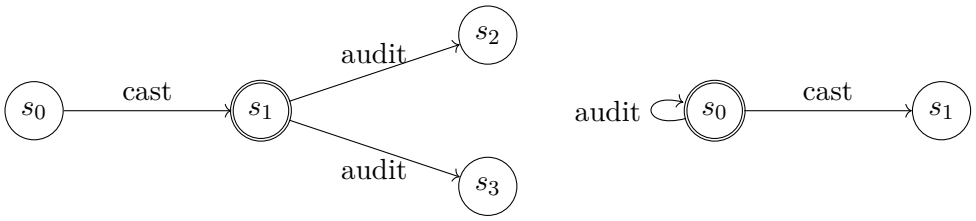


Figure 8.4: Cast-then-audit (left) and audit-or-cast (right) paradigms. Double circles represent probabilistic branchings.

In a cast-then-audit probabilistic voting protocol like BeleniosCaI, the voter gets a ballot, uses it to cast their vote, then makes a probabilistic choice, and finally performs a verification according to this choice. If we abuse the probabilistic processes grammar, it would look like $\texttt{CastThenAudit} = \texttt{in}(c, \mathit{ballot}); \texttt{Cast}; \big(\texttt{VerifL} +_{1/2} \texttt{VerifR}\big)$ In a audit-or-cast protocol, the voter first makes the probabilistic choice, then gets a ballot, and either casts it or audits it depending on the outcome of the choice. It would look like $\texttt{AuditOrCast} = (\texttt{in}(c, \mathit{ballot}); \texttt{Audit}; \texttt{AuditOrCast}) +_{1/2} (\texttt{in}(c, \mathit{ballot}); \texttt{Cast})$. This however does not fit in our language, because of the possible infinite branching.

A way to model this protocol would be to bound the number of times a voter can audit their ballot. This however does not lead to a fully satisfying representation. If, after having reached the bound, the voter gets a new ballot and always casts it, then the attacker can cheat without fear of being detected. On the other side, if the voter does not cast any ballot after having used all their audits„ then it deprives the attacker of a chance of winning.

Conclusion

During this thesis, we presented two contributions related to electronic voting. First, we designed and analysed a verifiable and private postal voting protocol. Then, we developed a method to analyse probabilistic protocols.

Postal voting. Few postal voting protocols exist in the literature. As preliminary work, we abstracted them and wrote a generic blueprint for any vote-by-mail scheme. Then, we found some attacks that apply to this blueprint and to protocols that do not implement dedicated countermeasures. This brought us to introduce new security properties that differ from those of electronic voting. These properties are weaker and more adapted to postal voting. Then, we designed a protocol, as simple as possible, that satisfies those properties. We conducted the analysis across all possible corruption scenarios using the ProVerif automated tool and its dedicated voting framework. One blind spot in our model is the short code used to avoid clash attacks. Since this code is short, an attacker could guess it. We wrote two distinct models: one where the adversary cannot guess the code at all, and another with no code at all. The impossibility of building a model where the attacker guessing the code is possible but unlikely brought us to our second contribution.

Probabilistic protocols Symbolic analysis is a classical approach for proving that a protocol satisfies a property. However, state-of-the-art tools suffer from several blind spots. In particular, they are not made to handle probabilities: neither protocols like STROBE, where the control flow is randomised, nor properties that hold with a certain probability, like cast-as-intended in BeleniosCaI, where the attacker can trick the voter with non-negligible probability, are within the scope of ProVerif or Tamarin. We started from the probabilistic protocols language of [CCK23] to develop an analysis method to prove probabilistic properties of those protocols. By restricting the class of properties we consider and under certain hypotheses on the protocol, we exhibited a deterministic sufficient condition for the probabilistic property to hold. We proposed a method relying on the ProVerif tool to verify those hypotheses by translating probabilistic processes into ProVerif processes. Then, we sketched a similar reasoning to prove deterministic security in the same way. A desirable next step would be to implement a compiler from the probabilistic syntax to the ProVerif language, to make the analysis more usable.

Perspectives. Postal voting is a topic related to electronic voting, with a few nuances. It is also the case for electronic signature collection for verifiable petitions, the topic of the contribution we did not present in this thesis. In both cases, we had to introduce novel definitions, security properties, and modeling components to account for the subtle differences between these domains and electronic voting. We applied the expertise, body of knowledge and methods of electronic voting to new problems. We believe that this approach could be useful in other

research fields, such as anonymous surveys, confidential feedback, auctions, or other unusual privacy-preserving protocols.

Bibliography

[AAOB+24] José Bacelar Almeida, Santiago Arranz Olmos, Manuel Barbosa, Gilles Barthe, François Dupressoir, Benjamin Grégoire, Vincent Laporte, Jean-Christophe Léchenet, Cameron Low, Tiago Oliveira, et al. Formally verifying Kyber: Episode V: Machine-checked IND-CCA security and correctness of ML-KEM in EasyCrypt. In *Annual International Cryptology Conference*, pages 384–421. Springer, 2024.

[ABD+18] David Adrian, Karthikeyan Bhargavan, Zakir Durumeric, Pierrick Gaudry, Matthew Green, J. Alex Halderman, Nadia Heninger, Drew Springall, Emmanuel Thomé, Luke Valenta, Benjamin VanderSloot, Eric Wustrow, Santiago Zanella-Béguelin, and Paul Zimmermann. Imperfect forward secrecy: how Diffie-Hellman fails in practice. *Commun. ACM*, 62(1):106114, December 2018.

[Adi08] Ben Adida. Helios: Web-based Open-Audit Voting. In *USENIX'08*, pages 335–348, 2008.

[AF01] Martin Abadi and Cédric Fournet. Mobile values, new names, and secure communication. In *POPL'01*, pages 104–115. ACM, 2001.

[BAF05] Bruno Blanchet, Martín Abadi, and Cédric Fournet. Automated verification of selected equivalences for security protocols. In *LICS'05*, pages 331–340. IEEE, 2005.

[BBK17] Karthikeyan Bhargavan, Bruno Blanchet, and Nadim Kobeissi. Verified models and reference implementations for the TLS 1.3 standard candidate. In *S&P'17*, pages 483–502, 2017.

[BBMP21] Sevdenur Baloglu, Sergiu Bursuc, Sjouke Mauw, and Jun Pang. Election Verifiability Revisited: Automated Security Proofs and Attacks on Helios and Belenios. In *34th IEEE Computer Security Foundations Symposium, (CSF 2021)*, pages 1–15, 2021.

[BCC22a] Bruno Blanchet, Vincent Cheval, and Véronique Cortier. ProVerif with lemmas, induction, fast subsumption, and much more. In *S&P'22*, pages 205–222. IEEE, 2022.

[BCC+22b] Mikaël Bougon, Hervé Chabanne, Véronique Cortier, Alexandre Debant, Emmanuelle Dottax, Jannik Dreier, Pierrick Gaudry, and Mathieu Turuani. Themis: an On-Site Voting System with Systematic Cast-as-intended Verification and Partial Accountability. In *CCS'22*. ACM, 2022.

[BCDS25] David Basin, Cas Cremers, Jannik Dreier, and Ralf Sasse. *Modeling and Analyzing Security Protocols with Tamarin*. Information Security and Cryptography. Springer, July 2025.

[BCG+15] David Bernhard, Veronique Cortier, David Galindo, Olivier Pereira, and Bogdan Warinschi. A comprehensive analysis of game-based ballot privacy definitions. In *Proceedings of the 36th IEEE Symposium on Security and Privacy (S&P'15)*, pages 499–516, San Jose, CA, USA, May 2015. IEEE Computer Society Press.

[BCL12] Gergei Bana and Hubert Comon-Lundh. Towards unconditional soundness: Computationally complete symbolic attacker. In Pierpaolo Degano and Joshua D. Guttman, editors, *Principles of Security and Trust*, pages 189–208, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.

[BDG+12] Gilles Barthe, François Dupressoir, Benjamin Grégoire, César Kunz, Benedikt Schmidt, and Pierre-Yves Strub. Easycrypt: A tutorial. *International School on Foundations of Security Analysis and Design*, pages 146–166, 2012.

[BDJ+24] David Baelde, Stéphanie Delaune, Charlie Jacomme, Adrien Koutsos, and Joseph Lallemand. The squirrel prover and its logic. *ACM SIGLOG News*, 11(2):62–83, 2024.

[BDS17] Alessandro Bruni, Eva Drewsen, and Carsten Schürmann. Towards a Mechanized Proof of Selene Receipt-Freeness and Vote-Privacy. In *E-Vote-ID'17*, pages 110–126. Springer, 2017.

[Ben07] Josh Benaloh. Ballot casting assurance via voter-initiated poll station auditing. EVT'07, page 14, USA, 2007. USENIX Association.

[Ben21] Josh Benaloh. Strobe-voting: Send two, receive one ballot encoding. In *E-Vote-ID'21*, pages 33–46. Springer, 2021.

[BG12] Stephanie Bayer and Jens Groth. Efficient Zero-Knowledge Argument for Correctness of a Shuffle. In David Pointcheval and Thomas Johansson, editors, *Advances in Cryptology – EUROCRYPT 2012*, pages 263–280, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.

[BKS26] David Baelde, Adrien Koutsos, and Justine Sauvage. Leveraging Cryptographic Simulator Synthesis for Formally Verifying the FOO E-Voting Protocol. In *Usenix 2026-35th Usenix security symposium*, 2026.

[Bla07] Bruno Blanchet. CryptoVerif: Computationally sound mechanized prover for cryptographic protocols. In *Dagstuhl seminar Formal Protocol Verification Applied*, volume 117, page 156, 2007.

[Bla16] Bruno Blanchet. Modeling and Verifying Security Protocols with the Applied Pi Calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1:1–135, 2016.

[BRT13] Josh Benaloh, Peter Y. A. Ryan, and Vanessa Teague. Verifiable postal voting. In *Security Protocols XXI*, pages 54–65. Springer, 2013.

[BS16] Bruno Blanchet and Ben Smyth. Automated Reasoning for Equivalences in the Applied Pi Calculus with Barriers. In *2016 IEEE 29th Computer Security Foundations Symposium (CSF)*, pages 310–324, 2016.

[BS23] Dan Boneh and Victor Shoup. *A Graduate Course in Applied Cryptography*. 2023.

[BSTP20] David A. Basin, Ralf Sasse, and Jorge Toro-Pozo. The EMV Standard: Break, Fix, Verify. *2021 IEEE Symposium on Security and Privacy (SP)*, pages 1766–1781, 2020.

[BT94] Josh Benaloh and Dwight Tuinstra. Receipt-free secret-ballot elections (extended abstract). In *Proceedings of the Twenty-Sixth Annual ACM Symposium on Theory of Computing*, STOC '94, page 544553, New York, NY, USA, 1994. Association for Computing Machinery.

[BY86] Josh Benaloh and Moti Yung. Distributing the power of a government to enhance the privacy of voters. In *Proceedings of the fifth annual ACM symposium on Principles of distributed computing*, pages 52–62, 1986.

[CCD+23] Vincent Cheval, Cas Cremers, Alexander Dax, Lucca Hirschi, Charlie Jacomme, and Steve Kremer. Hash gone bad: Automated discovery of protocol attacks that exploit hash function weaknesses. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 5899–5916, Anaheim, CA, August 2023. USENIX Association.

[CCK23] Vincent Cheval, Raphaëlle Crubillé, and Steve Kremer. Symbolic protocol verification with dice — process equivalences in the presence of probabilities. pages 1–38, 2023.

[CDC23] Véronique Cortier, Alexandre Debant, and Vincent Cheval. Election Verifiability with ProVerif. In *CSF'23*. IEEE, 2023.

[CDG22] Véronique Cortier, Alexandre Debant, and Pierrick Gaudry. A privacy attack on the Swiss Post e-voting system. In *Real World Crypto Symposium (RWC'22)*, Amsterdam, Netherlands, 2022. IACR.

[CDG25] Véronique Cortier, Alexandre Debant, and Pierrick Gaudry. Breaking verifiability and vote privacy in CHVote. In *Esorics 2025 - 30th European Symposium on Research in Computer Security*, Toulouse, France, 2025.

[CDGG23] Véronique Cortier, Alexandre Debant, Pierrick Gaudry, and Stéphane Glondu. Belenios with cast as intended. In *8th Workshop on Advances in Secure Electronic Voting (Voting'23)*, Bol, Croatia, May 2023.

[CDGL25] Véronique Cortier, Alexandre Debant, Pierrick Gaudry, and Léo Louistisserand. Vote&Check: Secure Postal Voting with Reduced Trust Assumptions. In *PoPETs'25*, 2025.

[CDS+18] Véronique Cortier, Constantin Catalin Dragan, Pierre-Yves Strub, Francois Dupressoir, and Bogdan Warinschi. Machine-checked proofs for electronic voting: privacy and verifiability for belenios. In *Proceedings of the 31st IEEE Computer Security Foundations Symposium (CSF'18)*, pages 298–312, 2018.

[CFS25] Margot Catinaud, Caroline Fontaine, and Guillaume Scerri. Proving e-voting mixnets in the CCSA model: zero-knowledge proofs and rewinding. 2025.

[CGG19] Véronique Cortier, Pierrick Gaudry, and Stéphane Glondu. *Belenios: A Simple Private and Verifiable Electronic Voting System*, pages 214–238. Springer International Publishing, 2019.

[CGGI14] Véronique Cortier, David Galindo, Stéphane Glondu, and Malika Izabachene. Election verifiability for Helios under weaker trust assumptions. In *Proceedings of the 19th European Symposium on Research in Computer Security (ESORICS'14)*, volume 8713 of *LNCS*, pages 327–344, Wroclaw, Poland, September 2014. Springer.

[CGGL24] Véronique Cortier, Pierrick Gaudry, Anselme Goetschmann, and Sophie Lemonnier. Belenios with cast-as-intended: towards a usable interface. In *Springer*, Terragona, Spain, October 2024. Springer.

[CGGR23] Véronique Cortier, Pierrick Gaudry, Stéphane Glondu, and Sylvain Ruhault. French 2022 legislative elections: a verifiability experiment. In *E-Vote-ID'23*, 2023.

[CGK+16] Véronique Cortier, David Galindo, Ralf Küsters, Johannes Müller, and Tomasz Truderung. SoK: Verifiability Notions for E-Voting Protocols. In *S&P'16*. IEEE, 2016.

[CGY24] Véronique Cortier, Pierrick Gaudry, and Quentin Yang. Is the JCJ voting system really coercion-resistant? In *37th IEEE Computer Security Foundations Symposium (CSF'24)*, Enschede, Netherlands, July 2024.

[Cha81] David L. Chaum. Untraceable electronic mail, return addresses, and digital pseudonyms. *Commun. ACM*, 24(2):8490, February 1981.

[Cha23] Swiss Federal Chancellery. Federal chancellery ordinance on electronic voting, 2023.

[CHH+17] Cas Cremers, Marko Horvat, Jonathan Hoyland, Sam Scott, and Thyla van der Merwe. A comprehensive symbolic analysis of tls 1.3. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, CCS '17, page 17731788, New York, NY, USA, 2017. Association for Computing Machinery.

[CHVW05] Lillie Coney, Joseph Lorenzo Hall, Poorvi L. Vora, and David A. Wagner. Towards a privacy measurement criterion for voting systems. volume 89 of *ACM International Conference Proceeding Series*, pages 287–288. Digital Government Research Center, 2005.

[CL18] Véronique Cortier and Joseph Lallemand. Voting: You can't have privacy without individual verifiability. In *25th ACM Conference on Computer and Communications Security (CCS'18)*, pages 53–66. ACM, 2018.

[CMFP+10] Benoît Chevallier-Mames, Pierre-Alain Fouque, David Pointcheval, Julien Stern, and Jacques Traoré. *On Some Incompatible Properties of Voting Schemes*, pages 191–199. Springer Berlin Heidelberg, Berlin, Heidelberg, 2010.

[CRH22] Braden L. Crimmins, Marshall Rhea, and J. Alex Halderman. RemoteVote and SAFE Vote: Towards Usable End-to-End Verification for Vote-by-Mail. In *FC'22*, pages 391–406. Springer, 2022.

[CS11] Véronique Cortier and Ben Smyth. Attacking and Fixing Helios: An Analysis of Ballot Secrecy. In *2011 IEEE 24th Computer Security Foundations Symposium*, pages 297–311, 2011.

[CSD+17] Véronique Cortier, Benedikt Schmidt, Constantin Catalin Dragan, Pierre-Yves Strub, Francois Dupressoir, and Bogdan Warinschi. Machine-checked proofs of privacy for electronic voting protocols. In *Proceedings of the 37th IEEE Symposium on Security and Privacy (S&P'17)*, pages 993–1008. IEEE Computer Society Press, 2017.

[dC85] Nicolas de Condorcet. Essai sur l'application de l'analyse à la probabilité des décisions rendues à la pluralité des voix. 1785.

[DDG+24] Constantin Cătălin Drăgan, François Dupressoir, Kristian Gjøsteen, Thomas Haines, Peter B. Rønne, and Morten Rotvold Solberg. Machine-Checked Proofs of Accountability: How to sElect Who is to Blame. In Gene Tsudik, Mauro Conti, Kaitai Liang, and Georgios Smaragdakis, editors, *Computer Security – ESORICS 2023*, pages 471–491, Cham, 2024. Springer Nature Switzerland.

[DGK+14] Jannik Dreier, Rosario Giustolisi, Ali Kassem, Pascal Lafourcade, Gabriele Lenzini, and Peter Y. A. Ryan. Formal Analysis of Electronic Exams. In *First Symposium on Digital Trust in Auvergne (SDTA'14)*, Clermont-Ferrand, France, December 2014.

[DH23] Alexandre Debant and Lucca Hirschi. Reversing, breaking, and fixing the French legislative election e-voting protocol. In *Usenix'23*, 2023.

[DJN03] Ivan Damgård, Mads Jurik, and Jesper Nielsen. A generalization of Paillier's public-key system with applications to electronic voting. *International Journal of Information Security*, 9:371–385, 04 2003.

[DKR07] Stéphanie Delaune, Steve Kremer, and Mark Ryan. Symbolic Bisimulation for the Applied Pi Calculus. In V. Arvind and Sanjiva Prasad, editors, *FSTTCS 2007: Foundations of Software Technology and Theoretical Computer Science*, pages 133–145, Berlin, Heidelberg, 2007. Springer Berlin Heidelberg.

[DLM24] Jannik Dreier, Pascal Lafourcade, and Dhekra Mahmoud. Shaken, not Stirred -Automated Discovery of Subtle Attacks on Protocols using Mix-Nets. In *Proceedings of the 33rd USENIX Conference on Security Symposium*, Philadelphia, United States, August 2024. USENIX Association.

[DPP+17] Jannik Dreier, Maxime Puys, Marie-Laure Potet, Pascal Lafourcade, and Jean-Louis Roch. Formally Verifying Flow Properties in Industrial Systems. In *Proceedings of the 14th International Joint Conference on e-Business and Telecommunications (ICETE 2017) - Volume 4: SECRYPT, Madrid, Spain, July 24-26, 2017.*, Proceedings of the 14th International Joint Conference on e-Business and Telecommunications (ICETE 2017) - Volume 4: SECRYPT, Madrid, Spain, July 24-26, 2017., pages 55–66, Madrid, Spain, July 2017.

[DPP24] Henri Devillez, Olivier Pereira, and Thomas Peters. Verifiable and private vote-by-mail. Cryptology ePrint Archive, Paper 2024/926, 2024. `https://eprint.iacr.org/2024/926`.

[DRS08] Stéphanie Delaune, Mark Ryan, and Ben Smyth. Automatic verification of privacy properties in the applied pi calculus. In Yücel Karabulut, John Mitchell, Peter Herrmann, and Christian Damsgaard Jensen, editors, *Trust Management II*, pages 263–278, Boston, MA, 2008. Springer US.

[DY83] D. Dolev and A. Yao. On the security of public key protocols. *IEEE Transactions on Information Theory*, 29(2):198–208, 1983.

[EHM23] Olivier Esseiva, Audhild Høgåsen, and Xavier Monnat. Improving the Swiss Post Voting System. In *E-Vote-ID 2023*, pages 10–18420. Gesellschaft für Informatik, Bonn, 2023.

[Elg85] Taher Elgamal. A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE Transactions on Information Theory*, 31(4):469–472, 1985.

[EMM06] Santiago Escobar, Catherine Meadows, and Jose Meseguer. A Rewriting-Based Inference System for the NRL Protocol Analyzer and its Meta-Logical Properties. *Theoretical Computer Science*, 367:162–202, 11 2006.

[HKLD17] Rolf Haenni, Reto E. Koenig, Philipp Locher, and Eric Dubuis. CHVote protocol specification. Cryptology ePrint Archive, Paper 2017/325, 2017.

[HLKD17] Rolf Haenni, Philipp Locher, Reto Koenig, and Eric Dubuis. Pseudo-code algorithms for verifiable re-encryption mix-nets. In Michael Brenner, Kurt Rohloff, Joseph Bonneau, Andrew Miller, Peter Y.A. Ryan, Vanessa Teague, Andrea Bracciali, Massimiliano Sala, Federico Pintore, and Markus Jakobsson, editors, *Financial Cryptography and Data Security*, pages 370–384, Cham, 2017. Springer International Publishing.

[HMVW17] Sven Heiberg, Tarvi Martens, Priit Vinkel, and Jan Willemson. Improving the Verifiability of the Estonian Internet Voting Scheme. In Robert Krimmer, Melanie Volkamer, Jordi Barrat, Josh Benaloh, Nicole Goodman, Peter Y. A. Ryan, and Vanessa Teague, editors, *Electronic Voting*, pages 92–107, Cham, 2017. Springer International Publishing.

[HS11] Rolf Haenni and Oliver Spycher. Secure internet voting on limited devices with anonymized DSA public keys. In *2011 Electronic Voting Technology Workshop/Workshop on Trustworthy Elections (EVT/WOTE 11)*, San Francisco, CA, August 2011. USENIX Association.

[HSB21] Lucca Hirschi, Lara Schmid, and David Basin. Fixing the Achilles Heel of E-Voting: The Bulletin Board. In *2021 IEEE 34th Computer Security Foundations Symposium (CSF)*, pages 1–17, 2021.

[Jam23] Wojciech Jamroga. Pretty Good Strategies for Benaloh Challenge. In Melanie Volkamer, David Duenas-Cid, Peter Rønne, Peter Y. A. Ryan, Jurlind Budurushi, Oksana Kulyk, Adrià Rodriguez Pérez, and Iuliia Spycher-Krivonosova, editors, *Electronic Voting*, pages 106–122, Cham, 2023. Springer Nature Switzerland.

[JCJ05] Ari Juels, Dario Catalano, and Markus Jakobsson. Coercion-resistant electronic elections. In *WPES'05*, pages 61–70. ACM, 2005.

[JKKR23] Charlie Jacomme, Elise Klein, Steve Kremer, and Maïwenn Racouchot. A comprehensive, formal and automated analysis of the EDHOC protocol. In *USENIX'23*, Anaheim, CA, United States, 2023.

[KBB17] Nadim Kobeissi, Karthikeyan Bhargavan, and Bruno Blanchet. Automated verification for secure messaging protocols and their implementations: A symbolic and computational approach. In *EuroS&P'17*, pages 435–450. IEEE, 2017.

[KM17] Ralf Küsters and Johannes Müller. Cryptographic security analysis of e-voting systems: Achievements, misconceptions, and limitations. In *International Joint Conference on Electronic Voting*, 2017.

[KMST16] Ralf Küsters, Johannes Müller, Enrico Scapin, and Tomasz Truderung. sElect: A Lightweight Verifiable Remote Voting System. In *CSF'16*. IEEE, 2016.

[KR05] Steve Kremer and Mark Ryan. Analysis of an Electronic Voting Protocol in the Applied Pi Calculus. In Mooly Sagiv, editor, *Programming Languages and Systems*, pages 186–200, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.

[KS19] Christian Killer and Burkhard Stiller. The Swiss Postal Voting Process and Its System and Security Analysis. In *E-Vote-ID'19*, pages 134–149. Springer, 2019.

[KTV10a] Ralf Küsters, Tomasz Truderung, and Andreas Vogt. Accountability: definition and relationship to verifiability. In *CCS'10*, pages 526–535. ACM, 2010.

[KTV10b] Ralf Küsters, Tomasz Truderung, and Andreas Vogt. A game-based definition of coercion-resistance and its applications. In *2010 23rd IEEE Computer Security Foundations Symposium*, pages 122–136, 2010.

[KTV12] Ralf Küsters, Tomasz Truderung, and Andreas Vogt. Clash attacks on the verifiability of e-voting systems. In *S&P'12*. IEEE, 2012.

[KZZ15] Aggelos Kiayias, Thomas Zacharias, and Bingsheng Zhang. End-to-End Verifiable Elections in the Standard Model. In *34th Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT'15)*, volume 9057 of *Lecture Notes in Computer Science*, pages 468–498, Sofia, Bulgaria, 2015. Springer.

[Low95] Gavin Lowe. An Attack on the Needham-Schroeder Public-Key Authentication Protocol. *Information Processing Letters*, 56(3):131–133, 1995.

[Low97] Gavin Lowe. A hierarchy of authentication specifications. In *Proceedings 10th Computer Security Foundations Workshop*, pages 31–43, 1997.

[LQT20] Wouter Lueks, Iñigo Querejeta-Azurmendi, and Carmela Troncoso. VoteAgain: A scalable coercion-resistant voting system. In *USENIX'20*, 2020.

[MBC+21] Eleanor McMurtry, Xavier Boyen, Chris Culnane, Kristian Gjøsteen, Thomas Haines, and Vanessa Teague. Towards verifiable remote voting with paper assurance. *CoRR*, abs/2111.04210, 2021.

[MKRV18] Karola Marky, Oksana Kulyk, Karen Renaud, and Melanie Volkamer. What Did I Really Vote For? On the Usability of Verifiable E-Voting Schemes. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, CHI '18, page 113, New York, NY, USA, 2018. Association for Computing Machinery.

[MPP24] Johannes Müller, Balázs Pejó, and Ivan Pryvalov. DeVoS: Deniable Yet Verifiable Vote Updating. In *Proceedings on Privacy Enhancing Technologies*, 2024.

[MSC03] Catherine Meadows, Paul Syverson, and Iliano Cervesato. Formal Specification and Analysis of the Group Domain of Intrepretation Protocol Using NPATRL and the NRL Protocol Analyzer. 07 2003.

[NS78] Roger M. Needham and Michael D. Schroeder. Using encryption for authentication in large networks of computers. *Commun. ACM*, 21(12):993999, December 1978.

[Pai99] Pascal Paillier. Public-Key Cryptosystems Based on Composite Degree Residuosity Classes. In Jacques Stern, editor, *Advances in Cryptology — EUROCRYPT '99*, pages 223–238, Berlin, Heidelberg, 1999. Springer Berlin Heidelberg.

[PIK94] Choonsik Park, Kazutomo Itoh, and Kaoru Kurosawa. Efficient Anonymous Channel and All/Nothing Election Scheme. In Tor Helleseth, editor, *Advances in Cryptology — EUROCRYPT '93*, pages 248–259, Berlin, Heidelberg, 1994. Springer Berlin Heidelberg.

[Pos24a] Swiss Post. Swiss post symbolic models - version 1.3. `https://gitlab.com/swisspost-evoting/e-voting/e-voting-documentation/-/tree/master/Symbolic-models`, 2024.

[Pos24b] Swiss Post. Swiss post voting system specification - version 1.4.1.1. `https://gitlab.com/swisspost-evoting/e-voting/e-voting-documentation/-/tree/master/System`, 2024.

[RBH+09] Peter Y. A. Ryan, David Bismark, James Heather, Steve A. Schneider, and Zhe Xia. Prêt à voter: a voter-verifiable voting system. *IEEE Trans. Inf. Forensics Secur.*, 2009.

[RRI16] Peter Y. A. Ryan, Peter Rønne, and Vincenzo Iovino. Selene: Voting with transparent verifiability and coercion-mitigation. In *VOTING'16*. Springer, 2016.

[RRR21] Peter Y. A. Ryan, Peter Rønne, and Simon Rastikian. Hyperion: An enhanced version of the Selene end-to-end verifiable voting scheme. In *E-Vote-ID'21*, 2021.

[RT09] Peter Y. A. Ryan and Vanessa Teague. Pretty Good Democracy. In *17th International Workshop on Security Protocols XVII*, pages 111–130. Springer, 2009.

[SFD+14] Drew Springall, Travis Finkenauer, Zakir Durumeric, Jason Kitcat, Harri Hursti, Margaret MacAlpine, and J. Halderman. Security Analysis of the Estonian Internet Voting System. *Proceedings of the ACM Conference on Computer and Communications Security*, pages 703–715, 11 2014.

[SHR23] Anggrio Sutopo, Thomas Haines, and Peter Rønne. On the Auditability of the Estonian IVXV System and an Attack on Individual Verifiability. In *Workshop on Advances in Secure Electronic Voting*, Bol, brac, Croatia, May 2023.

[Tea23] ODIHR Election Expert Team. Observation report on Estonia parliamentary elections, 2023.

[TW10] Björn Terelius and Douglas Wikström. Proofs of restricted shuffles. In Daniel J. Bernstein and Tanja Lange, editors, *Progress in Cryptology – AFRICACRYPT 2010*, pages 100–113, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg.

[WL20] Aiko Wagner and Josephine Lichteblau. Germany Going Postal? Comparing Postal and Election Day Voters in the 2017 German Federal Election. *German Politics*, 31:1–24, 05 2020.